

Plateforme de Gestion des Contrats de Maintenance

SolidWall Consulting

Rapport de Projet de Fin d'Études

Pour l'obtention du diplôme de

Licence Appliquée en Informatique

Réalisé par :

Rihab CHOUROU

Eya GHARBI

Encadré par :

Encadrant Académique : [Nom]

Encadrant Professionnel : [Nom]

Année Universitaire 2025-2026

Table des matières

Introduction générale	1
Webographie	2
Cadre général du projet	2
I Présentation de l'organisme d'accueil	4
1 Services de SolidWall Consulting	4
2 Organigramme de l'organisme	5
II Cadre du projet	5
1 Contexte du projet	5
2 Analyse de l'existant	5
2.1 Étude de l'existant	5
2.2 Critique de l'existant	10
2.3 Tableau comparative des solutions	11
2.4 Solution proposée	11
III Méthodologie adaptée	12
1 Méthodologie classique	12
1.1 Principes de la méthodologie classique	12
2 Méthodologie Agile	13
2.1 Les valeurs agiles	13
3 Choix de la méthode de travail	14
3.1 Choix de l'approche Agile	14
3.2 Les principales méthodes Agiles	14
3.3 Choix de la méthode Scrum	14
4 Fondements de SCRUM	15
5 Rôles de SCRUM	15
6 Evènements Scrum	16
7 Artéfacts Scrum	16
8 Cycle de vie	16
IV Langage de modélisation UML – Unified Modeling Language	17

Préparation du projet	18
I	Spécification des besoins 20
1	Identification des acteurs 20
2	Identification des besoins fonctionnels 21
2.1	Authentification et Gestion de Compte 21
2.2	Gestion des Utilisateurs et Demandes d’Inscription 22
2.3	Système de Rôles et Permissions 22
2.4	Gestion des Clients et Projets 23
2.5	Gestion des Contrats de Maintenance 23
2.6	Gestion des Demandes d’Intervention 24
2.7	Gestion Avancée des Tickets de Maintenance 24
2.8	Tableaux de Bord, Traçabilité et Documentation 25
3	Identification des besoins non fonctionnels 26
II	Modélisation des besoins 27
III	Pilotage du projet avec Scrum 28
1	Rôles de SCRUM 29
1.1	Backlog de produit 29
2	Planification des sprints 49
2.1	Sprint 1 – Gestion des Utilisateurs, Clients et Authentification . . . 50
2.2	Sprint 2 – Gestion des Projets et Contrats 51
2.3	Sprint 3 – Gestion des Demandes et Tickets 51
2.4	Sprint 4 – Collaboration, Documents et IA 51
3	Planning prévisionnel (Diagramme de Gantt) 52
IV	Environnement de Développement 53
1	Environnement matériel 53
2	Environnement logiciel 53
2.1	Langages de programmation et frameworks 53
2.2	Outils de développement et modélisation 54
2.3	Systèmes de gestion de bases de données 54
2.4	Conteneurisation et déploiement 55
2.5	Outil de gestion de projet Scrum 55
2.6	Environnement de composition de documents 56
2.7	Outil de design graphique 56
V	Architecture de l’application 56

1	Architecture physique	56
2	Architecture logique	57
VI	Conclusion	58
Gestion des Utilisateurs et Authentification		59
I	Sprint Planning Meeting	61
1	Backlog du premier sprint	61
II	Analyse	68
1	Diagramme des cas d'utilisation du premier sprint	68
2	Description textuelle de cas d'utilisation «S'authentifier»	69
3	Description textuelle de cas d'utilisation « Changer mot de passe »	70
4	Analyse de la fonctionnalité « Demander accès via OAuth »	71
4.1	Raffinement de cas d'utilisation « Demander accès via OAuth »	71
4.2	Description textuelle de cas d'utilisation « Soumettre demande d'inscription »	73
4.3	Description textuelle de cas d'utilisation « Valider demande »	74
5	Analyse de la fonctionnalité « Gérer utilisateurs »	74
5.1	Raffinement de cas d'utilisation « Gérer utilisateurs »	74
5.2	Description textuelle de cas d'utilisation « Consulter liste utilisateurs »	75
5.3	Description textuelle de cas d'utilisation « Ajouter utilisateur »	76
5.4	Description textuelle de cas d'utilisation « Désactiver un utilisateur »	76
6	Analyse de la fonctionnalité « Gérer profil »	76
6.1	Description textuelle de cas d'utilisation « Consulter Profil »	77
6.2	Description textuelle de cas d'utilisation « Modifier Profil »	77
7	Analyse de la fonctionnalité « Gérer rôles »	77
7.1	Raffinement de cas d'utilisation « Gérer rôles »	77
7.2	Description textuelle de cas d'utilisation « Archiver rôle »	78
7.3	Description textuelle de cas d'utilisation « Attribuer rôle à utilisateur »	79
7.4	Description textuelle de cas d'utilisation « Retirer rôle d'un utilisateur »	79
8	Analyse de la fonctionnalité « Gérer permissions »	79
8.1	Raffinement de cas d'utilisation « Gérer permissions »	80
8.2	Description textuelle de cas d'utilisation « Consulter liste des permissions »	80

8.3	Description textuelle de cas d'utilisation « Attribuer permission à rôle »	81
III	Diagramme de classes de première sprint	81
IV	Diagramme de séquence	81
1	Diagramme de séquences du cas d'utilisation « S'authentifier »	82
2	Diagramme de séquences du cas d'utilisation « Mot de passe oublié »	83
3	Diagramme de séquences du cas d'utilisation « Désactiver utilisateur »	84
4	Diagramme d'activité « Authentification et demande d'accès via OAuth »	85
V	Schéma relationnel	86
VI	Réalisation	86
1	Diagramme de composants	86
2	Description des interfaces	86
2.1	Interface de connexion	86
2.2	Interface de réinitialisation du mot de passe	87
2.3	Interface de gestion des utilisateurs	87
2.4	Interface de gestion des rôles et permissions	87
2.5	Interface de gestion des demandes d'inscription OAuth	87
VII	Tests et Validation	87
1	Stratégie de tests adoptée	88
1.1	Tests unitaires	88
1.2	Tests d'intégration	90
1.3	Tests End-to-End	93
	Réalisation	94
I	Sprint Planning Meeting	96
1	Objectif du Sprint	96
1.1	Backlog du deuxième sprint	96
II	Analyse	106
1	Diagramme des cas d'utilisation du deuxième sprint	106
2	Analyse de la fonctionnalité « Gérer clients »	106
2.1	Raffinement de cas d'utilisation « Gérer clients »	106
2.2	Description textuelle de cas d'utilisation « Ajouter client »	108
2.3	Description textuelle de cas d'utilisation « Consulter profil client »	109
2.4	Description textuelle de cas d'utilisation « Désactiver client »	110

3	Analyse de la fonctionnalité « Gérer Contrats »	110
3.1	Raffinement de cas d'utilisation « Gérer Contrats »	110
3.2	Description textuelle de cas d'utilisation « Créer contrat »	111
3.3	Description textuelle de cas d'utilisation « Archiver contrat »	111
3.4	Description textuelle de cas d'utilisation « Configurer intégration Slack »	112
4	Analyse de la fonctionnalité « Suivre contrats »	113
4.1	Description textuelle du cas d'utilisation « Consulter ses contrats »	113
4.2	Description textuelle du cas d'utilisation « Soumettre demande de résiliation »	113
5	Analyse de la fonctionnalité « Consulter journal d'activités »	114
5.1	Description textuelle du cas d'utilisation « Consulter journal d'acti- vités »	114
III	Diagramme de classes du deuxième sprint	114
IV	Diagramme de séquence	114
1	Diagramme de séquences du cas d'utilisation « Demande Résiliation »	115
2	Diagramme de séquences du cas d'utilisation « Résilier contrat »	116
V	Schéma relationnel	116
VI	Réalisation	117
1	Architecture technique	117
2	Interfaces utilisateur	117
VII	Tests et Validation	118
0.1	Tests unitaires	118
0.2	Tests d'intégration	120
0.3	Tests End-to-End	120
Chapitre 5		121
I	Analyse	127
1	Diagramme des cas d'utilisation du Sprint 3	127
2	Description textuelle des cas d'utilisation	127
II	Conception	128
1	Diagrammes de séquence	128
2	Modèle de données	128
III	Réalisation	129

1	Architecture technique	129
2	Interfaces utilisateur	129
IV	Tests	130
1	Tests unitaires	130
2	Tests d'intégration	130
Chapitre 6		130
I	Analyse	138
1	Diagramme des cas d'utilisation du Sprint 4	138
2	Description textuelle des cas d'utilisation	138
II	Conception	139
1	Diagrammes de séquence	139
2	Modèle de données	139
III	Réalisation	140
1	Architecture technique	140
2	Interfaces utilisateur	140
IV	Tests	141
1	Tests unitaires	141
2	Tests d'intégration	141

Introduction générale

Dans un environnement technologique en constante évolution, la maintenance des systèmes informatiques représente un enjeu stratégique majeur pour les entreprises de services numériques.

Les organisations doivent concilier la réactivité face aux incidents, le respect des accords de niveau de service SLA (*Service Level Agreement*) et la traçabilité des interventions.

SolidWall Consulting, acteur reconnu dans le domaine du développement web et mobile, n'échappe pas à cette réalité. En tant que prestataire de services numériques gérant un portefeuille croissant de clients et de contrats de maintenance, l'entreprise se trouve confrontée à des exigences de plus en plus élevées en matière de réactivité, de traçabilité et de respect des engagements de service. La maîtrise de ces enjeux constitue pour elle un facteur clé de compétitivité et de fidélisation de sa clientèle.

C'est dans ce contexte que s'inscrit notre projet de fin d'études, visant à concevoir et réaliser **SolidMaint**, une plateforme web intégrée de gestion des contrats de maintenance. L'objectif principal consiste à automatiser et centraliser l'ensemble du processus métier, depuis la soumission des demandes clients jusqu'à la résolution des incidents, tout en garantissant une traçabilité complète et le respect des engagements contractuels.

Ce présent rapport s'articule autour de six chapitres retraçant la démarche complète du projet :

Le **premier chapitre** établit le cadre général avec l'analyse de l'existant, l'étude comparative des solutions du marché, et la définition de la solution proposée.

Le **deuxième chapitre** détaille la spécification des besoins fonctionnels et non fonctionnels, l'identification des acteurs du système, la modélisation UML des cas d'utilisation, la planification Scrum avec constitution du Product Backlog, l'environnement de développement, et l'architecture technique de l'application.

Le **troisième chapitre** présente la réalisation du premier sprint : authentification sécurisée, système rôles et permissions, gestion des utilisateurs et des clients.

Les **chapitres suivants** couvrent les sprints restants : gestion des contrats, processus des demandes et tickets, fonctionnalités de collaboration et intelligence artificielle, accompagnés des tests et validations.

Webographie

1. **Atlassian**, Qu'est-ce que la méthodologie Agile? - <https://www.atlassian.com/fr/agile>
2. **Jira Software**, Guide utilisateur - <https://www.atlassian.com/software/jira>
3. **ManageEngine**, Documentation officielle - <https://www.manageengine.com>
4. **NestJS**, Documentation - <https://docs.nestjs.com>
5. **Next.js**, Documentation officielle - <https://nextjs.org/docs>
6. **OMG**, Why is UML important? - <https://www.omg.org/uml/why-uml-is-important.htm>
7. **React**, Guide officiel - <https://react.dev>
8. **SolidWall Consulting**, Site officiel - <https://solidwall.com.tn/>



CHAPITRE 1

Cadre général du projet



Sommaire

1

I	Présentation de l'organisme d'accueil	4
II	Cadre du projet	5
III	Méthodologie adaptée	12
IV	Langage de modélisation UML – Uni- fied Modeling Language	17

Introduction

Ce chapitre pose les fondations du projet en présentant le contexte organisationnel et technique de sa réalisation. Après la présentation de **SolidWall Consulting**, entreprise d'accueil du stage, nous analysons les processus actuels de gestion de maintenance et leurs limitations. L'étude comparative des solutions du marché permet d'identifier les axes d'amélioration et de définir les contours de **SolidMaint**. Enfin, nous exposons la méthodologie Scrum, le langage UML et la planification prévisionnelle du développement.

I Présentation de l'organisme d'accueil

«Solid Wall Consulting est une société de services informatiques, spécialisée dans le développement web, le développement mobile et l'intégration de solutions et d'infrastructures systèmes « Microsoft ». Forte d'une équipe dynamique et diversifiée et de profils reconnus, l'entreprise a pour objectif constant d'apporter une forte valeur ajoutée et de proposer des solutions modernes permettant aux entreprises de se démarquer sur leurs marchés respectifs. » [1] Fondée avec la vision d'accompagner les organisations publiques et privées vers l'excellence technologique et opérationnelle, Solid Wall Consulting s'est rapidement imposée comme un partenaire de confiance à l'international. » [2].



FIGURE I.1 – Logo SolidWall Consulting

1 Services de SolidWall Consulting

SolidWall Consulting propose des services conseil pour accompagner ses clients dans la réalisation de leurs projets et assure la qualité de ses prestations ainsi que le développement de solutions numériques.

L'approche agile de **SolidWall Consulting** garantit une gestion optimisée et par conséquent, d'un résultat de qualité.

2 Organigramme de l'organisme

Pour mettre en valeur la répartition des responsabilités et des fonctions au sein de l'organisation, nous présentons la structure hiérarchique de SolidWall Consulting avec ses principales divisions ainsi que les relations de subordination entre les différentes unités et postes, comme le montre la figure suivante.

II Cadre du projet

1 Contexte du projet

Dans le cadre de notre projet de fin d'études à l'**Institut Supérieur des Études Technologiques de Radès**, nous avons effectué un stage de quatre mois au sein de **SolidWall Consulting**, du 02 février au 31 mai 2026. Notre mission consiste à concevoir et à réaliser « **SolidMaint : une application web de gestion des contrats de maintenance** ».

Cette solution vise à automatiser et à centraliser l'ensemble du processus métier, garantissant ainsi une traçabilité complète des interventions techniques ainsi que le respect rigoureux des engagements contractuels.

2 Analyse de l'existant

Pour mieux comprendre les besoins, nous allons étudier les solutions existantes à deux niveaux : celles déjà en place au sein de l'entreprise **SolidWall Consulting**, puis celles utilisées dans le monde.

2.1 Étude de l'existant

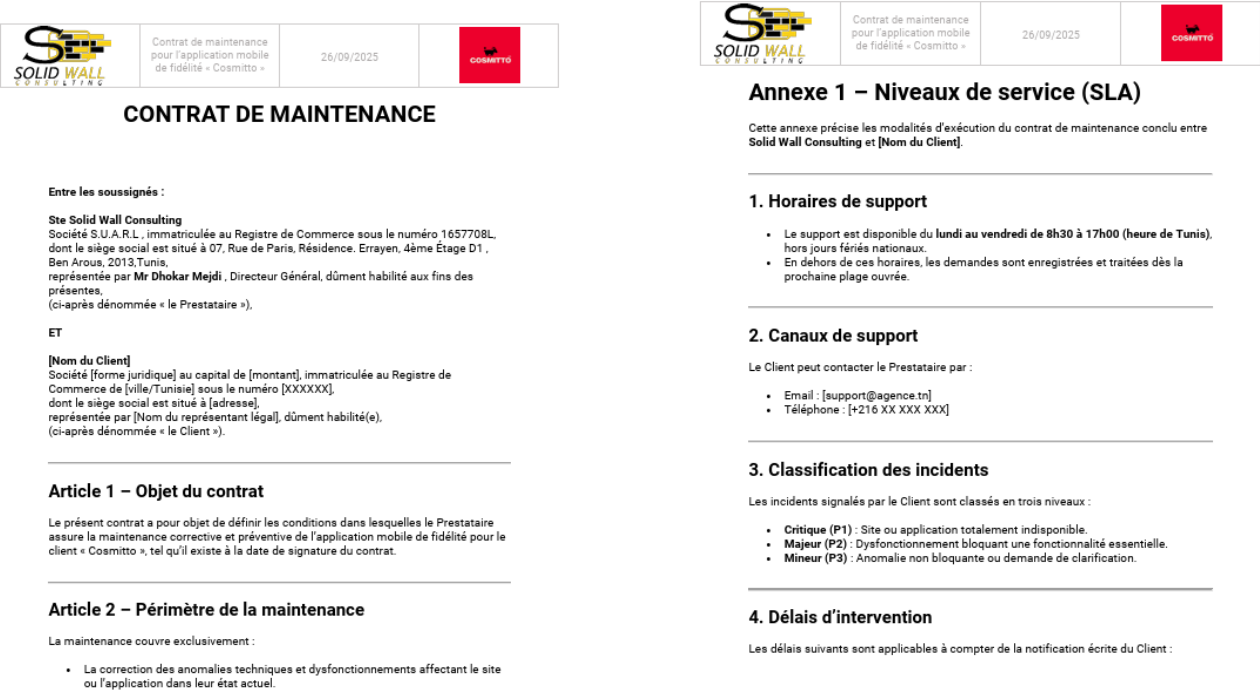
L'étude de l'existant constitue une étape fondamentale de notre démarche projet. Elle permet d'identifier les processus actuels, d'analyser leurs limitations et de définir les axes d'amélioration nécessaires pour concevoir une solution parfaitement adaptée aux besoins de SolidWall Consulting.

a) Étude de l'existant au niveau de l'entreprise

– Processus de gestion des contrats de maintenance

L'entreprise utilise actuellement des documents Word pour stocker et suivre les informations relatives aux contrats. Ces documents permettent aux responsables de formaliser la relation contractuelle avec les clients et de définir les engagements de chaque partie concernant les services de maintenance.

Afin de mieux illustrer ce mode de fonctionnement, la figure ci-dessous présente des exemples de gestion des contrats de maintenance dans le système actuel.



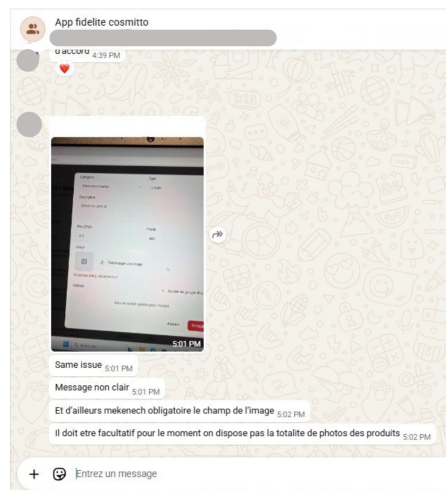
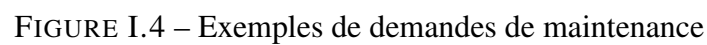


FIGURE I.3 – Demande de maintenance effectuée via l'application WhatsApp



- **Processus de suivi des tickets de maintenance**

Le suivi des interventions repose sur des fils de discussion par e-mail. L'affectation des tickets aux développeurs s'effectue par le responsable, généralement via un message direct.

Afin d'illustrer plus concrètement ce processus de gestion, un exemple de suivi des tickets de maintenance réalisé sous forme du fichier Excel est présenté ci-dessous.

The screenshot shows the 'Ratour net API (DEV)' application. The top menu bar includes 'Fichier', 'Accueil', 'Insertion', 'Partage', 'Mise en page', 'Formules', 'Données', 'Révision', 'Affichage', 'Aide', and 'Dessin'. The main workspace is divided into a table with columns: 'Contenu', 'Auteur', 'Commentaires', and 'Actions'. The table contains several rows of data, including 'Verification', 'Mise en bord', 'Quelque des commandes', and 'Préparation'. Each row has a corresponding 'Contenu' cell with detailed instructions and a 'Mise en bord' cell with a small image. The bottom of the interface shows a status bar with 'Feuille 1' and a zoom level of '100%'.

FIGURE I.5 – Exemple de suivi des interventions à l'aide d'un fichier Excel

- **Processus de traçabilité des actions et des documents**

Les documents contractuels sont stockés dans des répertoires locaux ou sur des espaces de stockage cloud. Les fichiers sont organisés selon les pratiques de chaque responsable, et les modifications sont effectuées directement sur les documents existants.

b) Étude de l'existant à l'échelle internationale

Avant de concevoir notre propre solution, nous allons étudier les solutions disponibles sur le marché afin d'analyser les fonctionnalités proposées. Cette démarche permet d'identifier les bonnes pratiques et d'orienter efficacement le développement de notre système.

Dans ce cadre, nous avons étudié plusieurs solutions existantes couvrant différents segments du marché : solutions open-source, solutions SaaS et solutions enterprise.

– GLPI (Gestion Libre de Parc Informatique)

Cette solution de gestion des services informatiques, développée en 2003 et maintenue par la société française Teclib, est open source, ce qui signifie que son code est librement accessible et peut être modifié et adapté selon les besoins des utilisateurs. Principalement utilisée par les équipes internes d'une organisation, elle peut également, selon sa configuration, recevoir des demandes externes. Elle assure un système de gestion de tickets structurant les demandes des utilisateurs. De plus, elle prend en charge différents canaux de communication, garantissant ainsi la centralisation des échanges. La version communautaire est gratuite, mais nécessite une formation de 2 à 3 jours pour les administrateurs. La version entreprise, avec support professionnel, coûte à partir de **1500€** par an.

ID	TITLE	ENTITY	PRIORITY	STATUS	REQUESTER - REQUESTER	ASSIGNED TO - TECHNICIAN	OPENING DATE	LAST UPDATE *	TIME TO RESOLVE - PROGRESS
11769	facilitate compelling users	Root * Quikowski-Rundstedt * Kihm LLC	High	Pending	Laron Changlin	Wava Brokke	09-09-2017 17:45	22-12-2021 16:37	
4647	synthesize next-generation schemas	Root * Quikowski-Rundstedt * Spencer, Hahn and Murzak	Low	New	Calle Farrell	Jaylon Hartmann	11-08-2015 15:26	14-12-2021 21:12	
13429	facilitate strategic partnerships	Root * Murray and Sons * Dach, Kemmer and Conners	Low	Processing (assigned)	Viviane Rogahn		15-03-2018 16:30	08-12-2021 16:24	
17459	transform customized synergies	Root * Cronin, Turcotte and Conners * Schneider, O'Connor and Lebsack	Low	Pending	Felton Reichert Anjali Abshire Maurine Yundt		11-07-2019 16:48	06-12-2021 14:59	
17460	strategize customized applications	Root * Cronin, Turcotte and Conners * Schneider, O'Connor and Lebsack	Low	Pending	Anjali Abshire Maurine Yundt	William McDermott	11-07-2019 17:24	06-12-2021 14:59	
17905	optimize cross-platform portals	Root * Cronin, Turcotte and Conners * Schneider, O'Connor and Lebsack	Low	Pending	Maurine Yundt	William McDermott	16-09-2019 18:00	06-12-2021 14:59	
18425	repurpose granular mindshare	Root * Adams, Ortiz and Anderson	Medium	Processing (assigned)	Manley Boyer		08-11-2019 16:25	06-12-2021 14:59	
18427	optimize cross-media e-commerce	Root * Adams, Ortiz and Anderson	Medium	Processing (assigned)	Manley Boyer		08-11-2019 17:10	06-12-2021 14:59	
18605	integrate strategic schemas	Root * Skiles, Crooks and Ziemecki * Hartmann, White and Pfister	Medium	Pending	Lisette Strawn		29-11-2019 09:52	06-12-2021 14:59	

FIGURE I.6 – Interface de GLPI

– Freshdesk

Cette solution cloud de service desk, développée en 2011 par la société indienne Freshworks, est une solution SaaS hébergée dans le cloud et accessible via navigateur web. **Freshdesk** se caractérise par sa facilité de déploiement et une maintenance entièrement assurée par le fournisseur.

Elle propose une interface intuitive pour la gestion des tickets avec support multicanal (courriel, chat, téléphone, réseaux sociaux). **Freshdesk** intègre des fonctionnalités d'automatisation avancées et d'intelligence artificielle pour la catégorisation automatique des tickets.

La solution propose un plan gratuit limité, tandis que les offres payantes démarrent à partir de **15€ par agent et par mois**

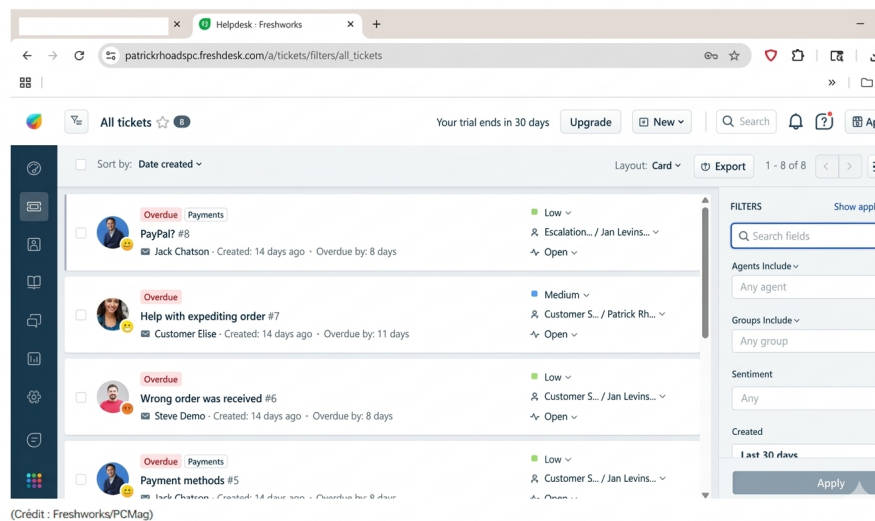


FIGURE I.7 – Interface de Freshdesk

– Jira Service Management (Atlassian)

Développée en 2002 et maintenue par Atlassian, entreprise australienne leader dans les outils de gestion de projet, cette solution entreprise s'intègre parfaitement à l'écosystème Atlassian (Jira Software, Confluence). Elle propose des fonctionnalités avancées de gestion des incidents, des changements et des actifs, ainsi que des capacités d'IA pour l'automatisation et la prédiction. Les tarifs démarrent à **20€/agent/mois**, mais augmentent considérablement avec les modules avancés. La maîtrise de l'écosystème Atlassian est nécessaire pour exploiter pleinement ses capacités.

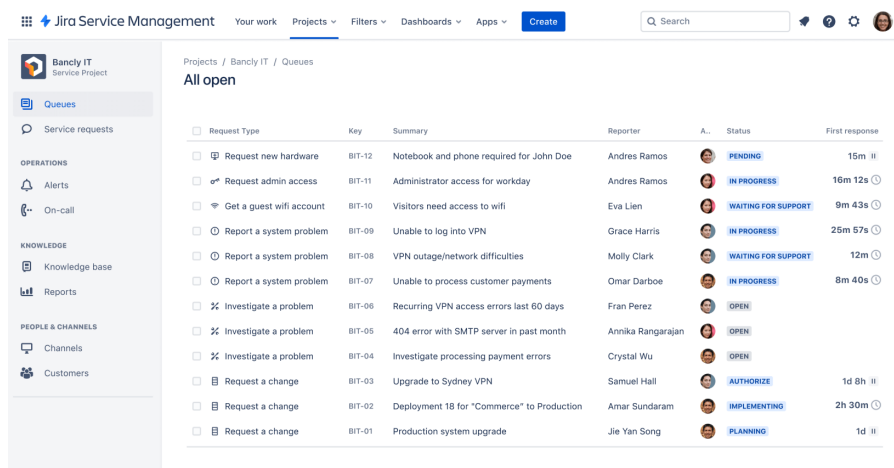


FIGURE I.8 – Interface de Jira Service Management

2.2 Critique de l'existant

Actuellement, la gestion des contrats de maintenance au sein de SolidWall Consulting se fait par échange de courriels. Cette approche présente certaines limites.

Limitations du processus actuel

- **processus fragmenté** : La communication via e-mail fragmente la gestion. Les informations étant dispersées, le suivi des composants manquants devient complexe. En l'absence de structure centralisée, il devient difficile de retrouver l'historique complet d'une intervention.
- **Absence de traçabilité** : L'absence de système de traçabilité rend le suivi des interventions difficile. Cela peut entraîner une perte de visibilité sur leur état, en cas de conflit avec un client ou lors d'un audit interne, ce qui représente un risque pour la crédibilité et la fiabilité de la société.
- **Risque de perte d'informations** : En raison de la fragmentation des processus, il existe un risque accru de perte d'informations importantes. Les documents sont stockés de manière non structurée, ce qui rend leur suivi et leur consolidation difficiles.
- **Complexité de la communication** : la communication au sein de l'équipe et avec les clients repose sur des échanges dispersés (courriels, messages), ce qui rend le flux des demandes peu clair et entraîne des retards dans le traitement des interventions.

Limitations des solutions existantes

Complexité d'utilisation et courbe d'apprentissage

Les solutions existantes présentent une complexité d'usage importante. GLPI nécessite une formation de 2 à 3 jours pour les administrateurs avec une interface peu intuitive. Jira Service Management requiert une connaissance approfondie de l'écosystème Atlassian, créant une dépendance technique majeure.

Coût des licences et investissement financier

L'investissement financier représente un obstacle majeur. Les tarifs s'échelonnent de 15€/agent/mois pour Freshdesk à 24€/utilisateur/mois pour Helpdesk, tandis que Jira Service Management coûte 20€/agent/mois. GLPI, bien que gratuit en version communautaire, nécessite 1500€/an minimum pour le support entreprise. Pour une équipe de 10 utilisateurs, le coût annuel varie de 1500€ à 12000€.

Dépendance aux extensions et problèmes d'intégration

Les solutions du marché souffrent d'une architecture modulaire générant des coûts cachés. De nombreuses fonctionnalités avancées nécessitent des extensions payantes supplémentaires. Les mises à jour engendrent des problèmes de compatibilité, et l'intégration avec des systèmes existants nécessite souvent des développements spécifiques onéreux.

2.3 Tableau comparative des solutions

Ces limitations se reflètent clairement dans l'analyse comparative des principales solutions du marché. Le tableau suivant synthétise les forces et faiblesses de chaque solution selon des critères fonctionnels, techniques et financiers :

Critères	GLPI	Jira SM	Freshdesk
Gestion tickets	✓	✓	✓
Gestion contrats	Limité	Limité	×
Messagerie intégrée	×	Limité	✓
IA/Suggestions	×	✓	✓
Facilité d'utilisation	Moyenne	Moyenne	Facile
Coût (10 users/an)	1500€	2000€	1800€

TABLE I.1 – Tableau comparatif des solutions de gestion de maintenance existantes

Note sur les coûts : Les tarifs indiqués correspondent au coût total annuel pour une équipe de 10 utilisateurs, calculés selon les grilles tarifaires officielles de chaque éditeur. Ces montants incluent uniquement les fonctionnalités de base et peuvent augmenter significativement avec l'ajout de modules complémentaires, de support premium ou de services de personnalisation.

2.4 Solution proposée

C'est dans ce cadre que s'inscrit **SolidMaint**, une plateforme web intégrée visant à automatiser et à centraliser l'ensemble du processus de gestion des contrats de maintenance au sein de **SolidWall Consulting**. Elle s'articule autour de trois modules fonctionnels complémentaires :

Module 1 : Gestion des Clients, Projets et Contrats

Ce module centralise la gestion des clients, des projets et des contrats de maintenance. Il offre une visibilité complète sur les engagements contractuels par client et intègre un système d'alertes préventives permettant d'anticiper les échéances et de garantir le respect des accords de niveau de service (SLA).

Module 2 : Gestion des Demandes et Tickets

Ce module structure le cycle de vie complet des demandes clients, depuis leur soumission jusqu'à leur clôture. Il automatise la génération des tickets, facilite leur affectation aux développeurs compétents et assure un suivi précis du temps d'intervention, contribuant ainsi à une meilleure maîtrise des délais de résolution.

Module 3 : Collaboration et Assistance Intelligente

Ce module regroupe les fonctionnalités avancées de communication et d'intelligence artificielle, notamment la messagerie en temps réel, la gestion documentaire et les notifications multicanaux, afin de fluidifier les échanges entre les différents acteurs de la plateforme.

En réponse aux insuffisances identifiées, **SolidMaint** centralisera l'ensemble des processus de maintenance au sein d'un environnement unifié, fiable et évolutif, permettant à **SolidWall Consulting** d'améliorer sa réactivité et de renforcer la qualité de ses prestations.

III Méthodologie adaptée

Le choix méthodologique est déterminant pour la réussite d'un projet de développement. Une méthodologie appropriée structure le travail, optimise la gestion des ressources et garantit la qualité du livrable dans le respect des délais impartis.

1 Méthodologie classique

La méthodologie classique de la gestion de projet s'articule autour d'une série d'étapes consécutives, où l'approbation de chaque phase est essentielle pour progresser vers la suivante. Les projets se déroulent selon des phases distinctes et structurées, représentées par différents modèles de cycle de vie : cascade, V ou spirale.

Cette approche convient particulièrement aux projets dont les besoins sont stables et exhaustivement définis dès le départ. Elle offre une planification détaillée facilitant le suivi des délais et des coûts. Cependant, sa rigidité constitue une limitation majeure face aux changements de besoins en cours de développement, rendant difficile l'intégration de retours clients durant le processus.

Afin de mieux comprendre le fonctionnement de cette approche, il convient de présenter ses caractéristiques fondamentales.

1.1 Principes de la méthodologie classique

Cette approche repose sur une organisation linéaire où chaque phase doit être validée avant progression. Elle exige une définition exhaustive des besoins en phase initiale et une planification servant de base à la gestion. Le contrôle et la validation s'effectuent de manière progressive tout au long du cycle de vie.

Dans ce contexte, la méthodologie Agile a été adoptée comme une alternative à cette approche.

2 Méthodologie Agile

« La méthodologie Agile décompose le travail en phases, mettant l'accent sur la livraison continue et l'amélioration. Agile profite aux équipes en permettant une planification adaptative, une exécution rapide et une évaluation continue, garantissant des résultats plus réactifs et plus réussis. » [3]

La méthodologie Agile est adoptée pour répondre rapidement aux retours des clients. Elle valorise la collaboration plutôt que la négociation contractuelle, en proposant d'abord une version minimale, puis en intégrant des fonctionnalités supplémentaires. L'approche Agile est une démarche moderne qui offre souplesse et performance dans la gestion de projet.

Afin de mieux comprendre les fondements de cette approche, il est important de présenter ses valeurs essentielles.

2.1 Les valeurs agiles

Les valeurs de la méthodologie Agile constituent des principes fondamentaux qui guident l'organisation et le fonctionnement des projets agiles. Elles permettent d'améliorer la collaboration, la qualité du travail et la réactivité de l'équipe face aux changements.

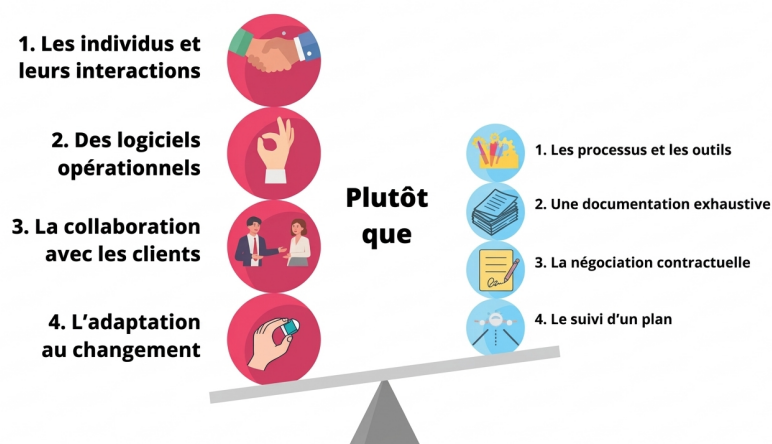


FIGURE I.9 – les valeurs agiles

Cette figure présente les quatre valeurs principales de la méthode Agile qui mettent l'accent sur :

- Les individus et leurs interactions plus que les processus et les outils
- Des logiciels opérationnels plus qu'une documentation exhaustive
- La collaboration avec les clients plus que la négociation contractuelle
- L'adaptation au changement plus que le suivi d'un plan

À partir de ces valeurs, plusieurs méthodes Agile ont été développées et peuvent être comparées selon différents critères.

3 Choix de la méthode de travail

Après avoir présenté les différentes approches et méthodes de gestion de projet, il convient de justifier les choix adoptés dans le cadre de ce travail.

3.1 Choix de l'approche Agile

Le domaine de la maintenance impose des besoins variables et imprévisibles, rendant impossible une planification exhaustive. Cette réalité justifie l'adoption d'une méthodologie agile privilégiant la flexibilité et la collaboration continue avec les clients. Parmi les méthodes disponibles, Scrum offre le cadre itératif le plus adapté à notre contexte.

3.2 Les principales méthodes Agiles

Le tableau suivant met en évidence les méthodes Agile les plus couramment utilisées, selon différents critères de comparaison.

Critères	Scrum	Kanban	Extreme Programming (XP)
Type de projet	Incrémental	Continu	Technique
Critère d'évaluation	Vélocité	Fluidité	Qualité
Gestion des risques	Détection	Visualisation	Prévention
Implication client	Ponctuelle	Variable	Très forte et continue

TABLE I.2 – Les principales méthodes Agiles.

3.3 Choix de la méthode Scrum

Scrum a été retenue comme cadre de travail pour structurer le développement en itérations courtes tout en s'adaptant aux changements fréquents. Cette méthode offre plusieurs avantages décisifs : – Collaboration continue entre l'équipe et les parties prenantes

- Adaptation rapide aux changements grâce aux cycles itératifs
- Livraisons régulières de fonctionnalités opérationnelles
- Détection précoce des problèmes et ajustements immédiats
- Transparence totale sur l'avancement du projet



FIGURE I.10 – les piliers de la méthode Scrum

4 Fondements de SCRUM

Scrum est une méthode agile structurant le développement en sprints, cycles courts de 2 à 4 semaines. Les besoins sont exprimés sous forme de user stories décrivant les fonctionnalités attendues et leur priorité. Cette approche vise à maximiser la productivité tout en assurant la satisfaction client par des livraisons régulières.

La méthode repose sur trois piliers fondamentaux : la transparence garantit la visibilité sur l'avancement, l'inspection permet d'évaluer régulièrement le produit et le processus, et l'adaptation autorise les ajustements nécessaires en fonction des retours.

5 Rôles de SCRUM

L'équipe Scrum est auto-organisée et prend les décisions sur la manière dont le travail sera réalisé afin d'atteindre l'objectif du sprint. Elle est composée d'un propriétaire de produit, d'un Scrum Master et d'une équipe de développement :

Le **Product Owner** est l'expert du domaine métier du projet qui porte la vision du produit à réaliser. Ce rôle repose sur la représentation de la voix du client et veille à ce que le produit reste aligné sur la vision stratégique de l'organisation.

Le **Scrum Master** a pour objectif d'assurer la productivité de l'équipe dans l'application de Scrum. Sa collaboration avec l'équipe vise à maximiser la performance globale, en alignant les efforts sur les objectifs stratégiques.

L' **équipe de développement** est constituée des personnes chargées de la réalisation du sprint, ayant

pour mission de transformer les besoins exprimés en fonctionnalités utilisables du produit.

6 Evènements Scrum

Ce sont des rendez-vous clés qui rythment le projet Scrum, créent une routine de transparence et d'inspection, et permettent à l'équipe de s'adapter pour livrer la meilleure valeur possible.

- **Sprint** : Un sprint est une itération de développement d'une durée fixe (généralement **2 à 4 semaines**) durant laquelle une version potentiellement livrable du produit est réalisée.
- **Sprint Planning** : Réunion de planification du sprint où l'équipe définit les objectifs et sélectionne les éléments du Product Backlog à réaliser pendant le sprint.
- **Daily Scrum** : Réunion quotidienne de **15 minutes** permettant à l'équipe de synchroniser ses actions, d'identifier les obstacles et d'ajuster le plan de travail.
- **Revue de Sprint** : Réunion à la fin du sprint pour présenter le travail accompli, recueillir les retours et adapter le Product Backlog si nécessaire.
- **Sprint Rétrospective** : Réunion d'amélioration continue où l'équipe analyse le déroulement du sprint, identifie ce qui a bien fonctionné et ce qui doit être amélioré.

7 Artéfacts Scrum

Ce sont les supports d'information qui matérialisent le travail, la valeur et l'avancement du projet, garantissant une transparence totale entre l'équipe et les parties prenantes.

- **Product Backlog** : Liste priorisée de toutes les fonctionnalités, évolutions, corrections et tâches à réaliser pour le produit.
- **Sprint Backlog** : Sous-ensemble du Product Backlog sélectionné pour le sprint en cours, accompagné d'un plan pour livrer l'incrément.
- **Incrément** : Somme de tous les éléments du Product Backlog terminés durant un sprint et les sprints précédents, représentant une version utilisable du produit.

8 Cycle de vie

Le cycle de vie Scrum se compose de sprints successifs, chacun comprenant une planification, le développement, des réunions quotidiennes, une revue et une rétrospective. Ce processus itératif permet d'adapter le produit en continu aux besoins du client et d'améliorer l'équipe à chaque itération.

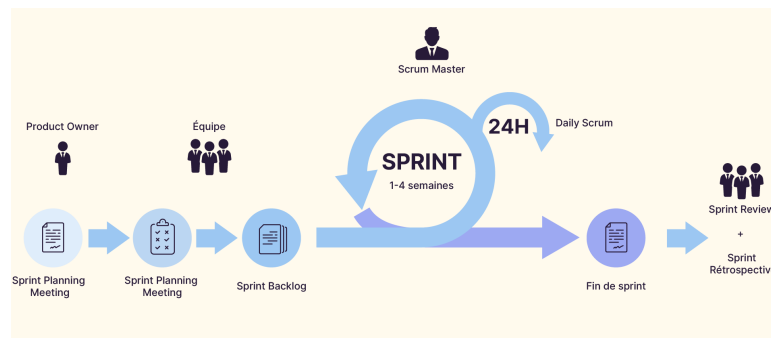


FIGURE I.11 – Cycle de vie Scrum

IV Langage de modélisation UML – Unified Modeling Language

« Le langage de modélisation unifié (UML) est un langage permettant de spécifier, visualiser, construire et documenter les artefacts des systèmes logiciels, ainsi que de modéliser des systèmes métier et d'autres systèmes non logiciels. L'UML représente un ensemble de bonnes pratiques d'ingénierie qui ont fait leurs preuves dans la modélisation de systèmes vastes et complexes. » [4]



FIGURE I.12 – Logo UML

Pour concevoir notre application, nous avons choisi UML comme langage de modélisation. Ce choix s'appuie sur plusieurs points forts tels que la communication entre l'équipe et la cohérence architecturale qui se fait de manière simplifiée et standardisée.

Conclusion

Ce chapitre a établi le cadre général de notre projet **SolidMaint** en présentant l'organisme d'accueil SolidWall Consulting et en analysant les enjeux de la gestion des contrats de maintenance. L'étude critique de l'existant a révélé des limitations significatives dans les processus actuels, tandis que l'analyse comparative des solutions du marché a confirmé la pertinence de développer une solution sur mesure. La solution **SolidMaint** proposée répond à ces défis par une approche intégrée centralisant

l'ensemble du cycle de vie des contrats. L'adoption de la méthodologie Scrum et la planification en quatre sprints garantissent une approche itérative et collaborative pour la réalisation du projet. Le chapitre suivant détaillera la spécification des besoins et la modélisation de la solution.

Références

- [1] SolidWall Consulting. Site officiel. Disponible sur : <https://solidwall.com.tn/> (Consulté le 21/04/2026)
- [2] SolidWall Consulting. Publication LinkedIn PFE 2025-2026. Disponible sur : https://www.linkedin.com/posts/solid-wall-consulting_pfe-book-2025-2026-activity-7402287392465862656-MR2f (Consulté le 21/04/2026)
- [3] Atlassian. Qu'est-ce que la méthodologie Agile ? Disponible sur : <https://www.atlassian.com/fr/agile> (Consulté le 21/04/2026)
- [4] OMG. Why is UML important ? Disponible sur : <https://www.omg.org/uml/why-uml-is-important.htm> (Consulté le 21/04/2026)
- [5] Miro. Qu'est-ce qu'un diagramme de Gantt ? Disponible sur : <https://miro.com/fr/planification-strategique/qu-est-ce-qu-un-diagramme-de-gantt/> (Consulté le 21/04/2026)
- [6] Openfire. (s.d.). Contrat de maintenance : comment simplifier la gestion de vos contrats d'entretien annuels ? Disponible sur : <http://openfire.fr/blog/toutes-actualites-3/contrat-de-maintenance-comment-simplifier-la-gestion-de-vos-contrats-dentretien-annuels-74> (Consulté le 21/04/2026)
- [7] Freshworks. Freshdesk - Customer Support Software. Disponible sur : <https://www.freshworks.com/freshdesk/> (Consulté le 21/04/2026)
- [9] Atlassian. Jira Service Management. Disponible sur : <https://www.atlassian.com/software/jira/service-management> (Consulté le 21/04/2026)
- [11] AXELOS. ITIL 4 Foundation - IT Service Management. Disponible sur : <https://www.axelos.com/certifications/itil-service-management> (Consulté le 21/04/2026)



CHAPITRE 2

Préparation du projet



Sommaire

2

I	Spécification des besoins	20
II	Modélisation des besoins	27
III	Pilotage du projet avec Scrum	28
IV	Environnement de Développement	53
V	Architecture de l'application	56
VI	Conclusion	58

Introduction

Ce chapitre présente l'analyse et la spécification des besoins de notre plateforme **SolidMaint**. Nous identifierons les acteurs du système et leurs besoins fonctionnels et non fonctionnels, puis procéderons à la modélisation UML. Nous détaillerons ensuite l'approche de pilotage Scrum avec la planification des sprints, et présenterons l'environnement de développement ainsi que l'architecture technique de l'application.

I Spécification des besoins

L'étape de spécification des besoins est importante pour prouver qu'un besoin réel existe, elle permet de définir explicitement les objectifs à atteindre.

Pour cela nous allons identifier les acteurs, leurs besoins fonctionnels et les besoins non fonctionnels.

1 Identification des acteurs

Un acteur représente une personne, une organisation ou un autre système qui interagit avec le système. Notre projet identifie six acteurs principaux :

Acteur	Rôle
Visiteur	Soumet une demande de création de compte via OAuth (Google, GitHub) et attend la validation par un administrateur.
Utilisateur	Accède aux fonctionnalités communes : authentification, gestion du profil, notifications multicanaux et historique d'activités.
Client	Soumet des demandes de maintenance, consulte l'état d'avancement de ses tickets, accède à ses contrats et communique avec l'équipe projet via la messagerie intégrée.
Développeur	Traite les tickets assignés, gère le chronomètre de travail, signale les blocages, propose des solutions et consulte la base de connaissances.
Chef de projet	Valide les demandes clients, assigne les tickets aux développeurs, supervise le respect des SLA, gère les équipes et génère les procès-verbaux.
Administrateur	Gère les utilisateurs, rôles, permissions, clients, projets et contrats. Configure les SLA, supervise les tableaux de bord et administre l'ensemble du système.

TABLE II.1 – Acteurs du système SolidMaint et leurs rôles

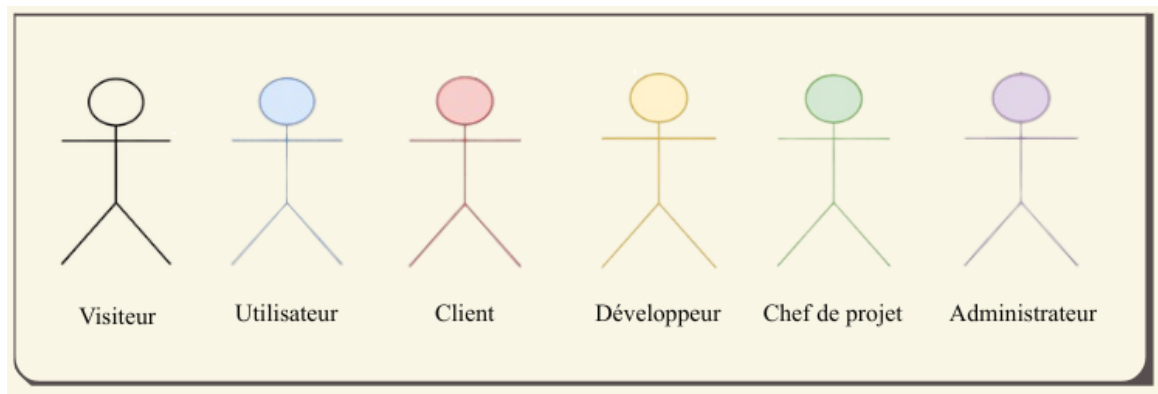


FIGURE II.1 – Illustration des acteurs du système SolidMaint

2 Identification des besoins fonctionnels

L'identification des besoins fonctionnels constitue une étape fondamentale dans la conception de **SolidMaint**. Ces spécifications définissent précisément les opérations que la plateforme doit être capable d'exécuter pour répondre aux exigences métier de SolidWall Consulting.

2.1 Authentification et Gestion de Compte

Ce module assure l'authentification sécurisée via identifiants classiques ou OAuth 2.0 (Google, GitHub) avec gestion automatique des jetons d'accès. L'innovation distinctive réside dans l'obligation de changement de mot de passe lors de la première connexion pour les comptes créés par un administrateur. Un processus de réinitialisation obligatoire bloque l'accès à l'application jusqu'à la définition d'un mot de passe personnel, garantissant que seul l'utilisateur légitime détient les informations d'authentification définitives.

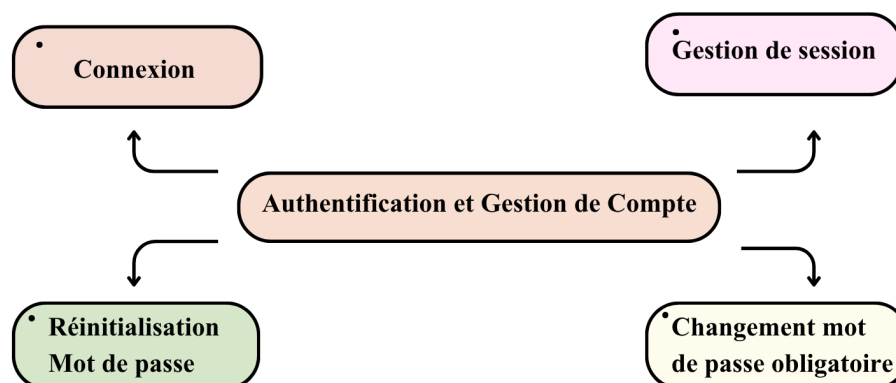


FIGURE II.2 – Authentification et Gestion de Compte

2.2 Gestion des Utilisateurs et Demandes d'Inscription

Ce module permet aux administrateurs de gérer le cycle de vie complet des comptes utilisateurs avec création automatisée, gestion des profils et contrôle des activations/désactivations. L'innovation principale réside dans le mécanisme de recherche et filtrage multicritères avancé qui permet de localiser rapidement un utilisateur spécifique selon des critères combinés (nom, rôle, statut, date de création). Le traitement des demandes d'inscription s'effectue via une interface centralisée permettant approbation, rejet ou mise en attente avec notifications automatiques.

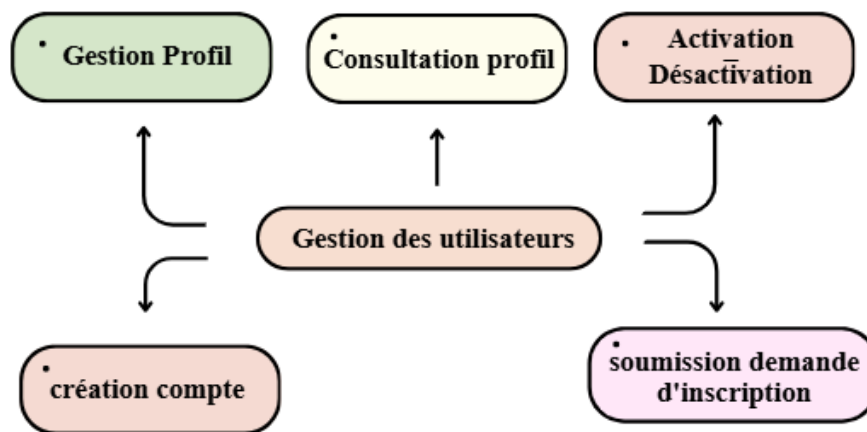


FIGURE II.3 – Gestion des utilisateurs

2.3 Système de Rôles et Permissions

Le contrôle d'accès s'appuie sur un modèle RBAC (Role-Based Access Control) offrant une granularité fine dans la gestion des autorisations. L'innovation majeure réside dans la conception modulaire permettant aux administrateurs de créer des rôles personnalisés en combinant dynamiquement les permissions existantes selon les besoins organisationnels spécifiques. Cette flexibilité permet d'ajuster les autorisations d'un rôle sans impact sur les autres configurations, facilitant l'évolution du système de sécurité.

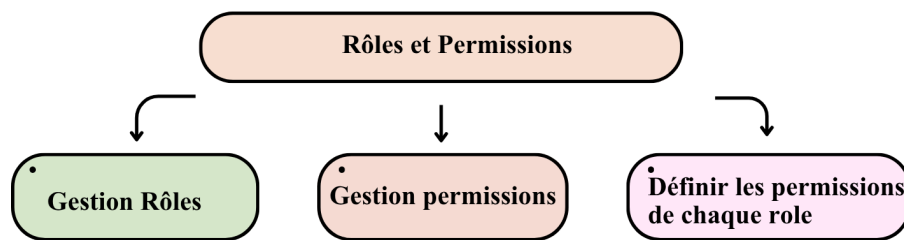


FIGURE II.4 – Rôles et permissions

2.4 Gestion des Clients et Projets

Ce module centralise la gestion des clients avec création automatisée des comptes et récupération automatique des logos d'entreprise depuis les ressources en ligne. L'aspect distinctif est la génération automatique d'organigrammes visuels et interactifs pour chaque projet, présentant la hiérarchie complète (client, chef de projet, équipe) avec accès direct aux coordonnées de tous les interlocuteurs. Cette visualisation facilite l'identification rapide des contacts et optimise la communication en cas de problème, tout en maintenant la traçabilité via des notes confidentielles horodatées.

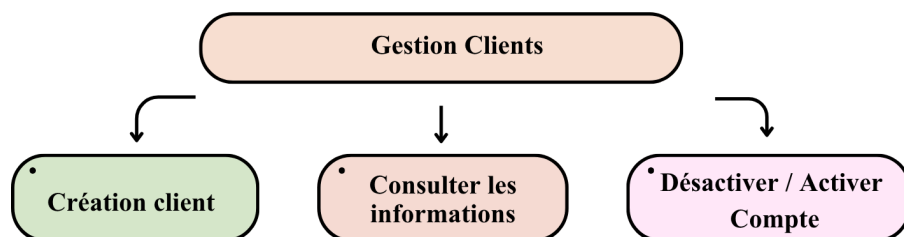


FIGURE II.5 – Gestion des Clients

2.5 Gestion des Contrats de Maintenance

Ce module automatise le cycle de vie complet des contrats de maintenance avec système d'alertes préventives échelonnées (30, 15, 7 jours avant expiration). L'innovation majeure réside dans l'import intelligent à partir de fichiers PDF, utilisant l'extraction automatique de données pour pré-remplir les formulaires de création. Un dispositif de détection de doublons basé sur le calcul d'empreintes cryptographiques garantit l'unicité des contrats et prévient toute duplication accidentelle lors des téléversements.

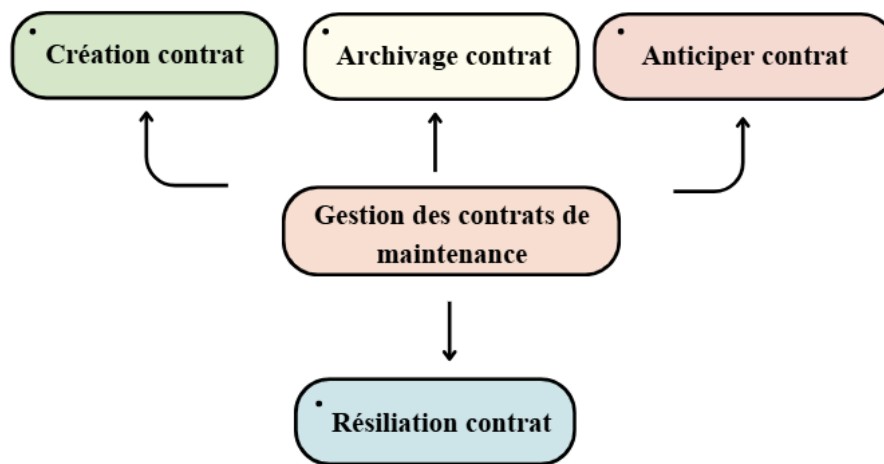


FIGURE II.6 – Gestion des contrats de maintenance

2.6 Gestion des Demandes d’Intervention

Ce module structure la soumission des demandes d’intervention avec vérification automatique de couverture contractuelle et validation des types d’intervention éligibles. L’aspect distinctif impose une contrainte temporelle intelligente de vingt-quatre heures entre deux demandes de même type pour un projet identique, prévenant la surcharge des équipes de support. Le workflow de validation intègre un système de relance automatique garantissant le traitement dans les délais impartis avec escalade vers les responsables en cas de dépassement des seuils temporels.

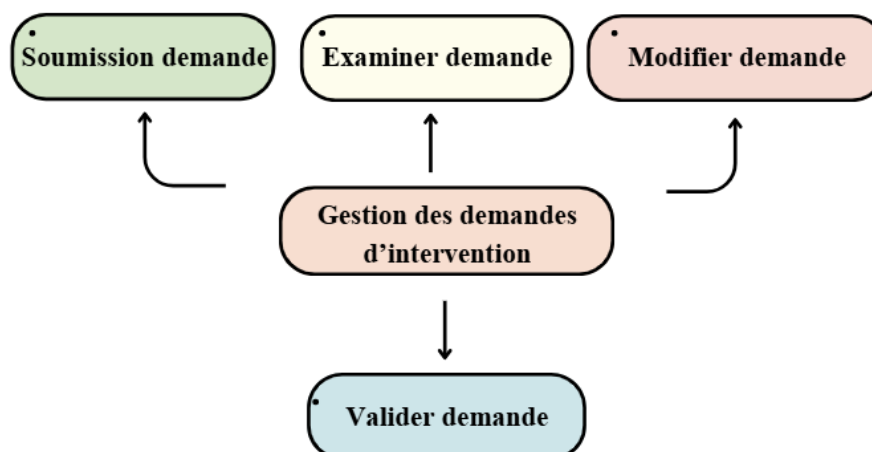


FIGURE II.7 – Gestion des demandes d’intervention

2.7 Gestion Avancée des Tickets de Maintenance

Ce module implémente un workflow structuré avec assignation intelligente limitée à cinq tickets actifs par développeur pour optimiser la charge de travail. L’innovation majeure est le mécanisme de rattachement de tickets complexes permettant de lier plusieurs interventions traitant du même contexte ou de problèmes dérivés. Les tickets rattachés héritent automatiquement du contrat parent et

maintiennent un historique cohérent, facilitant la gestion des problèmes récurrents tout en préservant la traçabilité individuelle de chaque intervention.

2.8 Tableaux de Bord, Traçabilité et Documentation

Ce module offre des tableaux de bord personnalisés par rôle avec indicateurs clés de performance et visualisations analytiques en temps réel. Il maintient un historique exhaustif de toutes les opérations avec traçabilité complète (utilisateur, action, valeurs avant/après, horodatage) et système de notifications multicanaux (web, email, Slack). L'innovation majeure réside dans la génération automatique de procès-verbaux PDF à partir des informations de tickets (temps passé, description, solution), s'activant lors de la résolution pour produire automatiquement une documentation contractuelle structurée.

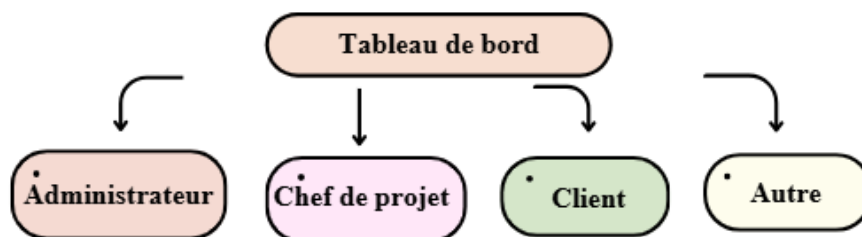


FIGURE II.8 – Tableau de bord et reportage

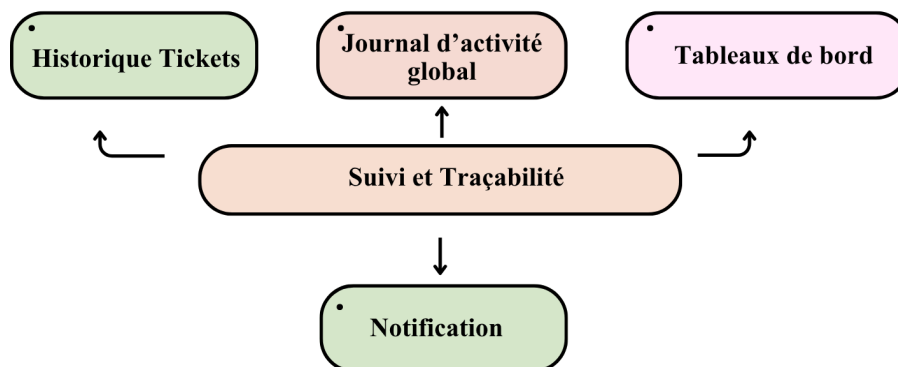


FIGURE II.9 – Suivi et Traçabilité

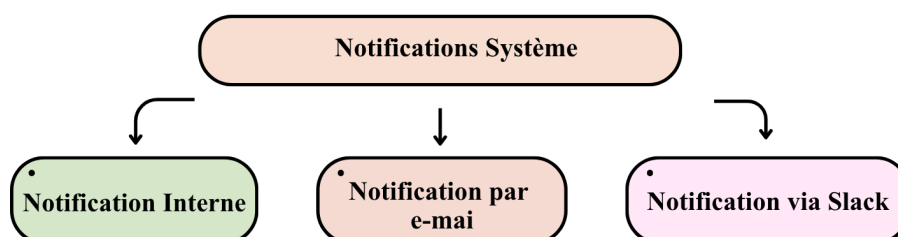


FIGURE II.10 – Notifications Système

3 Identification des besoins non fonctionnels

Durant la réalisation de ce projet, nous prenons en considération les besoins non fonctionnels qui désignent les attributs de qualité d'un système.

Les critères que notre plateforme doit satisfaire sont :

- **Ergonomie** : L'interface intégrera **shadcn/ui** et **Tailwind CSS** pour assurer une cohérence visuelle et une utilisabilité optimale. Un système de thématisation dynamique (mode clair/sombre) et une architecture responsive garantiront une adaptation automatique aux différentes résolutions d'écran. Les composants React permettront des mises à jour en temps réel sans rechargement, avec des transitions animées via **Framer Motion** et un système de retour d'information clair pour guider l'utilisateur.
- **Sécurité** : La sécurité intégrera des mesures robustes : hachage des mots de passe avec **bcrypt** (facteur 10), authentification **JWT** avec rotation des tokens (access token 15 minutes, refresh token 7 jours) et **OAuth 2.0** (Google, GitHub). Un mécanisme de verrouillage de compte est activé après trois tentatives d'authentification échouées. Les protections incluront la validation des entrées via **class-validator**, les requêtes paramétrées via **Prisma ORM** contre les injections SQL, et la vérification du type MIME des fichiers téléversés. La journalisation complète des actions via le module **ActivityLog** et le contrôle d'accès basé sur les rôles (**RBAC**) granulaire renforcent la traçabilité et l'intégrité du système.
- **Performance** : L'architecture modulaire permettra une évolution horizontale avec scalabilité selon la croissance des utilisateurs et du volume de données. Un cache en mémoire (**Redis**) avec une durée de vie de 60 secondes est utilisé pour les permissions et les données fréquemment consultées. L'optimisation de la base de données s'appuiera sur une stratégie d'indexation appropriée (identifiants, dates, statuts, combinaisons fréquentes) et l'utilisation de **Prisma ORM** pour générer des requêtes SQL optimisées. La compression **gzip** (niveau 6) est appliquée sur toutes les réponses HTTP supérieures à 1 Ko, réduisant la taille des payloads de 60 à 80%. Le chargement sélectif des relations via les mécanismes `select` et `include` évite les problèmes de requêtes N+1, garantissant des temps de réponse rapides.
- **Temps réel** : La communication en temps réel est assurée via le protocole **WebSocket** (Socket.IO 4), permettant des échanges bidirectionnels instantanés entre le serveur et les clients. Cette technologie supporte la messagerie instantanée entre l'équipe et les clients, la mise à jour en direct du tableau Kanban, ainsi que la diffusion des notifications. Des alertes proactives sont déclenchées automatiquement à 30, 15 et 7 jours avant l'expiration des contrats, via des tâches

planifiées (**Cron jobs**).

- **Maintenabilité** : L'architecture modulaire et l'adoption des principes SOLID avec les conventions **Next.js** et **NestJS** garantiront la lisibilité et la facilité de modification du code. La documentation technique est générée automatiquement via **Swagger/OpenAPI** et maintenue à jour lors de chaque modification. Les tests sont assurés par **Jest** pour le backend (tests unitaires et d'intégration), **Vitest** pour le frontend (tests unitaires des composants et hooks), et **Playwright** pour les tests end-to-end, exécutés automatiquement avant chaque déploiement.

II Modélisation des besoins

L'analyse des besoins fonctionnels se poursuit par une modélisation dynamique à l'aide du diagramme de cas d'utilisation UML. Ce diagramme offre une vision synthétique des fonctionnalités offertes par le système du point de vue de chaque acteur, sans se préoccuper de leur ordre d'exécution ou des détails techniques.

La figure II.11 présente le diagramme de cas d'utilisation global de notre plateforme de service desk. Ce diagramme structure les interactions entre les six acteurs identifiés et le système. Chaque acteur possède un périmètre d'actions bien défini.

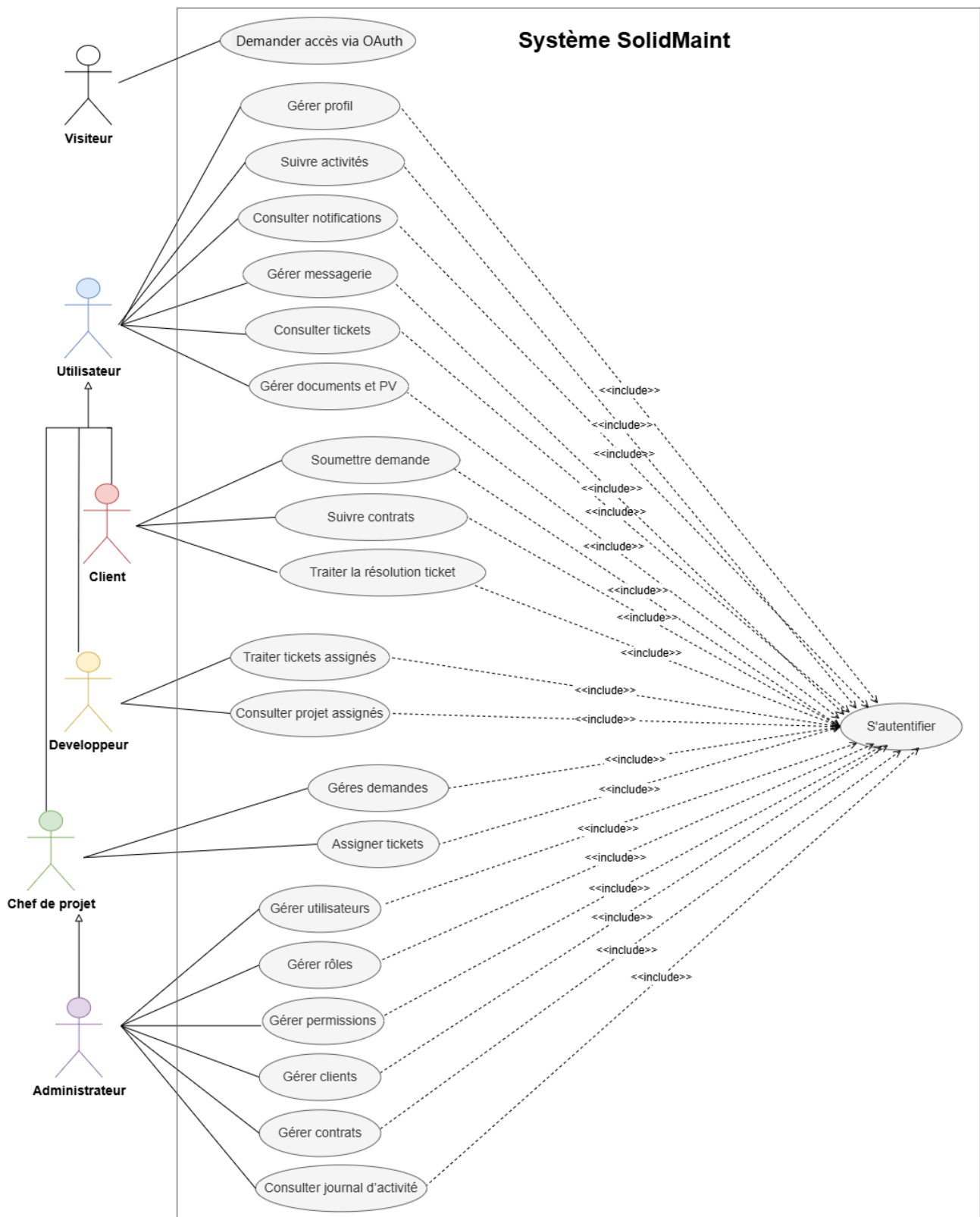


FIGURE II.11 – Diagramme de cas d'utilisation global du système

III Pilotage du projet avec Scrum

1 Rôles de SCRUM

Dans notre projet, Monsieur Mejdi Dhokar sera le Product Owner, Madame Maïssa Ben Fredj sera le Scrum Master, et nous, Rihab Chourou et Eya Gharbi, formons l'équipe de développement.



FIGURE II.12 – Équipe Scrum et rôles

1.1 Backlog de produit

Comme évoqué dans le premier chapitre, le Product Backlog constitue l'un des artefacts fondamentaux de la méthodologie Scrum. Il permet de formaliser les exigences fonctionnelles ainsi que les fonctionnalités attendues du système. Le tableau ci-dessous présente le Product Backlog de notre solution **SolidMaint**, structuré selon les éléments suivants :

- **Id EPIC** : Identifiant unique attribué à chaque EPIC afin de faciliter son identification et sa gestion dans le système de projet.
- **EPIC** : Ensemble de tâches interconnectées visant à atteindre un objectif majeur dans la gestion agile de projet ou le développement logiciel.
- **Id User Story** : Valeur unique et incrémentale associée à chaque user story pour assurer sa traçabilité.
- **User Story** : Description de la fonctionnalité du point de vue de l'utilisateur final, suivant le format "En tant que [rôle], je souhaite [action] afin de [bénéfice]".
- **Critères d'acceptation** : Conditions spécifiques et mesurables qui doivent être remplies pour que la user story soit considérée comme terminée et acceptée par le Product Owner.
- **Priorité** : Niveau d'importance défini par le Product Owner pour chaque tâche, permettant de prioriser les développements.

Légende des priorités : + : Faible ++ : Moyenne +++ : Élevée.

ID Epic	Épic	ID US	User Story	Critères d'acceptation	Priorité
1	Authentification et Sécurité	US01	En tant qu' utilisateur , je souhaite m'authentifier avec email et mot de passe afin d'accéder à mes informations de manière sécurisée.	L'utilisateur authentifié avec succès accède à son tableau de bord personnalisé selon son rôle.	+++
		US02	En tant qu' utilisateur , je souhaite réinitialiser mon mot de passe afin de retrouver l'accès à mon compte.	L'utilisateur reçoit un email avec un lien valide 24h et peut définir un nouveau mot de passe conforme aux règles de sécurité.	+++
		US03	En tant qu' utilisateur , je souhaite m'authentifier via Google ou GitHub afin de me connecter rapidement sans créer de compte.	L'utilisateur peut se connecter via son compte Google ou GitHub et accéder immédiatement à l'application.	++
		US04	En tant qu' utilisateur , je souhaite me déconnecter de la plateforme afin de sécuriser ma session.	Après déconnexion, l'utilisateur ne peut plus accéder aux pages protégées sans se reconnecter.	++
		US05	En tant qu' utilisateur , je souhaite que mon compte soit protégé contre les tentatives de connexion malveillantes afin de garantir la sécurité de mes données.	Après 3 tentatives échouées, le compte est temporairement verrouillé et l'utilisateur reçoit une notification par email.	+++
1	Gestion des utilisateurs	US06	En tant qu' utilisateur , je souhaite gérer mon profil afin que mes informations soient toujours à jour.	L'utilisateur peut modifier et sauvegarder ses informations personnelles qui sont validées avant enregistrement.	++

		US07	En tant qu' utilisateur , je souhaite consulter l'historique de mes activités afin de suivre mes actions dans le système.	L'utilisateur visualise toutes ses actions passées avec possibilité de filtrer par date et type d'action.	++
		US08	En tant qu' administrateur , je souhaite créer des utilisateurs afin de gérer les accès au système.	Le nouvel utilisateur reçoit un email d'invitation avec un mot de passe temporaire pour sa première connexion.	+++
		US09	En tant qu' administrateur , je souhaite modifier les informations d'un utilisateur afin de maintenir des données à jour.	Les modifications sont enregistrées avec traçabilité et l'utilisateur concerné est notifié des changements.	++
		US10	En tant qu' administrateur , je souhaite activer et désactiver un compte utilisateur afin de contrôler l'accès au système.	Un compte désactivé ne peut plus se connecter et l'utilisateur reçoit une notification du changement de statut.	+++
		US11	En tant qu' administrateur , je souhaite consulter la liste des utilisateurs afin de visualiser et gérer efficacement les comptes.	La liste complète des utilisateurs est affichée avec possibilité de filtrer, rechercher et exporter les données.	++
3	Gestion des rôles et permissions	US12	En tant qu' administrateur , je souhaite gérer des rôles afin de structurer les accès au système.	L'administrateur peut créer, modifier et archiver des rôles avec vérification des utilisateurs associés.	+++
		US13	En tant qu' administrateur , je souhaite attribuer des permissions granulaires aux rôles afin de contrôler précisément les accès.	L'administrateur sélectionne les permissions par catégorie et visualise un aperçu avant validation.	+++

		US14	En tant qu' administrateur , je souhaite consulter la matrice rôles-permissions afin de visualiser rapidement les droits de chaque rôle.	La matrice affiche clairement tous les rôles et leurs permissions avec possibilité de filtrer et exporter.	++
4	Gestion des demandes d'inscription OAuth	US15	En tant qu' utilisateur externe , je souhaite demander l'accès via OAuth afin de m'inscrire facilement.	La demande est enregistrée et l'administrateur reçoit une notification pour traitement.	++
		US16	En tant qu' administrateur , je souhaite valider ou rejeter les demandes d'inscription OAuth afin de contrôler les accès.	L'utilisateur reçoit un email de confirmation ou rejet et peut accéder à l'application si validé.	+++
5	Gestion des Clients	US17	En tant qu' administrateur , je souhaite créer un profil client afin de centraliser ses informations.	Le profil client est créé avec toutes les informations nécessaires et un identifiant unique est généré.	+++
		US18	En tant qu'administrateur, je souhaite consulter la liste des clients afin de visualiser les clients existants.	La liste complète des clients est accessible avec possibilité de filtrer, rechercher et exporter.	++
		US19	En tant qu' administrateur , je souhaite consulter le profil détaillé d'un client afin de visualiser toutes ses informations.	Toutes les informations du client sont affichées incluant contrats, projets et historique des interactions.	++
		US20	En tant qu' administrateur , je souhaite modifier les informations d'un client afin de maintenir des données à jour.	Les modifications sont enregistrées avec traçabilité et le client est notifié si ses coordonnées changent.	++

		US21	En tant qu' administrateur , je souhaite activer et désactiver un compte client afin de contrôler l'accès.	Un compte client désactivé ne peut plus accéder au système et reçoit une notification du changement.	+++
		US22	En tant qu' administrateur , je souhaite imprimer l'annuaire d'un client afin de disposer d'une version papier.	Un document PDF professionnel contenant toutes les informations du client est généré.	+
		US23	En tant qu' administrateur , je souhaite créer un projet rattaché à un client afin d'organiser les interventions.	Le projet est créé et rattaché au client avec un chef de projet assigné.	+++
		US24	En tant qu' administrateur , je souhaite consulter la liste des projets afin de piloter l'activité.	La liste complète des projets est affichée avec possibilité de filtrer par client, statut ou chef de projet.	++
		US25	En tant qu' administrateur , je souhaite affecter des développeurs à un projet afin de constituer l'équipe.	Les développeurs sont affectés au projet et peuvent visualiser les informations du projet.	+++
		US26	En tant que Développeur , je souhaite consulter les informations du projet afin de comprendre le contexte.	Toutes les informations du projet sont accessibles incluant client, objectifs, équipe et contrat.	++
5	Gestion des Contrats	US27	En tant qu' administrateur , je souhaite créer un contrat de maintenance afin de formaliser les engagements de service avec le client.	Le contrat est créé avec client, dates, type SLA, cycle facturation et configuration des alertes d'expiration.	+++

		US29	En tant qu' administrateur , je souhaite consulter la liste des contrats afin de suivre les engagements contractuels.	La liste affiche tous les contrats avec filtres, indicateurs visuels d'expiration et possibilité d'export.	++
		US30	En tant qu' administrateur , je souhaite recevoir des alertes avant l'expiration des contrats afin d'anticiper les renouvellements.	Des alertes automatiques sont envoyées à J-30, J-15 et J-7 par email, in-app et Slack.	+++
		US31	En tant qu' administrateur , je souhaite archiver un contrat afin de conserver l'historique.	Le contrat est archivé avec un changement de statut, avec possibilité de restauration ultérieure.	++
		US32	En tant que client , je souhaite demander la résiliation de mon contrat afin d'interrompre le service.	La demande de résiliation est créée avec motif et date souhaitée, l'administrateur reçoit une notification et peut traiter la demande.	+++
		US33	En tant qu' administrateur , je souhaite résilier un contrat afin de l'interrompre avant terme.	Le contrat est résilié avec date et motif, les tickets ouverts sont clôturés et le client est notifié.	+++
		US34	En tant que chef de projet , je souhaite consulter les contrats dans un calendrier afin de visualiser les dates d'expiration.	Le calendrier affiche les contrats avec leur date d'expiration. Les contrats proches de l'expiration sont mis en évidence. L'administrateur peut exporter le calendrier.	++

		US35	En tant que client , je souhaite soumettre un retour d'expérience à l'expiration de mon contrat afin d'exprimer ma satisfaction.	Le retour d'expérience est accessible pour les contrats expirés ou proches de l'expiration. Un seul retour d'expérience par contrat est accepté.	++
		US36	En tant qu' administrateur , je souhaite générer un PDF formaté du contrat afin de le partager avec le client.	Le PDF généré contient les informations nécessaires ; les dates d'intervention	+
5	Gestion des demandes clients	US38	En tant que client , je souhaite soumettre une demande de maintenance afin de signaler un besoin.	Le client peut joindre des fichiers à sa demande. Le système génère un numéro unique de suivi. Le client reçoit une confirmation de réception.	+++
		US39	En tant que chef de projet , je souhaite consulter les demandes en attente afin de les traiter.	Le chef de projet visualise toutes les demandes nécessitant un traitement. Les demandes sont triées par date de création. Les demandes urgentes sont identifiables visuellement.	+++
		US40	En tant que chef de projet , je souhaite valider une demande afin de la convertir en ticket.	Le chef de projet peut approuver une demande. Le chef de projet définit la catégorie et la priorité. Un ticket est automatiquement créé et lié à la demande.	+++

		US41	En tant que chef de projet , je souhaite demander des informations complémentaires afin de clarifier une demande.	Le chef de projet peut solliciter des précisions au client. Le client est notifié de la demande d'informations. Un rappel est envoyé si aucune réponse après 48h.	++
		US42	En tant que client , je souhaite compléter ma demande afin de fournir les informations manquantes.	Le client peut répondre aux questions posées. Le chef de projet est notifié de la réponse. La demande redevient disponible pour traitement.	++
		US43	En tant que chef de projet , je souhaite requalifier une demande afin de corriger sa catégorie ou sa priorité.	Le chef de projet peut modifier la catégorie d'une demande. Le chef de projet peut ajuster la priorité. Une justification est obligatoire pour toute modification. Toutes les modifications sont tracées.	++
		US44	En tant qu' utilisateur , je souhaite commenter une demande afin de communiquer sur le contexte.	L'utilisateur peut ajouter des commentaires à une demande. L'utilisateur peut joindre des fichiers aux commentaires. Les autres parties prenantes sont notifiées des nouveaux commentaires.	++
		US45	En tant qu' utilisateur , je souhaite consulter l'historique d'une demande afin de suivre son évolution.	L'utilisateur visualise tous les événements liés à la demande. L'historique affiche la date et l'auteur de chaque action. L'utilisateur peut filtrer l'historique par type d'événement.	++

5	Gestion des tickets	US46	En tant que développeur , je souhaite visualiser mes tickets dans un tableau Kanban afin de gérer mes tâches.	Le développeur visualise ses tickets organisés par état d'avancement. Le développeur peut déplacer un ticket d'un état à un autre. Le développeur voit si un ticket risque de dépasser son délai.	+++
		US47	En tant que chef de projet , je souhaite assigner un ticket à un développeur afin de répartir la charge.	Le chef de projet sélectionne un développeur disponible. Le système alerte si le développeur a déjà trop de tickets. Le développeur assigné est notifié immédiatement.	+++
		US48	En tant que chef de projet , je souhaite assigner un ticket à plusieurs développeurs afin de collaborer sur des tâches complexes.	Le chef de projet peut assigner plusieurs développeurs au même ticket. Le chef de projet définit le rôle de chaque développeur. Le temps estimé est réparti entre les développeurs.	++
		US49	En tant que développeur , je souhaite me désassigner d'un ticket afin de libérer ma charge.	Le développeur doit fournir un motif pour se désassigner. Le ticket redevient disponible pour assignation. Le chef de projet est notifié de la désassignation.	++
		US50	En tant que développeur , je souhaite démarrer un chronomètre sur un ticket afin de tracker mon temps.	Le développeur peut démarrer le suivi du temps de travail. Une session de travail est créée avec l'heure de début. Le développeur voit le temps écoulé en temps réel.	+++

		US51	En tant que développeur , je souhaite mettre en pause mon chronomètre afin de gérer les interruptions.	Le développeur peut suspendre temporairement le suivi du temps. Le temps écoulé est enregistré. Le développeur peut reprendre le suivi ultérieurement.	++
		US52	En tant que développeur , je souhaite arrêter le chronomètre afin de finaliser ma session.	Le développeur peut arrêter le suivi du temps. Le développeur peut ajouter un commentaire sur le travail effectué. La session est ajoutée à l'historique. Le développeur est alerté si le temps dépasse l'estimation.	+++
		US53	En tant que développeur , je souhaite ajouter manuellement une session de travail afin de corriger des oublis.	Le développeur peut enregistrer une session de travail passée. Le développeur doit fournir la date, l'heure et la durée. Le développeur doit justifier l'ajout manuel. Le système vérifie qu'il n'y a pas de chevauchement avec d'autres sessions.	++
		US54	En tant que développeur , je souhaite changer le statut d'un ticket afin de refléter l'avancement.	Le développeur peut faire progresser un ticket selon le workflow défini. Le système empêche les transitions invalides. Toutes les parties prenantes sont notifiées lors d'un changement. L'historique des changements est conservé.	+++

		US55	En tant que client , je souhaite valider la résolution d'un ticket afin de confirmer que le problème est résolu.	Le client peut confirmer que le problème est résolu. Le client peut ajouter un commentaire de validation. Le ticket est fermé après validation. Le développeur est notifié de la validation.	+++
		US56	En tant que chef de projet , je souhaite rouvrir un ticket fermé afin de traiter un problème récurrent.	Le chef de projet doit justifier la réouverture. Le délai SLA est réinitialisé. Les développeurs assignés sont notifiés.	++
		US57	En tant que chef de projet , je souhaite consulter le temps passé par ticket afin d'analyser la productivité.	Le chef de projet visualise le temps total passé sur chaque ticket. Le chef de projet voit le détail par développeur. Les données sont présentées sous forme de graphiques.	++
		US58	En tant que développeur , je souhaite signaler un blocage afin d'alerter sur un obstacle.	Le développeur peut signaler un obstacle bloquant. Le développeur précise le type et l'impact du blocage. Le chef de projet reçoit une notification urgente. Le délai SLA est suspendu pendant le blocage.	+++
		US59	En tant que membre d'équipe , je souhaite résoudre un blocage afin de permettre au développeur de reprendre son travail.	Le blocage est résolu avec commentaire obligatoire, le chrono SLA reprend et l'action est tracée.	++

		US60	En tant que développeur , je souhaite demander une extension de délai SLA afin de disposer de plus de temps.	La demande d'extension est créée avec durée et justification pour validation par le chef de projet.	++
		US61	En tant que chef de projet , je souhaite approuver ou rejeter les demandes d'extension SLA.	La liste des demandes en attente est affichée, l'approbation ou rejet avec commentaire met à jour automatiquement le SLA.	++
		US62	En tant qu' utilisateur , je souhaite commenter un ticket avec mentions afin de communiquer efficacement.	Les commentaires sont ajoutés avec éditeur riche, autocomplétion @utilisateur et distinction internes/externes.	+++
		US63	En tant qu' utilisateur , je souhaite ajouter des pièces jointes aux tickets afin de fournir du contexte.	Les fichiers sont ajoutés par drag & drop (images/PDF/logs, max 10MB) avec prévisualisation.	++
		US64	En tant que développeur , je souhaite être informé lorsqu'un autre utilisateur modifie le même ticket.	Une notification temps réel via WebSocket affiche un toast et un indicateur "X est en train d'éditer".	++
		US65	En tant qu' utilisateur , je souhaite consulter l'historique complet d'un ticket afin de comprendre son évolution.	La timeline chronologique inversée affiche tous les événements avec filtres par type et export PDF.	++
10	Calendrier et planification	US66	En tant qu' utilisateur , je souhaite visualiser les tickets et demandes de maintenance sur un calendrier mensuel afin d'avoir une vue globale des interventions	L'utilisateur peut naviguer entre les mois, distinguer les types d'événements (Tickets/Demandes) par couleur et voir les titres tronqués.	+++

		US67	En tant qu’ administrateur , je souhaite identifier visuellement les urgences SLA sur le calendrier afin de prioriser les interventions critiques.	Les événements en retard ou proches de l’échéance apparaissent en rouge/orange avec un indicateur visuel (pulse) sur les urgences critiques.	+++
		US68	En tant que Chef de Projet , je souhaite accéder à une vue Gantt / Agenda afin d’analyser la charge de travail et la progression de chaque développeur.	La vue affiche la timeline des affectations, le pourcentage de progression des tickets et le taux de charge calculé en temps réel par développeur.	++
		US69	En tant qu’ utilisateur , je souhaite accéder aux détails d’un ticket directement depuis le calendrier afin de consulter ou modifier l’intervention rapidement.	Un clic sur un événement dans le calendrier ou le panneau latéral redirige l’utilisateur vers la page de détails correspondante.	++
5	Messagerie et collaboration	US66	En tant qu’ utilisateur , je souhaite accéder à une messagerie intégrée afin de communiquer avec mon équipe.	L’utilisateur visualise les canaux de discussion disponibles. L’utilisateur voit le nombre de messages non lus. L’utilisateur identifie les membres présents.	++
		US67	En tant qu’ utilisateur , je souhaite envoyer des messages dans un channel afin de communiquer.	L’utilisateur peut rédiger et envoyer des messages. L’utilisateur peut joindre des fichiers. Les messages sont reçus instantanément par les autres membres.	+++

		US68	En tant qu' utilisateur , je souhaite répondre à un message afin de créer un fil de discussion.	L'utilisateur peut répondre à un message spécifique. Les réponses sont regroupées dans un fil dédié. Seuls les participants au fil reçoivent les notifications.	++
		US69	En tant qu' utilisateur , je souhaite mentionner des collègues afin d'attirer leur attention.	L'utilisateur peut mentionner un collègue dans un message. Le collègue mentionné reçoit une notification. Le message mentionné est mis en évidence pour le destinataire.	+++
		US70	En tant qu' utilisateur , je souhaite éditer mes messages afin de corriger des erreurs.	L'utilisateur peut modifier un message récemment envoyé. Le message modifié est identifiable. L'historique des modifications est conservé.	+
		US71	En tant qu' utilisateur , je souhaite supprimer mes messages afin de retirer du contenu inapproprié.	L'utilisateur peut supprimer ses propres messages. Une confirmation est demandée avant suppression. Le message supprimé reste visible pour l'audit.	+
		US72	En tant qu' utilisateur , je souhaite épingler des messages importants afin de les retrouver facilement.	L'utilisateur autorisé peut épingler un message. Les messages épinglés sont facilement accessibles. L'utilisateur peut retirer l'épingle.	++

		US73	En tant que chef de projet , je souhaite voir qui est en ligne dans mon équipe afin de savoir qui je peux solliciter.	Le chef de projet visualise les membres actuellement connectés. Le chef de projet peut filtrer par équipe ou projet. Le chef de projet voit l'heure de dernière activité.	++
		US74	En tant qu' utilisateur , je souhaite voir qui est en ligne afin de savoir qui est disponible.	L'utilisateur voit le statut de disponibilité des autres membres. L'utilisateur peut modifier son propre statut. Le statut est visible par tous les membres.	++
5	Gestion des documents	US75	En tant qu' utilisateur , je souhaite uploader des documents afin de centraliser les fichiers.	L'utilisateur peut téléverser des documents. L'utilisateur peut associer un document à un contrat, projet ou ticket. L'utilisateur peut ajouter un titre, une description et des tags.	++
		US76	En tant qu' utilisateur , je souhaite télécharger des documents afin de les consulter hors ligne.	L'utilisateur peut télécharger un document individuel. L'utilisateur peut télécharger plusieurs documents simultanément. Le système vérifie les permissions avant le téléchargement. La consultation est enregistrée pour traçabilité.	++

		US77	En tant qu' utilisateur , je souhaite gérer les versions des documents afin d'assurer la traçabilité.	L'utilisateur peut téléverser une nouvelle version d'un document existant. Le système numérote automatiquement les versions. L'utilisateur doit fournir un commentaire pour chaque nouvelle version.	++
		US78	En tant qu' utilisateur , je souhaite consulter l'historique des consultations afin de voir qui a accédé au document.	L'utilisateur visualise qui a consulté le document. L'utilisateur voit la date et l'heure de chaque consultation.	+
		US79	En tant qu' utilisateur , je souhaite archiver des documents afin de nettoyer la vue active.	L'utilisateur peut archiver un document. Une confirmation est demandée avant archivage. Le document archivé peut être restauré ultérieurement.	+
		US80	En tant qu' utilisateur , je souhaite être alerté si un fichier identique existe déjà afin d'éviter les doublons.	Le système détecte les fichiers identiques lors du téléversement. L'utilisateur est alerté si un doublon existe. L'utilisateur peut choisir de remplacer, créer une version ou annuler.	++
		US81	En tant que chef de projet , je souhaite générer automatiquement un PV à partir d'un ticket afin de documenter les interventions.	Le chef de projet peut générer un procès-verbal depuis un ticket. Le document contient toutes les informations du ticket et les actions réalisées. Le document peut être signé électroniquement.	+++

		US82	En tant que client , je souhaite télécharger les PV de mes tickets afin de conserver une trace des interventions.	Le client peut accéder aux procès-verbaux depuis ses tickets. Le client visualise l'historique de tous les PV générés. Le client reçoit une notification lors de la génération d'un nouveau PV.	++
12	Notifications et alertes	US83	En tant qu' utilisateur , je souhaite recevoir des notifications in-app afin d'être informé en temps réel.	L'utilisateur visualise le nombre de notifications non lues. L'utilisateur peut consulter la liste des notifications. L'utilisateur peut marquer une notification comme lue. L'utilisateur peut accéder directement à l'élément concerné.	+++
		US84	En tant qu' utilisateur , je souhaite recevoir des notifications par email afin d'être informé hors connexion.	L'utilisateur peut configurer la fréquence des notifications email. L'utilisateur reçoit des emails pour les événements importants.	++
		US85	En tant que membre technique , je souhaite recevoir des notifications Slack afin d'être alerté sur mon outil de travail.	L'administrateur peut configurer l'intégration Slack par contrat ou projet. Les événements importants sont envoyés sur Slack.	++
		US86	En tant qu' utilisateur , je souhaite recevoir des notifications push navigateur afin d'être alerté même hors onglet.	L'utilisateur peut autoriser les notifications navigateur. L'utilisateur reçoit des alertes pour les événements critiques. L'utilisateur peut accéder à l'application en cliquant sur la notification.	++

		US87	En tant qu' utilisateur , je souhaite consulter l'historique de mes notifications afin de retrouver une information.	L'utilisateur visualise toutes ses notifications passées. L'utilisateur peut filtrer par type ou période. L'utilisateur peut rechercher dans l'historique.	++
5	Dashboards et rapports	US88	En tant qu' administrateur , je souhaite consulter un dashboard global afin de piloter le système.	L'administrateur visualise les indicateurs clés du système. L'administrateur peut filtrer les données par période. L'administrateur identifie les alertes nécessitant une action.	+++
		US89	En tant que chef de projet , je souhaite consulter un dashboard d'équipe afin de piloter l'activité.	Le chef de projet visualise la charge de travail de son équipe. Le chef de projet suit la performance par rapport aux délais. Le chef de projet compare le temps passé au temps estimé.	+++
		US90	En tant que développeur , je souhaite consulter mon dashboard personnel afin d'organiser ma journée.	Le développeur visualise ses tickets assignés. Le développeur voit ses priorités du jour. Le développeur suit son temps de travail. Le développeur identifie les échéances proches.	++
		US91	En tant que client , je souhaite consulter mon dashboard afin de suivre mes projets.	Le client visualise ses contrats actifs. Le client suit l'état de ses demandes et tickets. Le client accède à ses documents récents. Le client identifie ses contacts dans l'équipe.	++

		US92	En tant qu' administrateur , je souhaite générer un rapport d'activité mensuel afin de présenter les statistiques.	L'administrateur génère un rapport pour une période donnée. Le rapport contient les tickets traités et le temps passé. Le rapport indique le respect des délais. Le rapport peut être exporté.	+++
		US93	En tant qu' administrateur , je souhaite générer des rapports de conformité SLA afin de justifier les performances.	L'administrateur génère un rapport de conformité par contrat. Le rapport affiche le taux de respect des délais. Le rapport détaille les tickets en dépassement.	+++
		US94	En tant que chef de projet , je souhaite générer des rapports d'activité afin de présenter l'avancement.	Le chef de projet génère un rapport pour son équipe ou projet. Le rapport affiche les tickets traités et le temps passé. Le rapport présente l'évolution de l'activité.	++
14	Historique et traçabilité	US95	En tant qu' administrateur , je souhaite consulter l'historique complet de toutes les actions afin d'assurer la traçabilité.	L'administrateur visualise toutes les actions effectuées dans le système. L'historique indique l'utilisateur, la date et l'action réalisée. L'historique conserve les valeurs avant et après modification.	+++
		US96	En tant qu' administrateur , je souhaite exporter les logs d'audit afin de répondre aux exigences de conformité.	L'administrateur peut exporter l'historique des actions. L'administrateur peut filtrer par période, utilisateur ou type d'action. Les données exportées sont sécurisées.	++

15	Versioning avancé	US97	En tant qu' utilisateur , je souhaite créer une nouvelle version d'un document afin de maintenir l'historique.	L'utilisateur peut téléverser une nouvelle version d'un document. L'utilisateur doit fournir un commentaire pour la nouvelle version. Le système numérote automatiquement les versions. Toutes les versions précédentes sont conservées.	++
		US98	En tant qu' utilisateur , je souhaite comparer deux versions d'un document afin de voir les différences.	L'utilisateur peut sélectionner deux versions à comparer. Les deux versions sont affichées côte à côte. Les modifications sont mises en évidence.	+
		US99	En tant qu' administrateur , je souhaite voir qui a consulté un document afin de suivre sa diffusion.	L'administrateur visualise la liste des consultations du document. L'historique indique la date, l'heure et l'utilisateur. Des statistiques de consultation sont disponibles.	++
16	Assistance intelligente	US100	En tant que client , je souhaite que le système suggère automatiquement une catégorie pour ma demande.	Le système analyse le contenu de la demande. Le système propose une catégorie appropriée. Le client peut accepter ou modifier la suggestion.	+++
		US101	En tant qu' administrateur , je souhaite que le modèle de suggestion s'améliore afin d'offrir des recommandations précises.	Le système apprend des validations et rejets des suggestions. Le modèle se réentraîne périodiquement. La précision des suggestions est mesurée.	++

		US102	En tant que chef de projet , je souhaite que le système suggère une priorité afin de traiter les demandes efficacement.	Le système analyse le contenu et le contexte de la demande. Le système propose un niveau de priorité. Le chef de projet peut ajuster la priorité suggérée.	++
		US103	En tant que développeur , je souhaite des suggestions de solutions basées sur le passé afin de résoudre plus rapidement.	Le système identifie les tickets similaires résolus. Le système suggère les solutions précédemment appliquées. Le développeur peut accepter ou ignorer les suggestions.	++
		US104	En tant que développeur , je souhaite évaluer la pertinence des suggestions afin d'améliorer la qualité.	Le développeur peut noter la pertinence d'une suggestion. Les retours sont collectés pour améliorer le système. Le modèle s'ajuste en fonction des évaluations.	+
		US105	En tant qu' administrateur , je souhaite que le système détecte automatiquement les doublons afin d'éviter les demandes redondantes.	Le système analyse la similarité lors de la création d'une demande. Le système signale les demandes potentiellement identiques. Les demandes similaires sont automatiquement liées.	++

TABLE II.2 – Product Backlog

2 Planification des sprints

Un sprint représente une période de développement durant laquelle l'équipe s'engage à livrer un ensemble cohérent de fonctionnalités opérationnelles. Cette approche itérative permet d'obtenir des retours réguliers et d'ajuster les priorités au fil de l'avancement.

Notre projet est organisé en **4 sprints** de **3 à 4 semaines** chacun, couvrant une période totale de développement d'**environ 3 mois et demi**. La stratégie de répartition suit une logique architecturale : fondations sécuritaires

(Sprint 1), gestion contractuelle (Sprint 2), cœur métier opérationnel (Sprint 3), et fonctionnalités avancées (Sprint 4). Chaque sprint commence par une réunion de planification (Sprint Planning) où l'équipe sélectionne les user stories du Product Backlog à réaliser, et se termine par une revue de sprint (Sprint Review) et une rétrospective (Sprint Retrospective).

La figure suivante présente le déroulement chronologique de nos sprints.

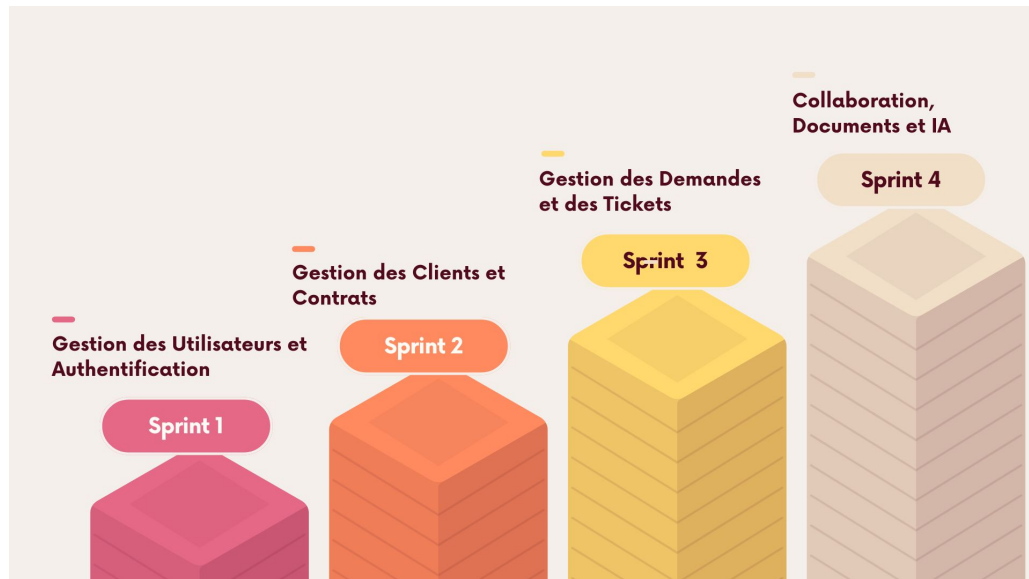


FIGURE II.13 – Planification chronologique des sprints

2.1 Sprint 1 – Gestion des Utilisateurs, Clients et Authentification

Période : du 17 février au 11 mars (3 semaines)

Objectif du sprint : Mettre en place le système d'authentification sécurisé, la gestion complète des utilisateurs et des clients avec contrôle d'accès basé sur les rôles (RBAC).

User Stories sélectionnées :

- **Epic 1 – Authentification et Sécurité :** US01 à US05 (5 user stories)
- **Epic 2 – Gestion des utilisateurs :** US06 à US11 (6 user stories)
- **Epic 3 – Gestion des rôles et permissions :** US12 à US14 (3 user stories)
- **Epic 4 – Demandes d'inscription OAuth :** US15 à US16 (2 user stories)
- **Epic 5 – Gestion des clients :** US17 à US23 (7 user stories)

Livrables attendus :

- Système d'authentification fonctionnel (JWT + OAuth Google/GitHub)
- Module de gestion des utilisateurs avec verrouillage de compte
- Système RBAC complet avec gestion des rôles et permissions granulaires
- Module de gestion des clients avec profils société

2.2 Sprint 2 – Gestion des Projets et Contrats

Période : du 12 mars au 2 avril (3 semaines)

Objectif du sprint : Développer la gestion complète des projets et contrats de maintenance avec système d’alertes d’expiration et configuration SLA.

User Stories sélectionnées :

- **Epic 6 – Gestion des projets :** US24 à US27 (4 user stories)
- **Epic 7 – Gestion des contrats :** US28 à US37 (10 user stories)
- **Epic 14 – Historique et traçabilité :** US95 à US96 (2 user stories)

Livrables attendus :

- Module de gestion des projets avec équipes et membres
- Système de gestion des contrats avec SLA, alertes à J-30/J-15/J-7 et workflow d’approbation
- Système d’audit et traçabilité complet (ActivityLog)

2.3 Sprint 3 – Gestion des Demandes et Tickets

Période : du 3 avril au 1 mai (4 semaines)

Objectif du sprint : Implémenter le cœur métier avec la gestion des demandes et tickets incluant suivi du temps, SLA et tableau Kanban.

User Stories sélectionnées :

- **Epic 8 – Gestion des demandes :** US38 à US45 (8 user stories)
- **Epic 9 – Gestion des tickets :** US46 à US65 (20 user stories)
- **Epic 13 – Dashboards :** US88 à US91 (4 user stories)

Livrables attendus :

- Module de gestion des demandes de maintenance avec classification
- Système complet de gestion des tickets avec vue Kanban (drag & drop)
- Chronomètre de travail par session et respect des SLA en minutes ouvrées
- Tableaux de bord pour pilotage (Admin, Chef de projet, Client)

2.4 Sprint 4 – Collaboration, Documents et IA

Période : du 2 mai au 31 mai (4 semaines)

Objectif du sprint : Finaliser avec les fonctionnalités avancées de collaboration, gestion documentaire, notifications multicanaux et intelligence artificielle.

User Stories sélectionnées :

- **Epic 10 – Messagerie :** US66 à US74 (9 user stories)
- **Epic 11 – Gestion des documents :** US75 à US82 (8 user stories)

- **Epic 12 – Notifications** : US83 à US87 (5 user stories)
- **Epic 13 – Rapports** : US92 à US94 (3 user stories)
- **Epic 15 – Versioning avancé** : US97 à US99 (3 user stories)
- **Epic 16 – Intelligence Artificielle** : US100 à US105 (6 user stories)

Livrables attendus :

- Système de messagerie et collaboration temps réel (WebSocket)
- Module de gestion documentaire avec versioning
- Système de notifications multicanaux (in-app, email, Web Push)
- Module de génération de rapports et procès-verbaux PDF
- Moteur d'IA pour classification automatique des demandes et assistance intelligente (Groq / LLaMA 3.3 70B)

Synthèse de la planification : Cette répartition assure une progression logique où chaque sprint s'appuie sur les livrables précédents. Les dépendances techniques sont respectées (authentification avant tickets, contrats avant demandes), et la valeur métier croît progressivement jusqu'aux fonctionnalités différenciantes du sprint final.

3 Planning prévisionnel (Diagramme de Gantt)

« Un diagramme de Gantt est un outil d'organisation et de gestion de projets. Il s'agit d'un diagramme à barres horizontales qui présente toutes les tâches d'un projet en fonction d'un calendrier. Il aide les chefs de projet à visualiser les tâches requises et à suivre l'avancement de leur projet. » [5]

Dans le cadre du développement de notre projet **SolidMaint**, le planning prévisionnel a été réalisé pour organiser les différentes phases de développement. La figure ci-dessous illustre cette planification :

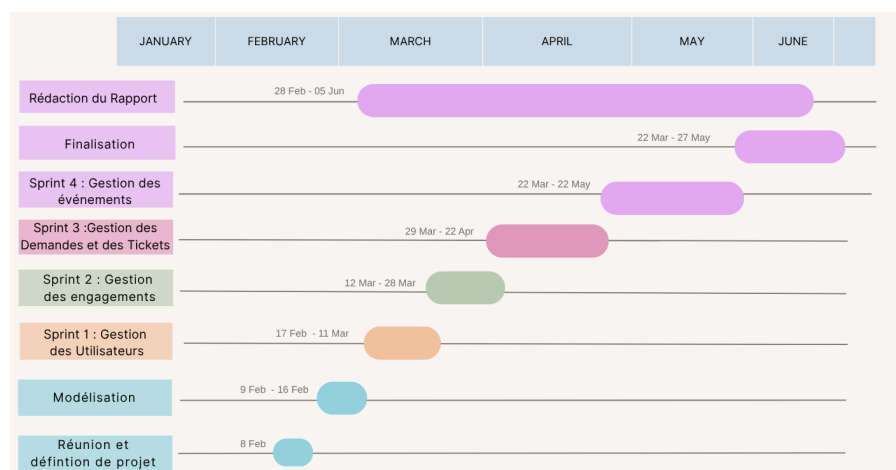


FIGURE II.14 – Diagramme de Gantt

IV Environnement de Développement

On va commencer par la présentation de l'environnement matériel et logiciel que nous utilisons.

1 Environnement matériel

Ordinateur	1	2
Propriétaire	Rihab Chourou	Eya Gharbi
Processeur	Core i3	Core i3
RAM	20 Go	16 Go
Disque dur	1 To	256 Go
Système d'exploitation	Windows 10 Professionnel N 64-bit	Windows 10 Professionnel 64-bit

TABLE II.3 – Environnement matériel de développement

2 Environnement logiciel

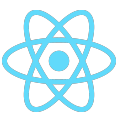
2.1 Langages de programmation et frameworks



NestJS v11 : « C'est un framework permettant de créer des applications serveur Node.js performantes et évolutives. Il utilise JavaScript progressif, est conçu avec TypeScript et le prend entièrement en charge (tout en permettant aux développeurs de coder en JavaScript pur). » [6]



Next.js v16 : « C'est un framework React permettant de créer des applications web complètes. Il configure également automatiquement les outils de bas niveau tels que les modules de regroupement et les compilateurs. » [7]



React 18.3 : « React est une bibliothèque JavaScript pour la construction d'interfaces utilisateur. Elle permet de créer des composants réutilisables qui gèrent leur propre état et se composent pour former des interfaces complexes. » [8]



Tailwind CSS v4.2 : « Un framework CSS utilitaire, doté de classes telles que `flex`, `pt-4`, `text-center` et `rotate-90`, qui peuvent être combinées pour créer n'importe quel design, directement dans votre balisage. » [9]



shadcn/ui v3.8 : « est un ensemble de composants élégants et accessibles, ainsi qu'une plateforme de distribution de code. Compatible avec vos frameworks et modèles d'IA préférés. Logiciel libre. » [10]

2.2 Outils de développement et modélisation



Visual Studio Code : « est un éditeur de code source léger mais puissant, disponible sur Windows, macOS et Linux. Il intègre la coloration syntaxique, la complétion intelligente du code (IntelliSense), le débogage, le contrôle de version Git et une riche bibliothèque d'extensions. » [11]



GitLab : « offre un espace de stockage de code en ligne ainsi que des fonctionnalités de suivi des problèmes et d'intégration continue/déploiement continu (CI/CD). Le dépôt permet d'héberger différentes branches de développement et versions. » [12]



UML : « est un langage permettant de spécifier, visualiser, construire et documenter les artefacts des systèmes logiciels, ainsi que de modéliser des systèmes métier et d'autres systèmes non logiciels. L'UML représente un ensemble de bonnes pratiques d'ingénierie qui ont fait leurs preuves dans la modélisation de systèmes vastes et complexes. » [13]



Lucidchart : « est un environnement de travail visuel qui associe création de diagrammes, visualisation de données et fonctionnalités de collaboration afin de faciliter la communication et stimuler l'innovation. » [14]



Swagger / OpenAPI : « Swagger permet la conception, la gouvernance et les tests tout au long du cycle de vie complet des API, garantissant ainsi la qualité à chaque étape. Il génère automatiquement une documentation interactive de l'API à partir des annotations du code source. » [15]

2.3 Systèmes de gestion de bases de données



Prisma ORM v7 : « est un ORM open source de nouvelle génération qui offre un accès rapide et sécurisé aux bases de données PostgreSQL, MySQL et autres. Il fournit un client typé généré automatiquement à partir du schéma de données, garantissant la sécurité des types à la compilation et des requêtes SQL optimisées à l'exécution. » [16]



PostgreSQL 16 : « est un système de gestion de bases de données objet-relationnelles (SGBD-OR) open source. Il a été pionnier dans de nombreux concepts qui ne sont devenus disponibles que bien plus tard dans certains systèmes de bases de données commerciaux. Il est reconnu pour sa robustesse, sa conformité aux standards SQL et ses performances sur de grands volumes de données. » [17]



Redis 7 : « est un système de stockage de données en mémoire open source, utilisé comme base de données, cache et courtier de messages. Sa structure de données en mémoire vive lui confère des performances de lecture/écriture exceptionnellement rapides. Dans notre projet, Redis est utilisé comme couche de cache pour les permissions utilisateurs et les données fréquemment consultées, avec une durée de vie (TTL) de 60 secondes. » [18]



Neon : « est un service de base de données PostgreSQL cloud natif, sans serveur et entièrement managé. Il offre une mise à l'échelle automatique, une séparation calcul/stockage et une compatibilité totale avec PostgreSQL, permettant de démarrer sans infrastructure à gérer. » [19]

2.4 Conteneurisation et déploiement



Docker & Docker Compose : « Docker est une plateforme open source qui permet d'automatiser le déploiement d'applications dans des conteneurs logiciels légers et portables. Docker Compose permet de définir et d'exécuter des applications multi-conteneurs à l'aide d'un fichier de configuration YAML. Dans notre projet, Docker Compose orchestre quatre services : PostgreSQL, Redis, le backend NestJS et le frontend Next.js. » [20]

2.5 Outil de gestion de projet Scrum



Notion : « est un puissant outil no-code de productivité. Il aide à gérer une base de connaissances, pour rendre plus facile et convivial le travail au sein des collaborateurs. Cette application se révèle indispensable autant pour les travailleurs autonomes que pour les professionnels en entreprise. C'est également un outil parfait pour gérer son projet agile (Product Backlog, suivi des sprints, rétrospectives). » [21]

2.6 Environnement de composition de documents



LTEX : « est un systme de composition de haute qualit; il comprend des fonctionnalits conues pour la production de documents techniques et scientifiques. LTEX est la norme de facto pour la communication et la publication de documents scientifiques. » [22]

2.7 Outil de design graphique



Canva : « est une plateforme de conception et de communication visuelle en ligne dont la mission est de permettre  chacun dans le monde de concevoir n’importe quoi et de publier n’importe o. » [23]

Le logo de **SolidMaint** a t conu via **Canva** en cohrence avec la charte visuelle de **SolidWall Consulting**. Il associe le **noir** et le **jaune dor**  des symboles vocateurs de maintenance, de scurit et de traabilit.



FIGURE II.15 – Logo de SolidMaint

V Architecture de l’application

1 Architecture physique

L’architecture systme est un ensemble de dcisions de conception qui dfinit l’interaction des composants entre eux. Le choix de l’architecture est trs sensible car elle permet d’optimiser les performances du systme et facilite son volution en fonction des volumes de donnes. La figure ci-dessous montre l’architecture physique de notre application :

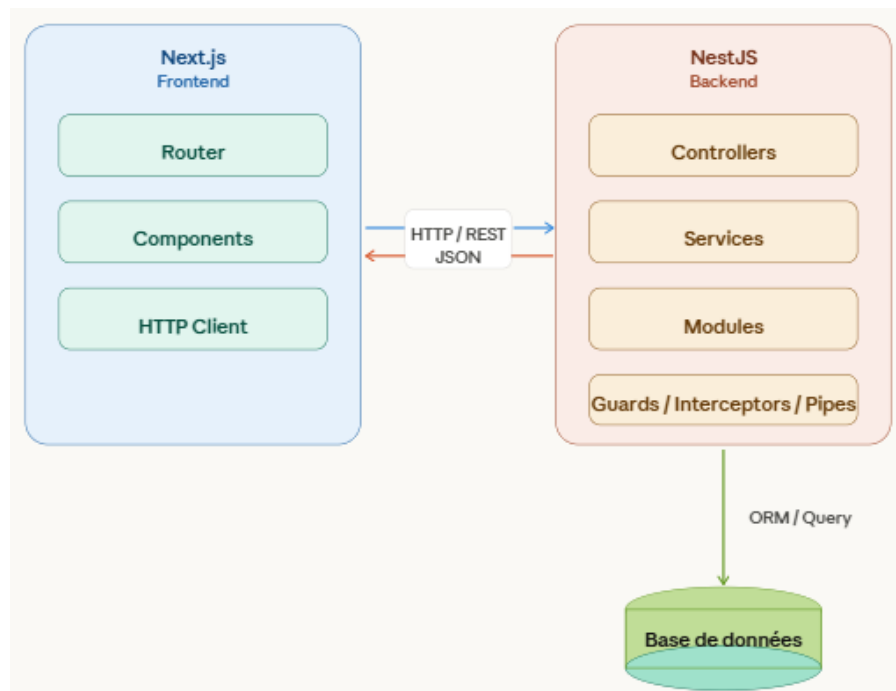


FIGURE II.16 – Architecture physique de SolidMaint

2 Architecture logique

Nous avons choisi comme architecture logicielle l'**architecture monolithique modulaire**. Ce choix est basé sur la combinaison entre la simplicité et la robustesse des applications monolithiques traditionnelles, tout en bénéficiant de la clarté structurelle des architectures orientées modules.

L'architecture du **monolithe modulaire** nous permet de travailler dans une base de code unifiée, avec des frontières clairement définies et des modules indépendants. Elle consiste à diviser le système en plusieurs modules fonctionnels cohésifs, chacun encapsulant sa propre logique métier, ses contrôleurs, ses services et ses DTOs.

Nous avons choisi cette architecture afin de maintenir l'indépendance des modules et d'obtenir une grande cohésion, une encapsulation des données, ainsi qu'une focalisation sur les fonctionnalités métiers, tout en conservant la simplicité de déploiement d'un monolithe.

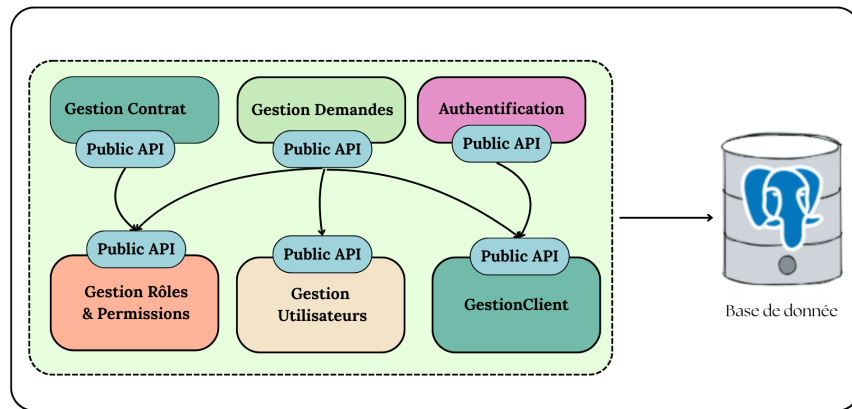


FIGURE II.17 – Architecture monolithique modulaire

VI Conclusion

Ce chapitre a établi les fondements techniques et méthodologiques de notre projet **SolidMaint**. L'identification des six acteurs et la spécification des besoins fonctionnels et non fonctionnels ont permis de définir précisément les exigences de la plateforme. La modélisation UML et l'approche de pilotage Scrum avec la planification en quatre sprints structurent la démarche de développement.

L'environnement de développement choisi et l'architecture modulaire garantissent une solution robuste et évolutive. Ces éléments constituent la base solide pour aborder la phase de réalisation.

Le chapitre suivant détaillera l'implémentation du premier sprint consacré à la gestion des utilisateurs et authentification.

Références

- **NestJS**, Documentation officielle — <https://docs.nestjs.com> [6]
- **Next.js**, Documentation officielle — <https://nextjs.org/docs> [7]
- **React**, Guide officiel — <https://react.dev> [8]
- **Tailwind CSS**, Documentation — <https://tailwindcss.com> [9]
- **shadcn/ui**, Guide des composants — <https://ui.shadcn.com/docs> [10]
- **Visual Studio Code**, Documentation officielle — <https://code.visualstudio.com/docs> [11]
- **GitLab**, Guide utilisateur — <https://about.gitlab.com> [12]
- **UML**, Spécifications officielles — <https://www.omg.org/uml> [13]
- **Lucidchart**, Outil de modélisation — <https://www.lucidchart.com> [14]
- **Swagger**, Documentation des APIs — <https://swagger.io> [15]
- **Prisma ORM**, Documentation — <https://www.prisma.io/docs> [16]
- **PostgreSQL**, Site officiel — <https://www.postgresql.org> [17]
- **Redis**, Documentation officielle — <https://redis.io/docs> [18]
- **Neon**, Cloud Postgres — <https://neon.tech/docs> [19]
- **Docker**, Documentation officielle — <https://docs.docker.com> [20]
- **Notion**, Espace de travail — <https://www.notion.so> [21]
- **L^AT_EX**, Système de composition — <https://www.latex-project.org> [22]
- **Canva**, Design graphique — <https://www.canva.com> [23]
- **Playwright**, Framework de test E2E — <https://playwright.dev/docs/intro> [24]
- **Vitest**, Framework de test unitaire — <https://vitest.dev> [25]

CHAPITRE 3

Gestion des Utilisateurs et Authentification

Sommaire



I	Sprint Planning Meeting	61
II	Analyse	68
III	Diagramme de classes de première sprint	81
IV	Diagramme de séquence	81
V	Schéma relationnel	86
VI	Réalisation	86
VII	Tests et Validation	87

Introduction

Dans le chapitre précédent, nous avons défini les besoins liés à notre solution et réparti les différentes tâches du projet afin que les phases de travail soient correctement planifiées.

Au cours de ce chapitre, nous présenterons les fonctionnalités liées à la gestion des utilisateurs, à l’authentification et au contrôle d’accès basé sur les rôles (RBAC), retenues dans le cadre du premier sprint de notre projet.

I Sprint Planning Meeting

La planification du sprint est essentielle à la réussite de notre solution, au bout de laquelle on clarifie les différentes fonctionnalités à implémenter lors du sprint à venir. Cette partie est efficace afin de préparer des objectifs clairs. L’objectif de ce sprint est de mettre en place les fondations fonctionnelles et sécuritaires de la solution, en implémentant les mécanismes d’authentification et de sécurisation des accès ainsi que les fonctionnalités de gestion des utilisateurs et de leurs profils. Il s’agit également d’assurer un contrôle efficace des accès et des permissions en introduisant l’administration du système à travers la gestion des comptes utilisateurs, des rôles et des permissions (RBAC).

1 Backlog du premier sprint

Voici l’ensemble des fonctionnalités issues du Product Backlog, qui seront réalisées dans le cadre de ce sprint.

1 : Effort faible, 2 : Effort moyen, 3 : Effort élevé

ID US	User Story	Description technique	Effort
Epic 1 - Authentification et Sécurité			
US01	En tant qu’ utilisateur , je souhaite m’authentifier avec email et mot de passe afin d’accéder à mes informations de manière sécurisée.	• Développer l’endpoint "POST /api/auth/login" • Mettre en place la gestion de session via JWT (access token 15 min, refresh token 7 jours stocké en base) • Réaliser l’interface connexion login • Consommer l’API d’authentification • Gérer les tentatives échouées (3 essais) avec verrouillage temporaire	2

US02	En tant qu' utilisateur , je souhaite réinitialiser mon mot de passe afin de retrouver l'accès à mon compte.	• Développer les endpoints "POST /api/auth/mot-de-passe-oublie" et "POST /api/auth/reinitialiser-mot-de-passe" • Générer un token sécurisé (bcrypt) avec expiration de 1 heure, stocké dans TokenReinitialisationMotDePasse • Invalider les anciens tokens non utilisés à chaque nouvelle demande • Configurer l'envoi d'un e-mail de réinitialisation et d'un e-mail de confirmation via Nodemailer • Créer les formulaires « mot de passe oublié » et de réinitialisation côté Next.js	2
US03	En tant qu' utilisateur , je souhaite m'authentifier via Google ou GitHub afin de me connecter rapidement sans créer de compte.	• Intégrer les stratégies OAuth2 via "passport-google-oauth20" et "passport-github2" dans NestJS (@nestjs/passport) • Implémenter les routes "GET /api/auth/google", "GET /api/auth/github" et leurs callbacks • Si l'utilisateur existe en base et est actif : générer les tokens JWT et rediriger vers le frontend avec ?oauth=success • Vérifier l'existence de l'utilisateur en base de données et une demande d'inscription OAuth est envoyée à l'admin • Implémenter les routes /auth/google, /auth/github et leurs callbacks, avec génération d'un JWT • Implémenter les boutons OAuth sur la page de login	3

US04	En tant qu' utilisateur , je souhaite me déconnecter de la plateforme afin de sécuriser ma session.	• Développer l'endpoint "POST /auth/logout" avec révocation du refresh token en base • Supprimer accessToken, refreshToken et user du localStorage • Vider le cache React Query (<code>queryClient.clear()</code>) • Réaliser le bouton de déconnexion	1
US05	En tant qu' utilisateur , je souhaite que mon compte soit protégé contre les tentatives de connexion malveillantes afin de garantir la sécurité de mes données.	• Implémenter le compteur de tentatives échouées (tentativesConnexionEchouees) en base de données • Verrouiller le compte après 3 tentatives échouées pendant 1 minute via le champ compteVerrouilleJusquA • Réinitialiser le compteur à 0 après une connexion réussie • Appliquer le rate limiting global via @nestjs/throttler sur les endpoints publics	2
Epic 2 - Gestion des utilisateurs			
US06	En tant qu' utilisateur , je souhaite gérer mon profil afin que mes informations soient toujours à jour.	• Développer les endpoints "GET /api/profil" et "PUT /api/profil" pour consulter et modifier les informations • Implémenter l'upload de la photo de profil "POST /api/profil/upload-photo" • Développer le changement de mot de passe via "POST /api/profil/change-password" avec vérification de l'ancien mot de passe (bcrypt) • Gérer les e-mails secondaires : "GET/POST/DELETE /api/profil/emails" et "PATCH /api/profil/emails/principal" • Réaliser l'interface profil côté Next.js avec formulaire prérempli	2

US07	En tant qu' utilisateur , je souhaite consulter l'historique de mes activités afin de suivre mes actions dans le système.	• Développer l'endpoint "GET /api/activity-logs/me" retournant les activités de l'utilisateur connecté (limité aux 30 derniers jours par défaut) • Développer "GET /api/activity-logs/me/stats" pour les statistiques personnelles (taux de succès, actions fréquentes, activité par jour) • Développer "GET /api/activity-logs/me/export" pour l'export CSV ou JSON • Implémenter le filtrage par entityType, action, status, plage de dates et recherche textuelle (contains) • Filtrer les entités sensibles selon le rôle (CLIENT et CHEF_PROJET ne voient pas les entités Client et Utilisateur) • Réaliser l'interface timeline avec filtres côté Next.js	2
US08	En tant qu' administrateur , je souhaite créer des utilisateurs afin de gérer les accès au système.	• Développer l'endpoint "POST /api/users" • Générer automatiquement un mot de passe temporaire haché via bcrypt • Générer un token de réinitialisation (1 heure) et construire le lien de réinitialisation • Créer automatiquement le "ProfilEmploye" avec matricule "EMP-{id}" • Valider les données d'entrée via class-validator • Envoyer un e-mail de bienvenue via Node-mailer contenant le mot de passe temporaire et le lien de réinitialisation • Réaliser le formulaire d'ajout côté Next.js	3

US09	En tant qu' administrateur , je souhaite modifier les informations d'un utilisateur afin de maintenir des données à jour.	• Développer l'endpoint "PUT /api/users/ :id" • Valider les données via class-validator • Réaliser le formulaire de modification côté Next.js	2
US10	En tant qu' administrateur , je souhaite activer et désactiver un compte utilisateur afin de contrôler l'accès au système.	• Développer les endpoints "PATCH /api/users/ :id/desactiver" et "PATCH /api/users/ :id/reactiver" • Implémenter les vérifications de réassignation des tickets et des demandes en cas de désactivation • Envoyer des notifications pour informer l'utilisateur de l'activation ou désactivation de son compte • Réaliser les boutons d'action avec appels des API	3
US11	En tant qu' administrateur , je souhaite consulter la liste des utilisateurs afin de visualiser et gérer efficacement les comptes.	• Développer l'endpoint "GET /api/users" retournant les utilisateurs hors rôles CLIENT et ADMIN, hors utilisateur connecté • Développer l'endpoint de recherche "GET /api/users/search?q=&limit=" avec filtre insensible à la casse • Réaliser l'interface d'administration avec vue grille et vue liste, filtres par statut et par rôle, barre de recherche dynamique	1

Epic 3 - Gestion des rôles et permissions

US12	En tant qu' administrateur , je souhaite gérer des rôles afin de structurer les accès au système.	• Développer les endpoints CRUD sous <code>"/api/admin/roles-permissions/roles"</code> : POST, GET, PATCH / :id, DELETE / :id • Implémenter l'archivage <code>"PATCH / :id/archive"</code> et la restauration <code>"PATCH / :id/restore"</code> • Attribuer automatiquement 5 permissions de base à chaque nouveau rôle (users.read.all, profile.read.self, profile.update.self, notifications.read, notifications.mark.read) • Protéger les rôles système (roleSysteme = true) contre la suppression et le renommage • Implémenter le PermissionsRolesGuard et le décorateur @Permissions() pour sécuriser les routes • Réaliser l'interface de gestion des rôles côté Next.js	3
US13	En tant qu' administrateur , je souhaite attribuer des permissions granulaires aux rôles afin de contrôler précisément les accès.	• Développer les endpoints d'attribution et de retrait : <code>"POST /api/admin/roles-permissions/roles/ :roleId/permissions"</code> et <code>"DELETE /api/admin/roles-permissions/roles/ :roleId/permissions/ :permissionId"</code> • Développer l'endpoint d'attribution directe : <code>"POST /api/admin/roles-permissions/assign-permission"</code> • Implémenter la détection des doublons • Développer l'interface interactive de type « Matrice de contrôle d'accès » avec cases à cocher organisées par catégorie	2

US14	En tant qu' administrateur , je souhaite consulter la matrice rôles-permissions afin de visualiser rapidement les droits de chaque rôle.	• Développer l'endpoint "GET /api/admin/roles-permissions/roles" retournant les rôles avec leurs permissions imbriquées (include : rolePermissions) • Développer l'endpoint "GET /api/admin/roles-permissions/permissions" retournant toutes les permissions triées par codePermission • Structurer les données côté frontend en matrice rôles × permissions avec cases cochées/décochées • Réaliser l'interface de visualisation avec vue « Par rôle » et export de la matrice	3
Epic 4 - Demandes d'inscription OAuth			
US15	En tant qu' utilisateur externe , je souhaite demander l'accès via OAuth afin de m'inscrire facilement.	• La demande est créée automatiquement dans la stratégie Passport (GoogleStrategy / GithubStrategy) lors du callback OAuth • Enregistrer la demande en base (DemandeInscriptionOAuth) avec statut EN_ATTENTE et le rôle souhaité (transmis via req.query.state ou req.query.role, défaut : CLIENT) • Détecter les doublons (même email + fournisseur) via upsert pour éviter les entrées multiples • Notifier tous les administrateurs actifs par e-mail (envoyerEmailDemandeInscriptionAdmin()) • Envoyer un e-mail de confirmation au visiteur (envoyerEmailDemandeRecue()) • Afficher un message d'attente côté frontend (?oauth=pending)	3

US16	En tant qu' administrateur , je souhaite valider ou rejeter les demandes d'inscription OAuth afin de contrôler les accès.	• Développer l'endpoint de liste : "GET /api/admin/demandes-inscription" avec filtre par statut et compteur "GET /api/admin/demandes-inscription/count-pending" • Développer l'endpoint d'approbation : "PATCH /api/admin/demandes-inscription/:id/approuver" — crée le compte utilisateur, attribue le rôle, marque la demande APPROUVEE • Développer l'endpoint de refus : "PATCH /api/admin/demandes-inscription/:id/refuser" avec motif optionnel, marque la demande REFUSEE • Configurer l'envoi d'e-mails de confirmation ou de refus via Nodemailer • Réaliser l'interface de gestion des demandes avec actions d'approbation/refus côté Next.js	3
------	--	--	---

II Analyse

Dans le processus de développement, l'analyse joue un rôle important ; elle permet d'identifier les exigences du système et de traduire les besoins exprimés en spécifications claires et cohérentes. C'est pour cela que nous allons présenter les diagrammes ainsi que les tableaux descriptifs des cas d'utilisation

1 Diagramme des cas d'utilisation du premier sprint

La figure ci-dessous présente le diagramme de cas d'utilisation du premier sprint :

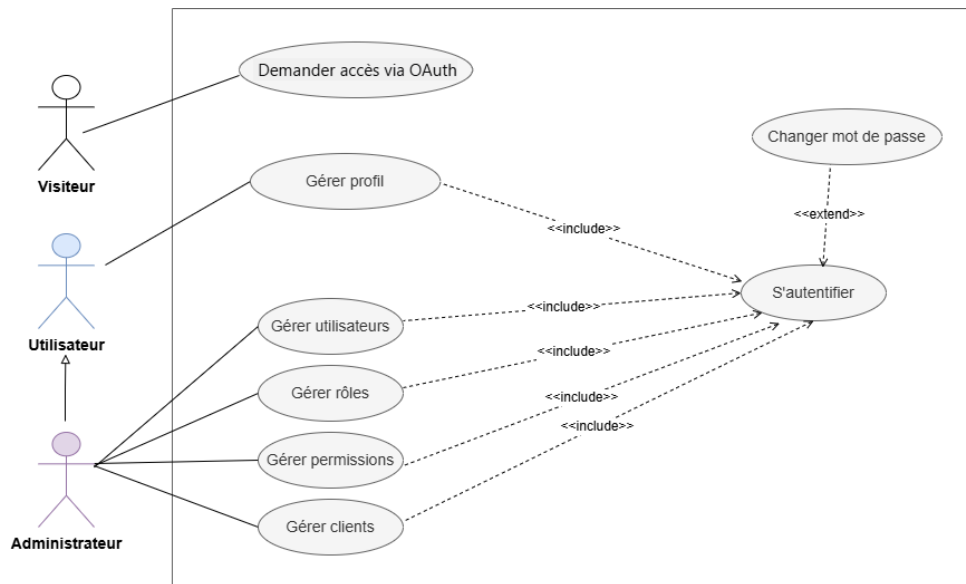


FIGURE III.1 – Diagramme des cas d'utilisation du premier sprint

Nous allons accompagner les cas d'utilisation d'un texte descriptif pour expliquer le scénario de chaque cas ainsi que ses exceptions.

2 Description textuelle de cas d'utilisation «S'autentifier»

La figure ci-dessous présente la description textuelle de cas d'utilisation «S'autentifier» :

Nom du cas d'utilisation	S'authentifier
Acteurs	Utilisateur
Préconditions	L'utilisateur dispose déjà d'un compte actif
Postconditions	L'utilisateur est authentifié et reçoit un jeton d'accès ainsi qu'un jeton de rafraîchissement
Scénario principal	1. Le système affiche le formulaire de connexion. 2. L'utilisateur saisit son adresse e-mail et son mot de passe. 3. L'utilisateur valide la connexion. 4. Le système vérifie l'existence du compte. 5. Le système vérifie la validité du mot de passe. 6. Le système vérifie que le compte est actif. 7. Le système détermine le profil de l'utilisateur et récupère ses rôles et permissions. 8. Le système met à jour la date de dernière connexion. 9. Le système génère un jeton d'accès et un jeton de rafraîchissement. 10. Le système renvoie les informations de l'utilisateur connecté. 11. Le système redirige l'utilisateur vers la page d'accueil ou le tableau de bord.
Scénarios alternatifs	• L'utilisateur s'authentifie via Google. • L'utilisateur s'authentifie via GitHub.
Scénarios d'exception	• L'utilisateur saisit un identifiant inexistant. • L'utilisateur saisit un mot de passe incorrect. • Le compte de l'utilisateur est désactivé. • Le formulaire contient des champs obligatoires vides ou invalides. • Le compte est temporairement verrouillé après 3 tentatives échouées consécutives (durée : 1 minute).

TABLE III.2 – Description textuelle du cas d'utilisation « S'authentifier »

3 Description textuelle de cas d'utilisation « Changer mot de passe »

La figure ci-dessous présente la description textuelle du cas d'utilisation « Changer le mot de passe » :

Nom du cas d'utilisation	Changer mot de passe
Acteurs	Utilisateurs
Préconditions	L'utilisateur possède déjà un compte
Postconditions	Le mot de passe est réinitialisé
Scénario principal	1. Le système affiche un formulaire de changement de mot de passe. 2. L'utilisateur saisit son e-mail. 3. L'utilisateur clique sur le bouton « Réinitialiser le mot de passe ». 4. Le système vérifie l'existence de l'email. 5. Le système envoie un e-mail à l'utilisateur. 6. L'utilisateur clique sur le lien reçu par e-mail. 7. Le système affiche le formulaire de réinitialisation de mot de passe. 8. L'utilisateur saisit son nouveau mot de passe deux fois pour confirmation. 9. Le système valide et enregistre le nouveau mot de passe. 10. Le système envoie un e-mail de confirmation à l'utilisateur.
Scénarios d'exception	• L'utilisateur saisit un e-mail qui ne correspond à aucun compte existant. • L'utilisateur a rempli un champ invalide, ou a laissé un champ vide. • Le lien de réinitialisation est expiré. • L'utilisateur ne confirme pas correctement le mot de passe.

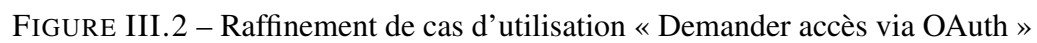
TABLE III.3 – Description textuelle de cas d'utilisation « Changer mot de passe »

4 Analyse de la fonctionnalité « Demander accès via OAuth »

Dans cette partie, nous allons présenter les cas d'utilisation raffinés relatifs à la demande d'accès via OAuth afin de comprendre les différentes étapes du processus d'inscription et d'authentification.

4.1 Raffinement de cas d'utilisation « Demander accès via OAuth »

Au cours de cette partie, nous allons analyser les sous-cas d'utilisation pour le processus d'authentification OAuth :



4.2 Description textuelle de cas d'utilisation « Soumettre demande d'inscription »

TABLE III.4 – Description textuelle « Soumettre demande d'inscription »

Nom du cas d'utilisation	Soumettre demande d'inscription
Acteurs	Visiteur
Préconditions	Le visiteur n'a pas de compte actif dans le système et a accès à la page d'authentification
Postconditions	La demande d'inscription est créée, enregistrée avec le statut <i>EN_ATTENTE</i> , et les administrateurs sont notifiés
Scénario principal	1. Le visiteur accède à la page de connexion / inscription. 2. Le visiteur sélectionne le rôle souhaité : Client, Développeur ou Chef de projet. 3. Le visiteur clique sur « Se connecter avec Google » ou « Se connecter avec GitHub ». 4. Le système redirige le visiteur vers le fournisseur OAuth sélectionné. 5. Le visiteur s'authentifie et autorise l'application à accéder à ses informations de base. 6. Le fournisseur OAuth redirige le visiteur vers l'application avec un code d'autorisation. 7. Le système récupère les informations du profil OAuth et vérifie s'il existe déjà un compte actif associé. 8. Si aucun compte actif n'existe, le système crée une demande d'inscription avec le statut <i>EN_ATTENTE</i>. 9. Le système envoie une notification interne et un e-mail à chaque administrateur actif pour l'informer de la nouvelle demande d'inscription. 10. Le système envoie un e-mail de confirmation au visiteur indiquant que sa demande a bien été reçue et est en cours d'examen. 11. Le système affiche un message d'attente à l'écran.
Scénarios alternatifs	• Compte actif déjà existant : le système connecte directement l'utilisateur sans créer de nouvelle demande. • Demande déjà soumise pour le même e-mail et le même fournisseur : le système conserve la demande existante et affiche un message d'information, sans créer de doublon. • Annulation de l'autorisation OAuth : le visiteur est redirigé vers la page d'authentification sans création de demande.
Scénarios d'exception	• Le fournisseur OAuth ne fournit pas l'adresse e-mail : le système affiche un message d'erreur et interrompt le traitement. • Le fournisseur OAuth est indisponible : le système affiche un message d'erreur technique. • Le visiteur refuse l'accès aux données OAuth : le système annule le flux et revient à l'écran d'authentification.

4.3 Description textuelle de cas d'utilisation « Valider demande »

TABLE III.5 – Description textuelle « Valider demande »

Nom du cas d'utilisation	Valider demande
Acteurs	Administrateur
Préconditions	L'administrateur est authentifié et possède la permission de traiter les demandes d'inscription
Postconditions	Le compte utilisateur est créé et activé, la demande est marquée <i>APPROUVEE</i> , et l'utilisateur est notifié
Scénario principal	1. L'administrateur accède au module de gestion des demandes d'inscription. 2. Le système affiche la liste des demandes en attente. 3. L'administrateur sélectionne une demande à traiter. 4. Le système affiche les informations du demandeur récupérées via OAuth. 5. L'administrateur clique sur « Approuver ». 6. Le système crée le compte utilisateur à partir des informations de la demande. 7. Le système attribue le rôle correspondant à l'utilisateur. 8. Le système marque la demande comme <i>APPROUVEE</i>. 9. Le système envoie un e-mail de confirmation à l'utilisateur. 10. Le système affiche un message de succès à l'administrateur.
Scénarios alternatifs	• L'administrateur clique sur « Refuser » : le système ouvre une boîte de dialogue pour saisir un motif de refus puis traite la demande comme rejetée. • Un compte avec le même e-mail existe déjà : le système signale le conflit et empêche la création d'un doublon. • La demande a déjà été traitée : le système affiche un message d'information et empêche une nouvelle validation.
Scénarios d'exception	• Erreur lors de la création du compte : le système annule l'opération et affiche un message d'erreur. • Erreur lors de l'envoi de l'e-mail : le compte peut être créé, mais le système journalise l'incident et affiche un avertissement. • Données de finalisation incomplètes : le système refuse la validation et demande de compléter les champs obligatoires.

5 Analyse de la fonctionnalité « Gérer utilisateurs »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la gestion des utilisateurs afin de comprendre les principales fonctionnalités du système, ce qui permet d'avoir une vue globale des interactions.

5.1 Raffinement de cas d'utilisation « Gérer utilisateurs »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des utilisateurs :

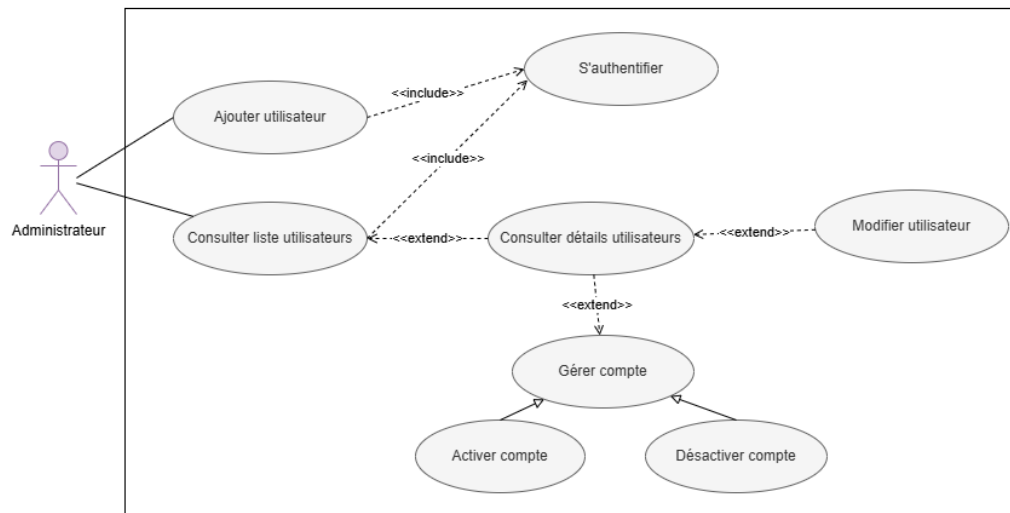


FIGURE III.3 – Diagramme des cas d'utilisation « Gérer utilisateurs »

5.2 Description textuelle de cas d'utilisation « Consulter liste utilisateurs »

TABLE III.6 – Description textuelle « Consulter liste utilisateurs »

Nom du cas d'utilisation	Consulter liste utilisateurs
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les utilisateurs
Postconditions	Liste des utilisateurs affichée
Scénario principal	1. L'administrateur accède à la section "Utilisateurs". 2. Le système affiche les utilisateurs sous forme de grille (par défaut) ou de liste selon le choix de l'utilisateur. 3. L'administrateur recherche un utilisateur en saisissant des informations clés dans la barre de recherche ou par des boutons de filtre (par statut, par rôle). 4. Le système filtre la liste.
Scénarios alternatifs	L'administrateur clique sur le bouton « Exporter ».

5.3 Description textuelle de cas d'utilisation « Ajouter utilisateur »

TABLE III.7 – Description textuelle « Ajouter utilisateur »

Nom du cas d'utilisation	Ajouter utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de créer des utilisateurs Liste des utilisateurs affichée
Postconditions	Utilisateur est ajouté
Scénario principal	1. L'administrateur clique sur le bouton "Nouvel Utilisateur". 2. Le système affiche le formulaire d'ajout. 3. L'administrateur remplit le formulaire. 4. L'administrateur valide le formulaire en cliquant sur (Créer le compte). 5. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur a rempli un champ invalide, ou a laissé un champ vide. • L'administrateur saisit un e-mail déjà utilisé dans un autre compte. • L'administrateur tente de fermer la fenêtre alors qu'il a commencé à saisir des données.
Scénarios alternatifs	Le système génère un mot de passe temporaire.

5.4 Description textuelle de cas d'utilisation « Désactiver un utilisateur »

TABLE III.8 – Description textuelle « Désactiver un utilisateur »

Nom du cas d'utilisation	Désactiver un utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de désactiver les utilisateurs Liste des utilisateurs affichée L'utilisateur à désactiver doit être actuellement Actif
Postconditions	Le compte est désactivé
Scénario principal	1. L'administrateur choisit un compte dans la liste. 2. L'administrateur clique sur l'action (Désactiver). 3. Le système vérifie si l'utilisateur possède des tickets actifs ou des demandes en attente. 4. Le système affiche une boîte de dialogue de confirmation. 5. L'administrateur confirme l'action. 6. Le système met à jour le statut de l'utilisateur. 7. Le système envoie un e-mail automatique à l'utilisateur pour l'informer de la suspension de son compte.
Scénarios alternatifs	L'administrateur réassigne les tickets ou les demandes.
Scénarios d'exception	• L'utilisateur tente de désactiver un utilisateur qui possède des tickets actifs ou des demandes en attente.

6 Analyse de la fonctionnalité « Gérer profil »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la gestion des profils afin de comprendre les principales fonctionnalités du système, ce qui permet d'avoir une vue globale des interactions.

6.1 Description textuelle de cas d'utilisation « Consulter Profil »

TABLE III.9 – Description textuelle « Consulter profil »

Nom du cas d'utilisation	Consulter profil
Acteurs	Utilisateur
Préconditions	L'utilisateur est authentifié
Postconditions	Le profil choisi est consulté
Scénario principal pour consulter son propre profil	1. L'utilisateur clique sur (Voir le profil). 2. Le système affiche la page personnelle. 3. L'utilisateur consulte ses données et son historique de modification.
Consulter le profil d'un utilisateur	1. L'administrateur choisit un utilisateur dans la liste puis clique sur (Voir profil). 2. Le système affiche les informations de l'employé.

6.2 Description textuelle de cas d'utilisation « Modifier Profil »

TABLE III.10 – Description textuelle « Modifier le Profil »

Nom du cas d'utilisation	Modifier le Profil
Acteurs	Administrateur, Utilisateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de modifier son propre profil ou celui d'autres utilisateurs (selon le rôle)
Postconditions	Le Profil est modifié
Scénario principal pour Modifier le profil d'un utilisateur	1. L'administrateur accède au profil de l'utilisateur. 2. L'administrateur modifie les accès en ajoutant et retirant des rôles système. 3. L'administrateur change le statut du compte (désactiver ou activer). 4. Le système valide chaque action.
Scénario principal pour Modifier son propre profil	1. L'utilisateur accède à sa page "Mon profil". 2. L'utilisateur met à jour sa photo de profil ou ses coordonnées. 3. L'utilisateur change son mot de passe en saisissant son ancien mot de passe pour validation. 4. L'utilisateur ajoute ou définit un nouvel e-mail et choisit lequel serait le principal. 5. Le système enregistre les changements.
Scénarios d'exception	• L'administrateur veut retirer le dernier rôle. • L'utilisateur veut modifier ses propres rôles.

7 Analyse de la fonctionnalité « Gérer rôles »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la gestion des rôles afin de comprendre les principales fonctionnalités du système, ce qui permet d'avoir une vue globale des interactions.

7.1 Raffinement de cas d'utilisation « Gérer rôles »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des rôles :

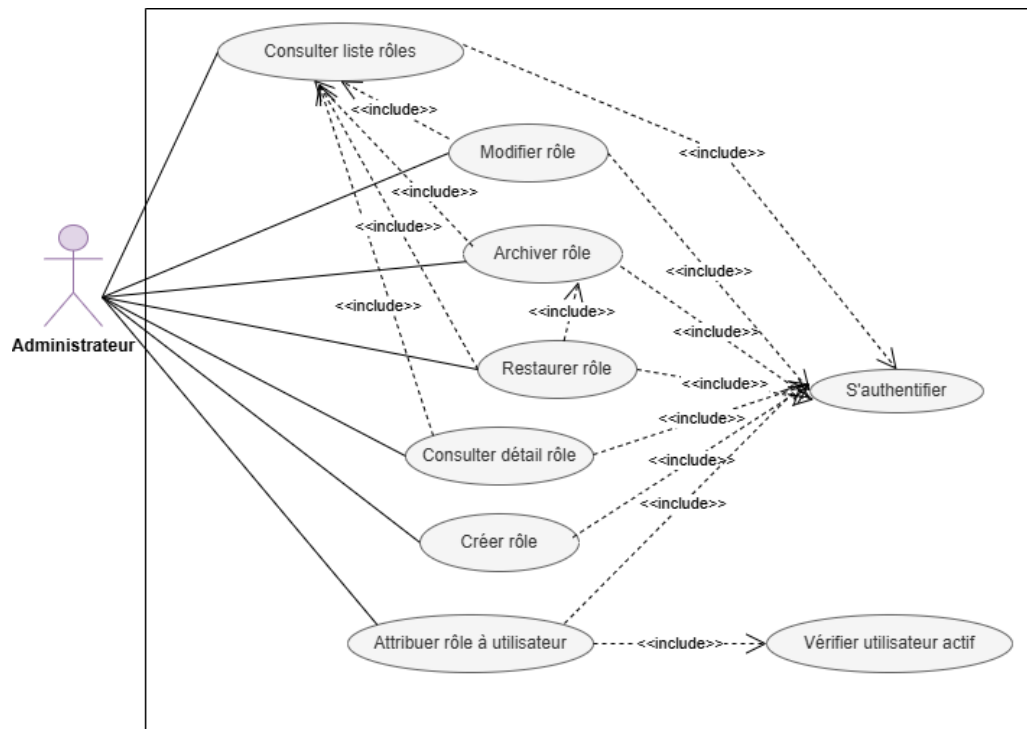


FIGURE III.4 – Raffinement de cas d'utilisation « Gérer rôles »

7.2 Description textuelle de cas d'utilisation « Archiver rôle »

TABLE III.11 – Description textuelle « Archiver rôle »

Nom du cas d'utilisation	Archiver rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission d'archiver les rôles Le rôle ne doit pas être assigné à des utilisateurs
Postconditions	Le rôle est archivé
Scénario principal	1. L'administrateur sélectionne un rôle dans la liste. 2. L'administrateur clique sur l'action (Archiver). 3. Le système vérifie que le rôle n'est pas assigné à des utilisateurs. 4. Le système affiche une confirmation. 5. L'administrateur confirme l'action. 6. Le système marque le rôle comme archivé. 7. Le système affiche le message de succès.
Scénarios d'exception	Le rôle est encore assigné à des utilisateurs.
Scénarios alternatifs	L'administrateur restaure un rôle archivé.

7.3 Description textuelle de cas d'utilisation « Attribuer rôle à utilisateur »

TABLE III.12 – Description textuelle « Attribuer rôle à utilisateur »

Nom du cas d'utilisation	Attribuer rôle à utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission d'attribuer des rôles Utilisateur et rôle existent
Postconditions	Le rôle est attribué à l'utilisateur
Scénario principal	1. L'administrateur consulte le profil d'un utilisateur. 2. L'administrateur clique sur "Ajouter un rôle". 3. Le système affiche la liste des rôles disponibles. 4. L'administrateur sélectionne un rôle. 5. L'administrateur valide la sélection. 6. Le système vérifie les règles d'exclusivité métier : – Un utilisateur ayant le rôle CLIENT ne peut pas recevoir les rôles CHEF_PROJET ou DEVELOPPEUR. – Un utilisateur ayant le rôle CLIENT ne peut pas recevoir le rôle ADMIN. – La combinaison ADMIN + CHEF_PROJET est autorisée. 7. Le système crée l'attribution. 8. Le système affiche le message de succès.
Scénarios alternatifs	L'administrateur peut aussi attribuer le rôle depuis les détails du rôle en sélectionnant un ou plusieurs utilisateurs à qui l'attribuer.
Scénarios d'exception	• Le rôle est déjà attribué à cet utilisateur. • Violation des règles d'exclusivité des rôles.

7.4 Description textuelle de cas d'utilisation « Retirer rôle d'un utilisateur »

TABLE III.13 – Description textuelle « Retirer rôle utilisateur »

Nom du cas d'utilisation	Retirer rôle utilisateur
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de retirer des rôles Le rôle est attribué à l'utilisateur
Postconditions	Le rôle est retiré de l'utilisateur
Scénario principal	1. L'administrateur consulte le profil d'un utilisateur. 2. L'administrateur visualise les rôles actuels de l'utilisateur. 3. L'administrateur clique sur l'icône de suppression d'un rôle. 4. Le système affiche une confirmation. 5. L'administrateur confirme l'action. 6. Le système retire le rôle. 7. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur tente de retirer le dernier rôle de l'utilisateur. • Le rôle n'est pas attribué à cet utilisateur.

8 Analyse de la fonctionnalité « Gérer permissions »

8.1 Raffinement de cas d'utilisation « Gérer permissions »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des permissions :

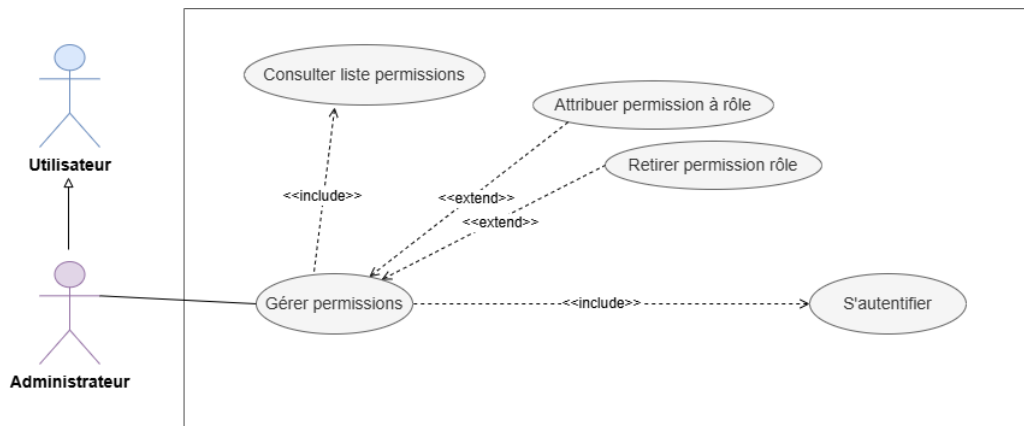


FIGURE III.5 – Raffinement de cas d'utilisation « Gérer permissions »

8.2 Description textuelle de cas d'utilisation « Consulter liste des permissions »

TABLE III.14 – Description textuelle « Consulter liste des permissions »

Nom du cas d'utilisation	Consulter liste des permissions
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les permissions
Postconditions	Liste des permissions affichée
Scénario principal	1. L'administrateur accède à la section "Permissions". 2. Le système affiche une matrice avec les rôles en horizontal et les catégories de permissions en vertical. 3. L'administrateur peut visualiser quelles permissions sont attribuées à chaque rôle (cases cochées). 4. L'administrateur peut filtrer par catégorie de permissions ou par rôle.
Scénarios alternatifs	• L'administrateur sélectionne la vue "Par rôle" pour afficher une liste des rôles avec leurs permissions respectives organisées par catégorie. • L'administrateur clique sur le bouton « Exporter » pour télécharger la matrice des permissions.

8.3 Description textuelle de cas d'utilisation « Attribuer permission à rôle »

TABLE III.15 – Description textuelle « Attribuer permission à rôle »

Nom du cas d'utilisation	Attribuer permission à rôle
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission d'attribuer des permissions aux rôles Rôle et permission existent
Postconditions	La permission est ajoutée au rôle
Scénario principal	1. L'administrateur consulte un rôle spécifique. 2. L'administrateur clique sur l'onglet "Permissions". 3. Le système affiche toutes les permissions organisées par catégorie avec des cases à cocher. 4. L'administrateur coche les permissions à attribuer. 5. L'administrateur clique sur "Enregistrer". 6. Le système crée les liaisons. 7. Le système affiche le message de succès.
Scénarios alternatifs	L'administrateur utilise la matrice des permissions pour cocher les cases correspondant aux permissions à attribuer pour plusieurs rôles simultanément.
Scénarios d'exception	La permission est déjà attribuée au rôle.

III Diagramme de classes de première sprint

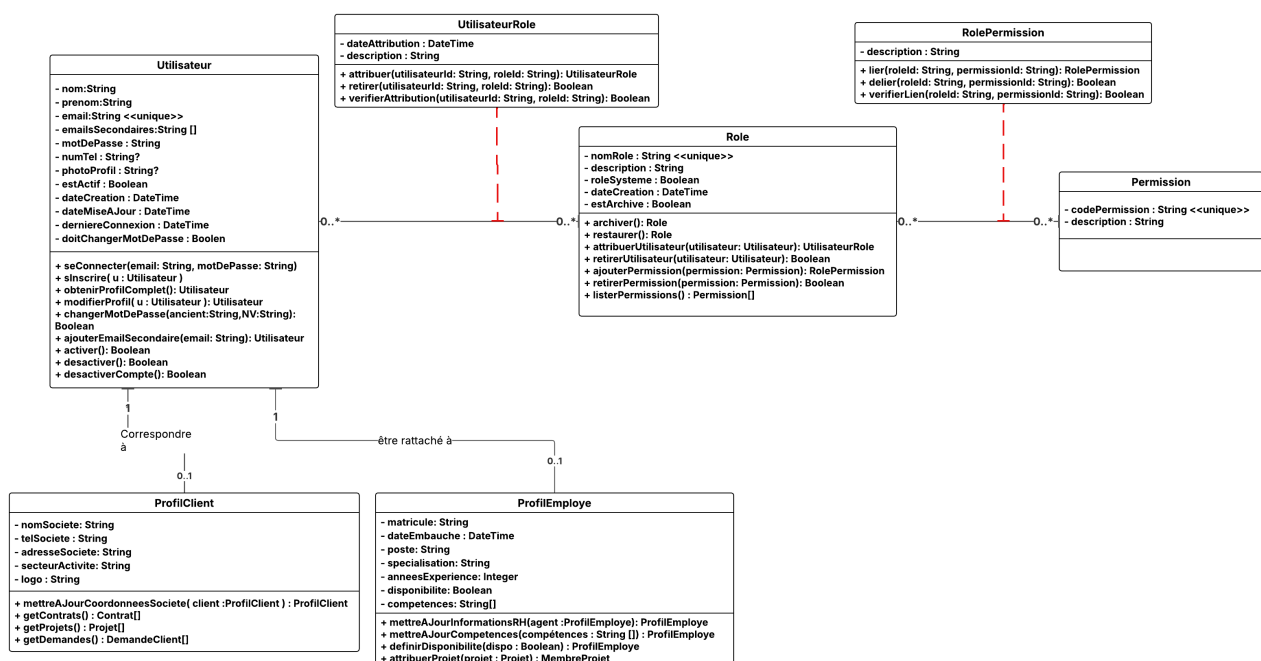


FIGURE III.6 – Diagramme de classes du Sprint 1

IV Diagramme de séquence

1 Diagramme de séquences du cas d'utilisation « S'authentifier »

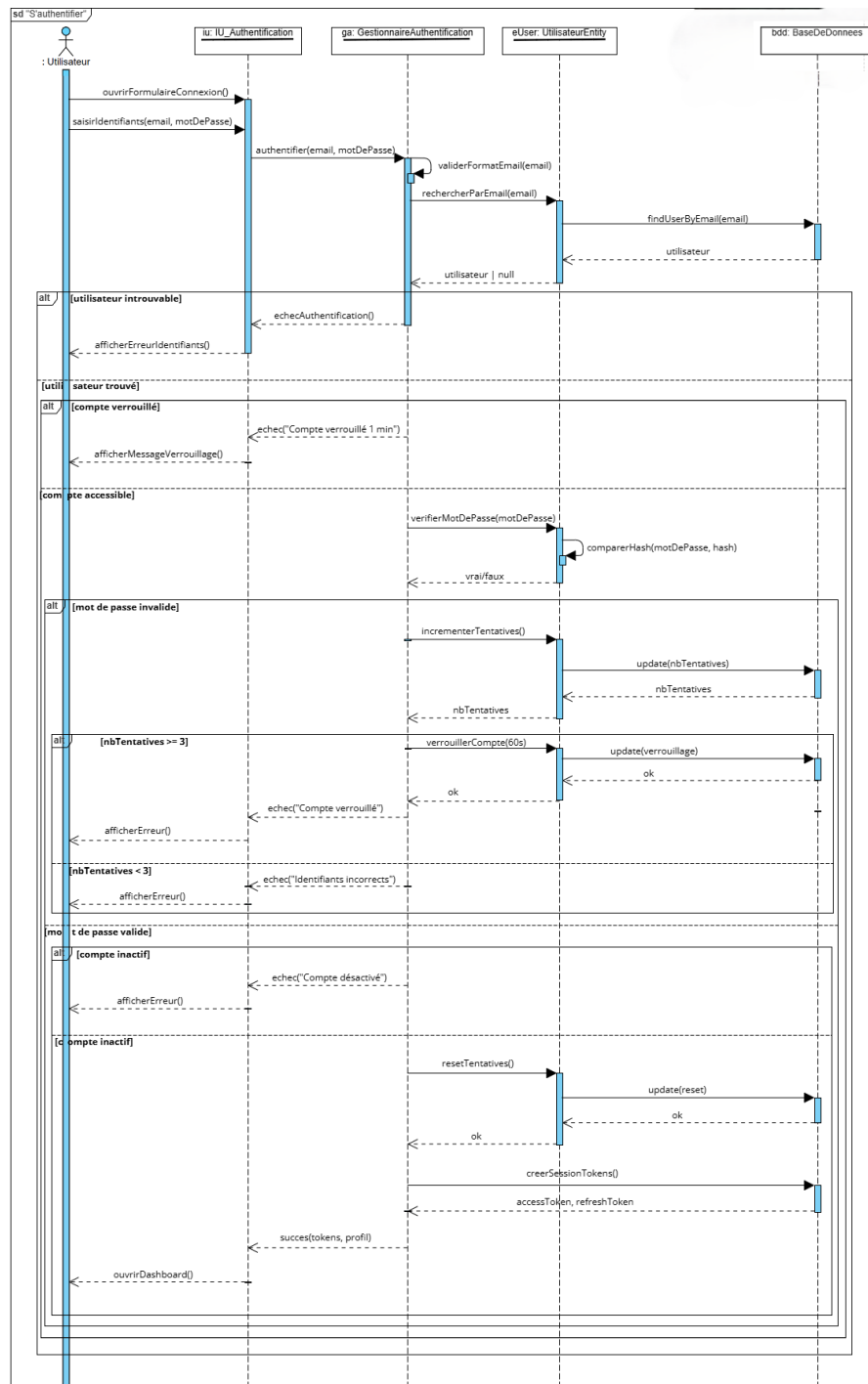


FIGURE III.7 – Diagramme de séquence du cas d'utilisation « S'authentifier »

2 Diagramme de séquences du cas d'utilisation « Mot de passe oublié »

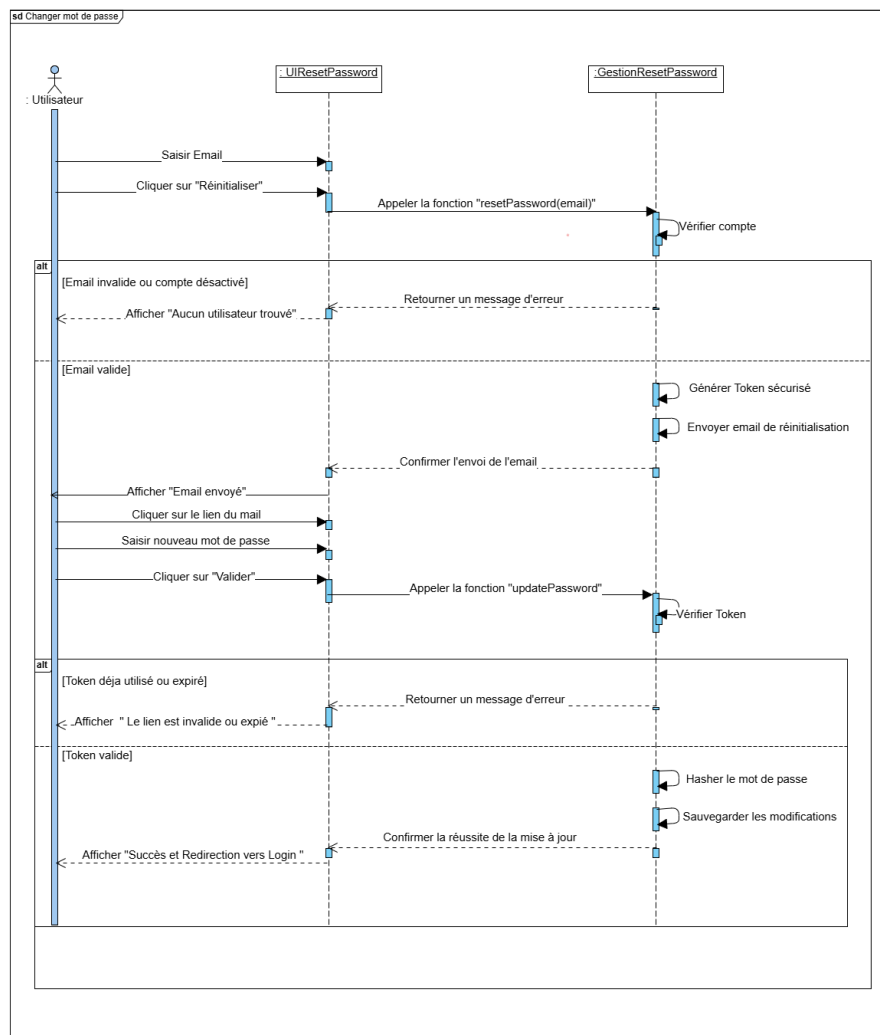


FIGURE III.8 – Diagramme de séquence du cas d'utilisation « Mot de passe oublié »

3 Diagramme de séquences du cas d'utilisation « Désactiver utilisateur »

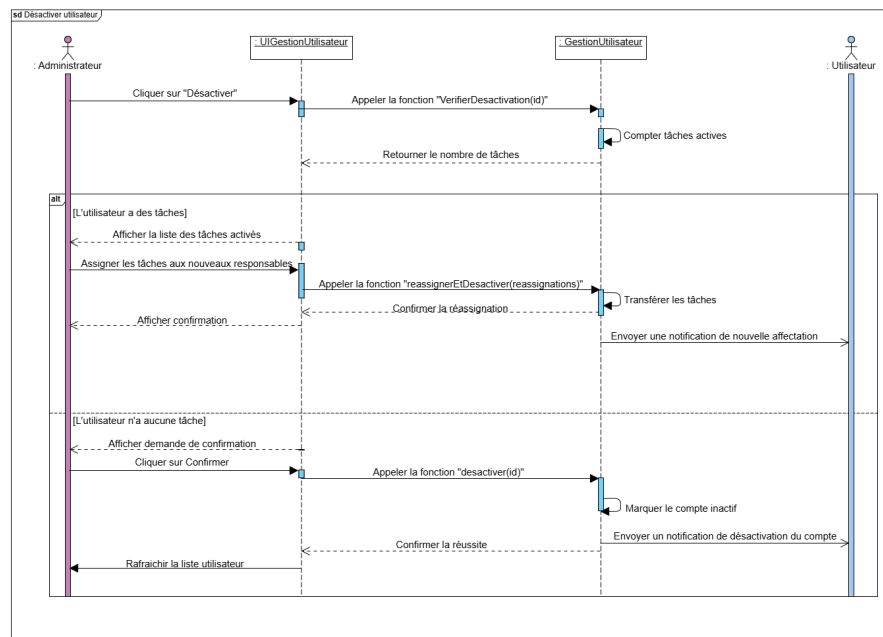


FIGURE III.9 – Diagramme de séquence du cas d'utilisation « Désactiver utilisateur »

4 Diagramme d'activité « Authentification et demande d'accès via OAuth »

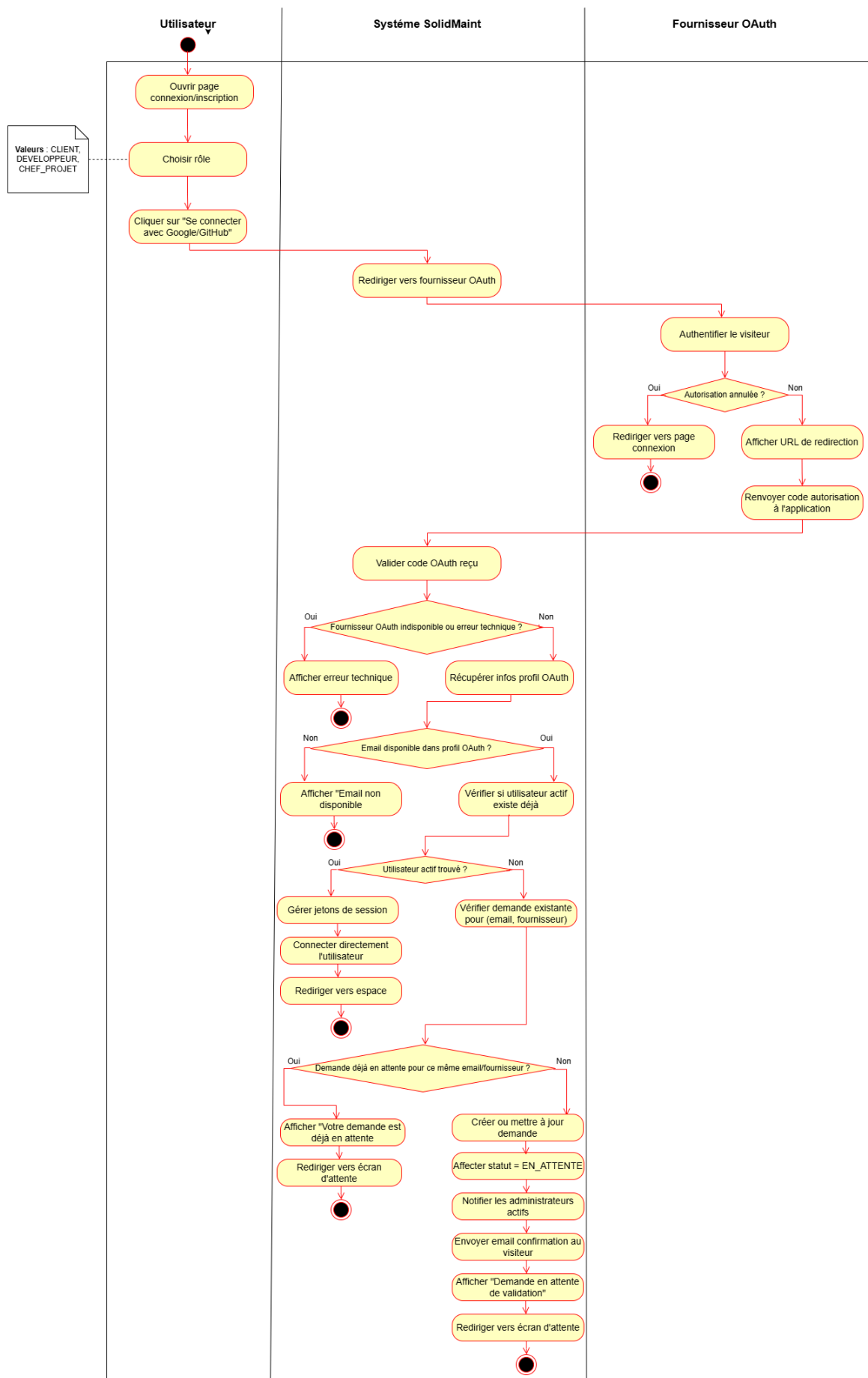


FIGURE III.10 – Diagramme d'activité d'authentification et demande d'accès via OAuth

V Schéma relationnel

Utilisateur (idUtilisateur, nom, prenom, email, motDePasse, estActif, doitChangerMotDePasse, tentativesConnexionEchouees, compteVerrouilleJusquA, photoProfil, dateCreation)

ProfilEmploye (idProfilE, #**idUtilisateur**, matricule, poste, specialisation, disponibilite, dateEmbauche, niveauSeniorite)

ProfilClient (idProfilC, #**idUtilisateur**, nomSociete, adresseSociete, secteurActivite, telSociete, logo)

Role (idRole, nomRole, description, roleSysteme, estArchive, dateCreation)

Permission (idPermission, codePermission, description)

UtilisateurRole (idUtilisateurRole, #**idUtilisateur**, #**idRole**, dateAttribution)

RolePermission (idRolePermission, #**idRole**, #**idPermission**)

DemandeInscriptionOAuth (idDemande, email, prenom, nom, fournisseur, photoProfil, role, statut, dateCreation, motifRefus)

VI Réalisation

Après avoir analysé les besoins et conçu les solutions, nous passons maintenant à la réalisation. Dans cette phase, nous avons développé toutes les user stories planifiées dans le sprint backlog. Cette section présente le diagramme de composants qui montre l'architecture de notre application, ainsi que les interfaces utilisateur que nous avons créées pour répondre aux besoins des utilisateurs.

1 Diagramme de composants

2 Description des interfaces

Cette section présente les principales interfaces utilisateur développées lors du premier sprint.

2.1 Interface de connexion

FIGURE III.11 – Interface de connexion — authentification classique et OAuth

La page de connexion propose deux modes d'authentification : le formulaire classique (e-mail + mot de passe) et les boutons OAuth (Google, GitHub). En cas de compte inexistant, le visiteur est redirigé vers un flux de demande d'inscription.

2.2 Interface de réinitialisation du mot de passe

FIGURE III.12 – Interface de réinitialisation du mot de passe

Cette interface permet à l'utilisateur de saisir son adresse e-mail pour recevoir un lien de réinitialisation valable 1 heure.

2.3 Interface de gestion des utilisateurs

FIGURE III.13 – Interface d'administration — liste des utilisateurs

L'administrateur dispose d'une vue en grille ou en liste, avec filtres par statut et par rôle, barre de recherche et actions rapides (modifier, désactiver, voir le profil).

2.4 Interface de gestion des rôles et permissions

FIGURE III.14 – Interface d'administration — gestion des rôles et permissions

La matrice des permissions affiche les rôles en colonnes et les catégories de permissions en lignes, avec des cases à cocher pour une attribution visuelle et intuitive.

2.5 Interface de gestion des demandes d'inscription OAuth

FIGURE III.15 – Interface d'administration — demandes d'inscription OAuth

L'administrateur consulte les demandes en attente, visualise les informations récupérées via OAuth et peut approuver ou refuser chaque demande avec un motif de refus facultatif.

VII Tests et Validation

La phase de test est essentielle pour garantir la qualité et la fiabilité de la solution. Elle valide que chaque fonctionnalité répond aux exigences métier et identifie les anomalies avant la mise en production. Dans ce sprint, nous avons testé les mécanismes d'authentification, la gestion des utilisateurs et le contrôle d'accès (RBAC) pour assurer la robustesse de nos implémentations.

1 Stratégie de tests adoptée

Afin de garantir la qualité et la fiabilité de l'application, nous avons adopté une stratégie de tests structurée, reposant sur une répartition des vérifications à plusieurs niveaux. Cette approche permet d'assurer une bonne couverture fonctionnelle avec un temps d'exécution raisonnable, tout en facilitant l'identification rapide des anomalies.

Les tests ont été organisés par module fonctionnel afin d'assurer une meilleure traçabilité des vérifications effectuées. Ainsi, les tests unitaires couvrent les règles métier les plus fines, les tests d'intégration valident les échanges avec l'API, et les tests End-to-End reproduisent les scénarios complets d'utilisation de l'application.

La pyramide de tests se compose donc de trois niveaux complémentaires :

1.1 Tests unitaires

Les tests unitaires ont été réalisés à l'aide du framework **Vitest**, en isolant chaque service du reste de l'application grâce à des mocks Prisma. Cette approche garantit l'indépendance des tests et permet une détection précoce des anomalies dès la phase de développement. Trois modules prioritaires ont été couverts : l'authentification, le contrôle d'accès basé sur les rôles (RBAC) et la gestion des clients.

Bilan : 45 tests exécutés — 45 réussis — 0 échec.

□ Module Authentification

Méthodes : `login()` · `demanderReinitialisationMotDePasse()` · `reinitialiserMotDePasse()`

Cette suite valide le processus d'authentification classique, la génération des jetons JWT et le mécanisme de réinitialisation du mot de passe. Les cas suivants ont été couverts :

- Un utilisateur actif fournit des identifiants valides : les jetons `accessToken` et `refreshToken` sont générés correctement.
- L'adresse e-mail fournie n'existe pas en base de données : la connexion est rejetée avec le code 401 `Unauthorized`.
- Le mot de passe fourni est incorrect : la connexion est rejetée avec le code 401 `Unauthorized`.
- L'utilisateur échoue l'authentification **3 fois consécutives** : le compte est verrouillé automatiquement pendant **1 minute** (`maxLoginAttempts = 3`).
- L'utilisateur tente de se connecter alors que son compte est verrouillé : la connexion est rejetée avec un message explicite.
- Le compte de l'utilisateur est désactivé : la connexion est rejetée avec le code 401 `Unauthorized`.
- L'adresse e-mail fournie pour la réinitialisation est inexistante : une `NotFoundException` est levée.
- Le token de réinitialisation a été généré depuis plus d'une heure : une `BadRequestException` est levée.

```

PASS src/modules/auth/auth.service.spec.ts
AuthService - Réinitialisation Mot de Passe
  login
    ✓ devrait authentifier un utilisateur actif et générer les jetons (13 ms)
    ✓ devrait rejeter la connexion si l'utilisateur est introuvable (28 ms)
    ✓ devrait rejeter la connexion si le mot de passe est invalide (4 ms)
    ✓ devrait verrouiller le compte après 3 échecs (4 ms)
    ✓ devrait rejeter une connexion quand le compte est déjà verrouillé (6 ms)
    ✓ devrait rejeter la connexion si le compte est désactivé (3 ms)
  demanderReinitialisationMotDePasse
    ✓ devrait lever NotFoundException si l'email n'existe pas en base (3 ms)
  reinitialiserMotDePasse
    ✓ devrait lever BadRequestException si le token a expiré depuis plus d'1 heure (4 ms)

Test Suites: 1 passed, 1 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       1.368 s
Ran all test suites matching src/modules/auth/auth.service.spec.ts.

```

Auth

□ Module Gestion des rôles et permissions (RBAC)

Méthodes: `creerPermission()`, `attribuerRoleUtilisateur()`, `obtenirRolesEtPermissions()`

Cette suite valide les règles d'exclusivité des rôles, l'unicité des permissions et l'intégrité des références.

Les cas suivants ont été couverts :

- Une permission est créée avec un code unique : elle est enregistrée correctement en base de données.
- Une permission est créée avec un code déjà existant : une `BadRequestException` est levée.
- Les rôles et permissions d'un utilisateur sont récupérés : la liste est retournée sans doublons.
- Un rôle est attribué à un utilisateur sans rôle existant : l'attribution est réussie.
- L'utilisateur cible est inexistant : une `NotFoundException` est levée.
- La règle d'exclusivité `CLIENT` et `CHEF_PROJET` est violée : une `BadRequestException` est levée.
- La règle d'exclusivité `CLIENT` et `DEVELOPPEUR` est violée : une `BadRequestException` est levée.
- La règle d'exclusivité `CLIENT` et `ADMIN` est violée : une `BadRequestException` est levée.
- La combinaison `ADMIN` et `CHEF_PROJET` est soumise : l'attribution est **autorisée** et réussie.
- Le rôle est déjà attribué à l'utilisateur : une `BadRequestException` est levée (doublon détecté).
- Le rôle à attribuer est inexistant : une `NotFoundException` est levée.

```

PS C:\Users\Eya\OneDrive\Documents\projets\gestionsTickets\plateformwebgestiontickets\backend-nestjs> npx je
st src/modules/roles-permissions/roles-permissions.service.spec.ts --detectOpenHandles
  createPermission
    ✓ devrait lever BadRequestException si le rôle existe déjà (57 ms)
    ✓ devrait créer une permission quand codePermission est unique (14 ms)
    ✓ devrait lever BadRequestException si la permission existe déjà (22 ms)
  getUserRolesAndPermissions
    ✓ devrait retourner la liste des rôles et des permissions sans doublons (11 ms)
  assignRoleToUser
    ✓ devrait attribuer un rôle à un utilisateur (11 ms)
    ✓ devrait lever NotFoundException si utilisateur introuvable (8 ms)
    ✓ devrait lever BadRequestException si règle exclusivité non respectée (CLIENT + CHEF_PROJET) (29 ms)
    ✓ devrait lever BadRequestException si le rôle est déjà attribué (11 ms)

Test Suites: 1 passed, 1 total
Tests:      11 passed, 11 total
Snapshots:  0 total
Time:       5.57 s
Ran all test suites matching src/modules/roles-permissions/roles-permissions.service.spec.ts.
PS C:\Users\Eya\OneDrive\Documents\projets\gestionsTickets\plateformwebgestiontickets\backend-nestjs>

```

RBAC

FIGURE III.16 – Résultats des tests unitaires — Auth et RBAC

1.2 Tests d'intégration

Les tests d'intégration ont pour objectif de valider les échanges entre les différentes couches de l'application (contrôleur, service et base de données) en testant les endpoints REST dans leur ensemble. Ces tests sont effectués via **Swagger UI** en soumettant des requêtes HTTP réelles contre le backend en cours d'exécution, ce qui permet de vérifier le comportement de l'API dans des conditions proches de la production.

□ Authentification

Endpoint : POST /api/auth/login

Cette suite valide le comportement complet de l'endpoint de connexion, notamment le retour des jetons d'authentification, des rôles, des permissions associées et la gestion des codes d'erreur HTTP selon les différents cas d'entrée.

Les cas suivants ont été couverts :

- Un utilisateur actif de rôle ADMIN fournit des identifiants valides : l'endpoint retourne le code **200 OK** accompagné des champs `accessToken`, `refreshToken`, des données de l'utilisateur, de ses rôles et de la liste complète de ses permissions.
- L'adresse e-mail fournie n'existe pas en base de données : l'endpoint retourne le code **401 Unauthorized**.
- Le mot de passe fourni est incorrect : l'endpoint retourne le code **401 Unauthorized**.
- Les données soumises sont invalides (adresse e-mail mal formatée ou champ obligatoire vide) : l'endpoint retourne le code **400 Bad Request** accompagné d'un message de validation généré par `class-validator`.
- Le compte de l'utilisateur est désactivé (`estActif = false`) : l'endpoint retourne le code **401 Unauthorized** avec le message « Ce compte est désactivé ».

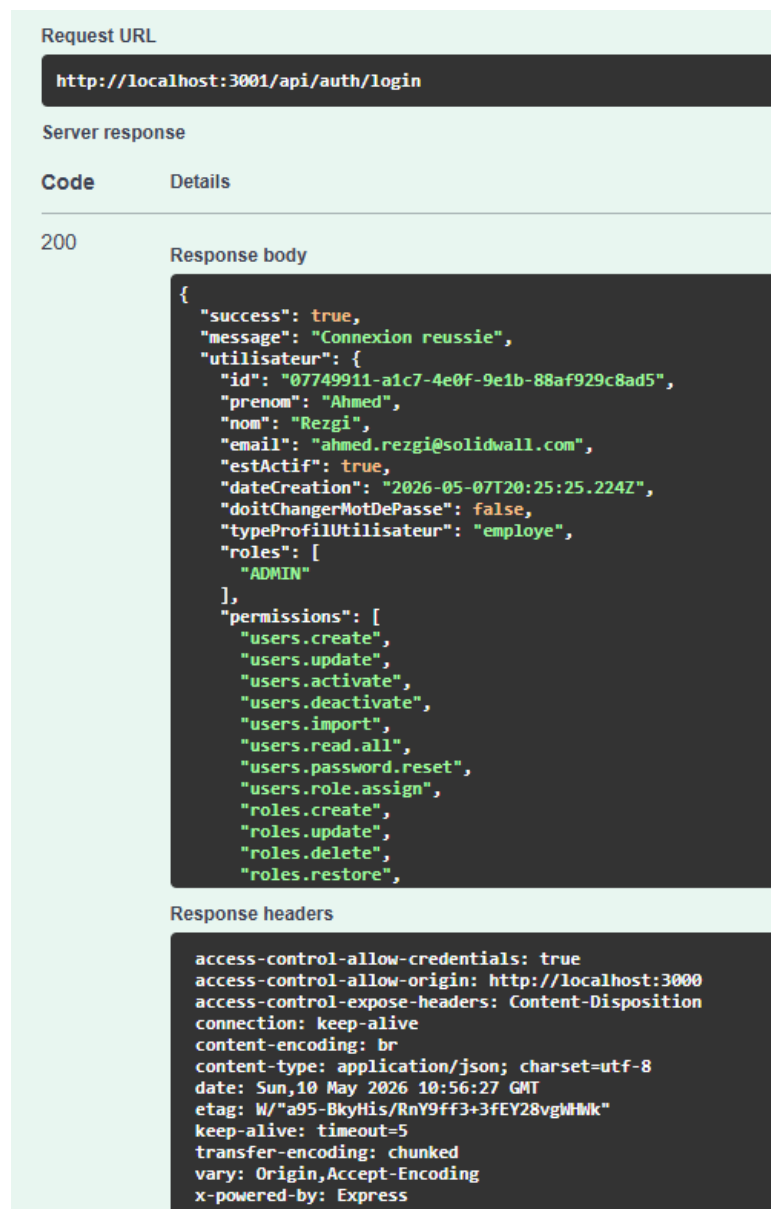


FIGURE III.17 – Test d'intégration — endpoint POST /api/auth/login via Swagger

□ Gestion du profil

Endpoint : *PUT /api/profil*

Cette suite de tests vise à valider le bon fonctionnement de la méthode **modifierProfil()** exposée via l'endpoint **PUT /api/profil**, qui permet à un utilisateur de mettre à jour les informations de son profil.

Les cas suivants ont été couverts :

- Cas où l'e-mail fourni dans la requête ne respecte pas le format attendu.



FIGURE III.18 – Test de mise à jour du profil

1.3 Tests End-to-End

Les tests End-to-End ont pour objectif de simuler les parcours utilisateurs complets dans un navigateur réel piloté par **Playwright**. Ces tests s'exécutent en interaction avec le frontend Next.js, le backend NestJS et la base de données PostgreSQL, reproduisant ainsi les conditions réelles d'utilisation de la plateforme.

Bilan : 11 tests exécutés — 11 réussis — 0 échec.

□ Authentification

10 scénarios : connexion classique, OAuth, déconnexion

Cette suite valide le flux complet d'authentification en simulant les actions réelles d'un utilisateur sur l'interface.

Les cas suivants ont été couverts :

- La page de connexion est chargée : le formulaire et les boutons d'authentification OAuth sont affichés correctement.
- Les champs du formulaire sont vides ou partiellement remplis : le bouton de soumission reste désactivé.
- L'utilisateur soumet des identifiants valides : le flux aboutit à une redirection vers la page d'accueil et une session est créée.
- L'utilisateur saisit un identifiant ou un mot de passe invalide : un message d'erreur approprié est affiché sans redirection.
- L'utilisateur échoue l'authentification **3 fois consécutives** : le compte est verrouillé pendant **1 minute** et un message d'information est affiché.
- L'utilisateur se déconnecte : les jetons sont supprimés du `localStorage` et l'utilisateur est redirigé vers la page `/connexion`.
- L'utilisateur s'authentifie via **Google** avec succès : les données du profil sont récupérées et une session valide est créée.
- L'authentification via **Google** échoue : un message d'erreur technique est affiché.
- L'utilisateur s'authentifie via **GitHub** avec succès : les données du profil sont récupérées et une session valide est créée.
- L'authentification via **GitHub** échoue : un message d'erreur technique est affiché.

```
C:\Users\Eya\OneDrive\Documents\projets\gestionTickets\plateformewebgestiontickets\e2e-tests>.\run-login-tests.cmd
Running 10 tests using 1 worker
✓ 1 login.edge.test.js:33:3 › Login - Page de connexion › Affiche le formulaire de connexion (5.2s)
✓ 2 login.edge.test.js:41:3 › Login - Page de connexion › Champs vides ou partiels : bouton de soumission désactivé (5.4s)
✓ 3 login.edge.test.js:63:3 › Login - Page de connexion › Login valide (Flux complet) (9.2s)
✓ 4 login.edge.test.js:73:3 › Login - Page de connexion › Login invalide (mauvais identifiants) (6.3s)
✓ 5 login.edge.test.js:83:3 › Login - Page de connexion › Verrouille après 5 échecs (7.2s)
✓ 6 login.edge.test.js:97:3 › Login - Page de connexion › Déconnexion : supprime les tokens et redirige (2.3s)
✓ 7 login.edge.test.js:121:3 › Login - Page de connexion › Connexion Google (succès) (2.3s)
✓ 8 login.edge.test.js:135:3 › Login - Page de connexion › Connexion Google (échec) (4.9s)
✓ 9 login.edge.test.js:141:3 › Login - Page de connexion › Connexion GitHub (succès) (2.2s)
✓ 10 login.edge.test.js:155:3 › Login - Page de connexion › Connexion GitHub (échec) (5.0s)
10 passed (55.2s)
```

FIGURE III.19 – Tests End-to-End — Authentification via Playwright

□ Gestion des utilisateurs

1 scénario : désactivation avec réassignation des tickets

Cette suite valide le flux complet de désactivation d'un utilisateur disposant de tickets actifs, en simulant les actions réelles d'un administrateur. Le cas suivant a été couvert :

- L'administrateur tente de désactiver un utilisateur actif disposant de **9 tickets actifs** (statuts OUVERT, EN_COURS ou EN_REVUE) : le système affiche le modal de réassignation présentant la liste des développeurs disponibles **triés par charge de travail croissante**. L'administrateur procède à la réassignation de chaque ticket, puis confirme l'opération via le bouton « Réassigner et Désactiver ». Le système effectue les réassignations dans une transaction atomique, désactive le compte et envoie un e-mail de notification à l'utilisateur concerné.

```

Recherche utilisateurs: 8 cartes, 0 lignes
✓ Trouvé 8 utilisateurs (8 cartes, 0 lignes)
✓ Utilisateur actif trouvé dans la carte 0 (peut avoir des tickets/demandes)
✓ Bouton menu trouvé, clic...
✓ Option "Désactiver" trouvée, clic...
# API Response: https://localhost:3001/api/users/6942a9c9-dc15-4743-9430-b56ea092d75b/demandes-reassignables - Status: 200
  Response body: {"demandes": [], "chefsProjets": [{"id": "16e04f83-f329-4415-99f7-83b7d1a0dbb9", "prenom": "Karim", "nom": "Ben Ali", "photoProfil": "https://testingbot.com/free-
-online-tools/random-avatar/240?u=karim-benali-ch
# API Response: https://localhost:3001/api/users/6942a9c9-dc15-4743-9430-b56ea092d75b/tickets-reassignables - Status: 200
  Response body: {"tickets": [{"assignmentId": "4770951b-7b67-42c2-a4f2-47f988556a04", "ticketId": "3c8773d8-4aaa-4070-bc3-c1e223498607", "titre": "Interface lente sur mobil
e", "statut": "OUVERT", "priorite": "HAUTEUR", "dateCre
  Modal détecté
  Elements modaux fixes trouvés: 1
  Nombre de dialogs trouvés: 1
  Contenu du dialog: Réassignation des Tickets
  Sofia Mansour
  Cet utilisateur a 9 ticket(s) actif(s). Réassignez chaque ticket avant de désactiver le compte.
  Interface lente sur mobile
  Application Mobile Beta Services
  ▲ Modal de réassignation détecté - utilisateur a des tickets/demandes en cours
  ■ Tentative de compléter la réassignation pour continuer le test...
  ■ 9 ticket(s)/demande(s) à réassigner
  ✓ Assignment du ticket/demande 1 au premier développeur/chef
  ✓ Assignment du ticket/demande 2 au premier développeur/chef
  ✓ Assignment du ticket/demande 3 au premier développeur/chef
  ✓ Assignment du ticket/demande 4 au premier développeur/chef
  ✓ Assignment du ticket/demande 5 au premier développeur/chef
  ✓ Assignment du ticket/demande 6 au premier développeur/chef
  ✓ Assignment du ticket/demande 7 au premier développeur/chef
  ✓ Assignment du ticket/demande 8 au premier développeur/chef
  ✓ Assignment du ticket/demande 9 au premier développeur/chef
  ■ Compte: 9/9 tickets assignés
  ✓ Bouton activé
  ✓ Clic sur "Réassigner & Désactiver"
  ✓ Réassignation et désactivation complétées avec succès
  ok | ...vation-complete.e2e.test.ts:64:9 › Activation/Désactivation utilisateur - COMPLET › CAS 1: Désactiver un utilisateur actif via le menu avec confirmation (25.4s)

```

FIGURE III.20 – Tests End-to-End — Désactivation utilisateur via Playwright



CHAPITRE 4

Réalisation



Sommaire

4

I	Sprint Planning Meeting	96
II	Analyse	106
III	Diagramme de classes du deuxième sprint	114
IV	Diagramme de séquence	114
V	Schéma relationnel	116
VI	Réalisation	117
VII	Tests et Validation	118

Introduction

Dans le chapitre précédent, nous avons mis en place les fondations du système : l’authentification, la gestion des utilisateurs avec leurs rôles et permissions, ainsi que le traitement des demandes d’inscription via OAuth. L’ensemble de ces fonctionnalités a été testé et validé.

Dans ce chapitre, nous abordons le cœur métier de la plateforme. Il s’agit principalement de la gestion des clients, des contrats de maintenance avec leurs niveaux de service (SLA), de la gestion des projets et de leurs équipes, ainsi que de la traçabilité complète des opérations à travers le journal d’activités.

I Sprint Planning Meeting

La réunion de planification du deuxième sprint nous a permis de définir les objectifs à atteindre durant les trois prochaines semaines. Nous avons sélectionné les user stories prioritaires et estimé l’effort nécessaire pour chacune d’elles. Cette étape est importante pour que toute l’équipe comprenne bien le travail à réaliser.

1 Objectif du Sprint

Ce sprint vise à développer les fonctionnalités essentielles du système. L’objectif principal est de mettre en place la gestion complète des clients et des contrats de maintenance, d’assurer le suivi des engagements de service à travers leurs SLA, de faciliter la collaboration via l’affectation des équipes aux projets, et de garantir la traçabilité complète des opérations effectuées à travers le journal d’activités.

1.1 Backlog du deuxième sprint

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l’effort estimé pour chaque user story.

1 : Effort faible, 2 : Effort moyen, 3 : Effort élevé

ID US	User Story	Description technique	Effort
Epic 5 - Gestion des Clients			

US17	En tant qu' administrateur , je souhaite créer un profil client afin de centraliser ses informations.	• Développer l'endpoint "POST /api/clients" pour créer simultanément le compte utilisateur (Utilisateur) et le profil entreprise (ProfilClient) • Implémenter le service de récupération automatique de logo (LogoService) à partir du nom de la société, avec upload manuel via "POST /api/clients/ :id/logo" (traitement Sharp, formats JPG/PNG/GIF/WEBP, 2 Mo max) • Intégrer le service de géocodage (GeocodingService) via "GET /api/clients/geocode?q=" pour l'auto-complétion d'adresse (ArcGIS, rate limit 20 req/min via @nestjs/throttler) • Réaliser le formulaire de création côté Next.js	1
US18	En tant qu'administrateur, je souhaite consulter la liste des clients afin de visualiser les clients existants.	• Développer l'endpoint "GET /api/clients" avec filtrage RBAC selon le rôle de l'utilisateur connecté (userId + userRoles passés au service) • Développer les endpoints de données enrichies par client : "GET /api/clients/ :id/projets", "GET /api/clients/ :id/contrats", "GET /api/clients/ :id/demandes", "GET /api/clients/ :id/tickets" • Réaliser l'interface de visualisation avec vue grille et vue liste côté Next.js	2

US19	En tant qu' administrateur , je souhaite consulter le profil détaillé d'un client afin de visualiser toutes ses informations.	• Développer l'endpoint "GET /api/clients/ :id" pour récupérer les informations complètes du client • Développer l'endpoint "GET /api/clients/ :id/business-overview" pour la vue métier consolidée (projets contextuels, contrats, demandes) • Développer l'endpoint "GET /api/clients/ :id/verifier-desactivation" pour vérifier les conditions avant désactivation • Réaliser l'interface de profil structurée en onglets côté Next.js	1
US20	En tant qu' administrateur , je souhaite modifier les informations d'un client afin de maintenir des données à jour.	• Développer l'endpoint "PUT /api/clients/ :id" pour la mise à jour des informations du client et de son profil entreprise • Valider les formats de données côté serveur via class-validator • Gérer la mise à jour du logo via "POST /api/clients/ :id/logo" et la suppression via "DELETE /api/clients/ :id/logo" • Réaliser l'interface d'édition côté Next.js	2

US21	En tant qu' administrateur , je souhaite activer et désactiver un compte client afin de contrôler l'accès.	• Développer l'endpoint "PATCH /api/clients/ :id/desactiver" et "PATCH /api/clients/ :id/reactiver" • Implémenter la vérification préalable via "GET /api/clients/ :id/verifier-desactivation" pour bloquer l'opération si le client possède des contrats actifs, des demandes en attente ou des tickets non résolus • Intégrer EmailService pour l'envoi automatique de notifications de changement de statut • Réaliser les boutons d'action avec modale de confirmation côté Next.js	3
US22	En tant qu' administrateur , je souhaite imprimer l'annuaire d'un client afin de disposer d'une version papier.	• Implémenter l'export CSV des données client (informations société, contrats, projets) via le composant ExportCSV côté frontend • Utiliser la bibliothèque xlsx pour la génération des fichiers tabulaires • Réaliser le bouton d'export dans l'interface de liste des clients	1
US23	En tant qu' administrateur , je souhaite créer un projet rattaché à un client afin d'organiser les interventions.	• Développer l'endpoint "POST /api/projets" avec validation des données via class-validator • Gérer la relation projet-client (ProfilClient) et l'attribution optionnelle d'un chef de projet (ProfilEmploye) • Envoyer des notifications aux membres de l'équipe lors de la création • Réaliser le formulaire de création côté Next.js	3

US24	En tant qu' administrateur , je souhaite consulter la liste des projets afin de piloter l'activité.	• Développer l'endpoint "GET /api/projets" (permissions projects.read.all ou projects.read.own) avec filtres par search, tags, statutContrat et statutP • Appliquer le contrôle d'accès RBAC selon le rôle : ADMIN voit tous les projets, CHEF_PROJET voit ses projets, DEVELOPPEUR voit ses projets affectés, CLIENT voit ses propres projets • Réaliser l'interface avec tableau et filtres avancés côté Next.js	2
US25	En tant qu' administrateur , je souhaite affecter des développeurs à un projet afin de constituer l'équipe.	• Développer les endpoints de gestion d'équipe : "POST /api/projets/ :id/equipe" (ajouter), "PUT /api/projets/ :id/equipe/ :employeId" (modifier le rôle), "DELETE /api/projets/ :id/equipe/ :employeId" (retirer) — / teams.developer.remove • Implémenter le workflow de proposition/validation : "POST /api/projets/ :id/assigner", "POST /api/projets/affectation/ :id/valider", "POST /api/projets/affectation/ :id/refuser" • Développer les endpoints de suivi : "GET /api/projets/mes-affectations-envoyees" et "GET /api/projets/statistiques-affectations" • Envoyer des notifications en temps réel à chaque étape du workflow • Réaliser les interfaces de gestion d'équipe côté Next.js	3

US26	En tant que membre d'équipe , je souhaite consulter les informations du projet afin de comprendre le contexte.	• Développer l'endpoint "GET /api/projets/ :id" (permissions projects.read.all ou projects.read.own) avec contrôle d'accès selon le rôle de l'utilisateur • Développer "GET /api/projets/ :id/equipe" pour consulter les membres et leur charge • Développer "GET /api/projets/ :id/charge" pour visualiser la charge de l'équipe • Réaliser la page de détail du projet organisée en sections côté Next.js	1
Epic 7 -Gestion des Contrats			
US28	En tant qu'administrateur, je souhaite créer un contrat de maintenance afin de formaliser les engagements de service avec le client.	• Développer l'endpoint "POST /api/contrats" avec upload de fichier via FileInterceptor (PDF, Word, Excel, 10 Mo max) et hashage anti-doublon (fichierHash) • Développer l'endpoint d'extraction PDF "POST /api/contrats/extract-pdf" via pdf-parse + regex pour récupérer automatiquement référence, dates, montants, SLA, client et projet • Développer l'endpoint d'upload de document scanné "POST /api/contrats/upload-scanner", formats PDF/JPG/PNG) • Réaliser l'interface avec choix du mode (formulaire manuel ou extraction PDF) et pré-remplissage automatique côté Next.js	2
	2		

US29	En tant qu'administrateur, je souhaite consulter la liste des contrats afin de suivre les engagements contractuels.	• Développer l'endpoint "GET /api/contrats" (permissions contracts.read.all ou contracts.read.own) • Développer les endpoints auxiliaires : "GET /api/contrats/clients", "GET /api/contrats/projets ?clientId=", "GET /api/contrats/demandes ?clientId=&projetId=", "GET /api/contrats/archived", "GET /api/contrats/client/ :clientId" • Développer "GET /api/contrats/ :id/fichier" pour le téléchargement du fichier contrat avec détection automatique du type MIME et streaming • Réaliser l'interface avec tableau, filtres, badges de statut et actions rapides côté Next.js	3
US30	En tant qu'administrateur, je souhaite recevoir des alertes avant l'expiration des contrats afin d'anticiper les renouvellements	• Développer le CRON job d'alertes automatiques déclenchant des notifications à J-30, J-15 et J-7 avant expiration (champs alerte30JoursEnvoyee, alerte15JoursEnvoyee, alerte7JoursEnvoyee en base pour éviter les doublons) • Envoyer les notifications par e-mail (Nodemailer) et in-app (NotificationsService) • Développer "GET /api/contrats/expiration/status" pour récupérer le statut d'expiration des contrats du client connecté • Réaliser l'interface d'alertes côté Next.js (ContratExpirationAlert, ContratExpirationModal)	2

US31	En tant qu'administrateur, je souhaite archiver un contrat afin de conserver l'historique.	• Développer l'endpoint "PATCH /api/contrats/ :id/archive" avec motif obligatoire (minimum 10 caractères) • Développer l'endpoint de restauration "PATCH /api/contrats/ :id/restore" • Développer l'endpoint de suppression définitive "DELETE /api/contrats/ :id" • Conserver les données historiques avec motifArchivage et dateArchivage • Réaliser le dialogue de confirmation côté Next.js	3
US32	En tant que client, je souhaite demander la résiliation de mon contrat afin d'interrompre le service.	• Développer l'endpoint "POST /api/contrats/ :id/demande-resiliation" avec motif obligatoire (minimum 10 caractères) et e-mail du client requis • Envoyer une notification par e-mail aux administrateurs et mettre à jour les champs dateDemandeResiliation et motifResiliation en base • Développer "GET /api/contrats/demandes-resiliation" pour que l'admin consulte les demandes en attente • Réaliser le formulaire de demande avec motif obligatoire côté Next.js	1
US33	En tant qu'administrateur, je souhaite résilier un contrat afin de l'interrompre avant terme.	• Développer l'endpoint "PATCH /api/contrats/ :id/resilier" avec motif obligatoire (minimum 10 caractères) • Mettre à jour le statut du contrat et notifier le client par e-mail (Nodemailer) • Journaliser l'action via ActivityLog (Action : RESILIER_CONTRACT) • Réaliser le dialogue de résiliation avec confirmation côté Next.js	2

US34	En tant que chef de projet, je souhaite consulter les contrats dans un calendrier afin de visualiser les dates d'expiration	• Utiliser l'endpoint "GET /api/contrats" avec filtrage côté frontend sur les champs dateDebutContratM et dateExpirationContratM • Calculer les indicateurs côté frontend : contrats expirant bientôt (J-30), contrats actifs, code couleur selon le statut (StatutContrat) • Réaliser la vue calendrier avec affichage des contrats par mois côté Next.js (composant GanttAgendaView ou TimelineAgenda)	1
US35	En tant que client , je souhaite soumettre un retour d'expérience à l'expiration de mon contrat afin d'exprimer ma satisfaction.	• Développer l'endpoint "POST /api/contrats/:id/feedback" pour enregistrer la note, le commentaire et l'intention de renouvellement dans FeedbackExpirationContrat • Vérifier que le profil client existe avant d'enregistrer le feedback • Implémenter la notification automatique aux administrateurs si le client souhaite renouveler • Journaliser l'action via ActivityLog • Réaliser le formulaire de feedback côté client avec affichage conditionnel selon le statut du contrat	1
US36	En tant qu' administrateur , je souhaite générer un PDF formaté du contrat afin de le partager avec le client.	• Développer l'endpoint de génération PDF en utilisant la bibliothèque PDFMake • Inclure les informations du contrat : référence, client, projet, dates, montants (HT, TVA, TTC), type SLA et conditions • Utiliser le template HTML existant comme base de mise en page • Réaliser le bouton de génération et de téléchargement côté Next.js (composant ContractPDFGenerator)	2

Epic 14 - Consulter journal d'activité			
US95	En tant qu'administrateur, je souhaite consulter l'historique complet de toutes les actions afin d'assurer la traçabilité.	• Développer l'endpoint "GET /api/activity-logs" avec pagination (page, limit), filtres (userId, entityType, action, status, startDate, endDate) et recherche textuelle • Développer "GET /api/activity-logs/stats" pour les statistiques globales (taux de succès, actions fréquentes, activité horaire) • Développer "GET /api/activity-logs/user/ :userId" et "GET /api/activity-logs/entity/ :type/ :id" pour les vues filtrées par utilisateur ou entité • Implémenter la sanitisation automatique des métadonnées sensibles (password, token, refreshToken → [REDACTED]) • Réaliser l'interface avec tableau filtrable, timeline et vue détaillée côté Next.js	3
US96	En tant qu'administrateur, je souhaite exporter les logs d'audit afin de répondre aux exigences de conformité.	• Développer l'endpoint "GET /api/activity-logs/me/export" avec formats CSV et JSON, filtres personnalisables et limite de 10 000 enregistrements • Implémenter le nettoyage automatique des logs anciens via CRON job quotidien à 2h du matin (@Cron(EVERY_DAY_AT_2AM)), rétention configurable via ACTIVITY_LOG_RETENTION_DAYS (défaut : 90 jours) • Développer l'endpoint de nettoyage manuel "POST /api/activity-logs/cleanup" • Réaliser l'interface d'export avec sélection de format et téléchargement direct côté Next.js	2
2			

II Analyse

1 Diagramme des cas d'utilisation du deuxième sprint

La figure ci-dessous présente le diagramme de cas d'utilisation du deuxième sprint :

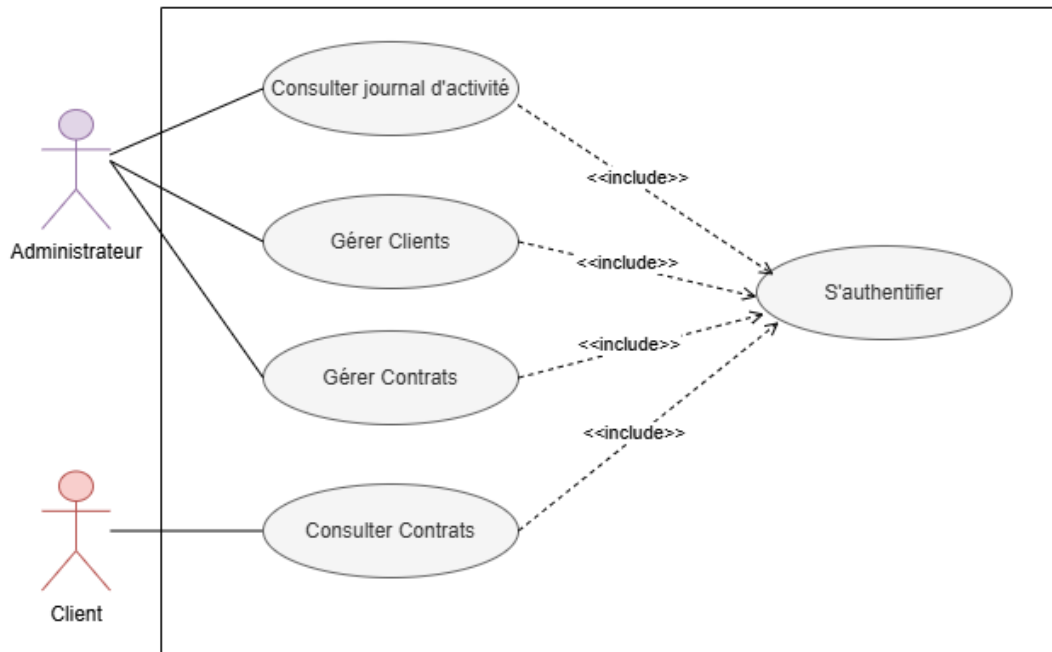


FIGURE IV.1 – Diagramme des cas d'utilisation du deuxième sprint

Nous allons accompagner les cas d'utilisation d'un texte descriptif pour expliquer le scénario de chaque cas ainsi que ses exceptions.

2 Analyse de la fonctionnalité « Gérer clients »

2.1 Raffinement de cas d'utilisation « Gérer clients »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des clients :

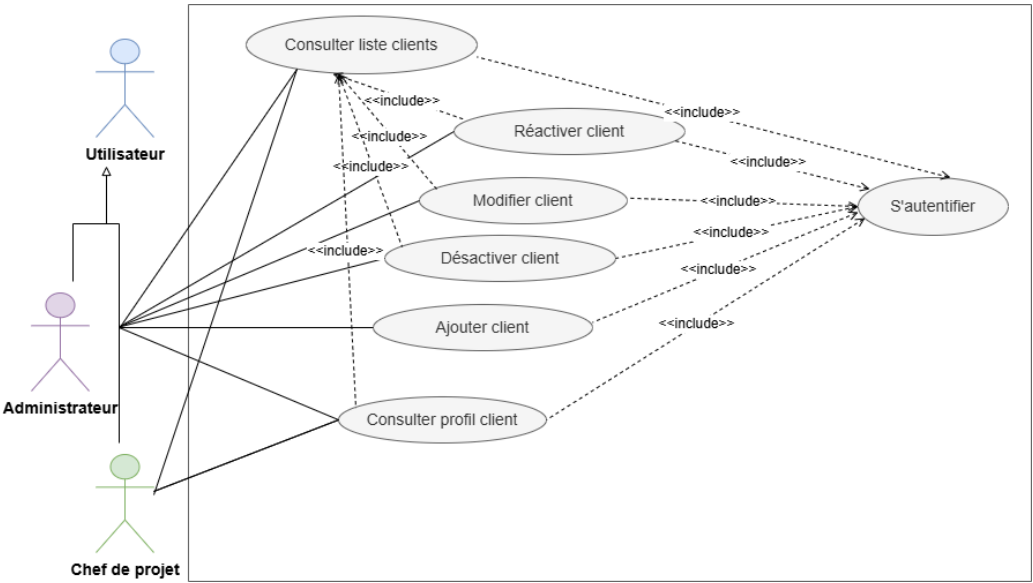


FIGURE IV.2 – Raffinement de cas d'utilisation « Gérer clients »

2.2 Description textuelle de cas d'utilisation « Ajouter client »

TABLE IV.2 – Description textuelle « Ajouter client »

Nom du cas d'utilisation	Ajouter un client
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié. L'utilisateur possède la permission de créer des clients. La liste des clients est accessible.
Postconditions	Le client est créé et enregistré dans le système. Le profil client associé est initialisé. Un message de confirmation est affiché.
Scénario principal	1. L'administrateur clique sur le bouton « Nouveau client ». 2. Le système affiche le formulaire de création. 3. L'administrateur saisit les informations du client : prénom, nom, e-mail et autres données requises. 4. L'administrateur renseigne ou sélectionne la société associée au client. 5. Si nécessaire, l'administrateur complète les informations de la société : nom, adresse, secteur d'activité et téléphone. 6. Le système peut récupérer automatiquement le logo de la société à partir d'une source externe, ainsi que l'adresse via un service de géocodage, si ces fonctionnalités sont activées. 7. L'administrateur valide le formulaire en cliquant sur « Créer le client ». 8. Le système vérifie la validité des données et l'unicité de l'e-mail. 9. Le système crée le client et associe son profil dans la base de données. 10. Le système envoie un e-mail de confirmation ou de bienvenue si cette fonctionnalité est prévue. 11. Le système affiche un message de succès et met à jour la liste des clients.
Scénarios alternatifs	• L'administrateur ajoute manuellement le logo de la société au lieu d'utiliser la récupération automatique. • L'administrateur associe le client à une société existante déjà enregistrée. • L'administrateur saisit manuellement l'adresse de la société lorsque la suggestion automatique du géocodage ne correspond pas. • L'administrateur complète plus tard certaines informations facultatives du client.
Scénarios d'exception	• Un champ obligatoire est vide ou contient une valeur invalide. • L'adresse e-mail saisie est déjà utilisée par un autre utilisateur. • Le système rencontre une erreur lors de l'enregistrement du client. • L'administrateur tente de quitter le formulaire alors que des données ont déjà été saisies.

2.3 Description textuelle de cas d'utilisation « Consulter profil client »

TABLE IV.3 – Description textuelle « Consulter profil client »

Nom du cas d'utilisation	Consulter profil client
Acteurs	Administrateur, Chef de projet, Client (son propre profil)
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de consulter les profils clients ou consulte son propre profil Pour Chef de projet : le client doit être associé à au moins un de ses projets
Postconditions	Profil client consulté
Scénario principal	1. L'utilisateur accède au profil du client. 2. Le système affiche les informations personnelles et société. 3. Le système affiche les statistiques et éléments accessibles selon le rôle de l'utilisateur. 4. L'utilisateur peut naviguer vers les éléments disponibles (projets, demandes, tickets, documents).
Restrictions d'accès	• Administrateur : accès complet (contrats, projets, demandes, tickets, notes privées, documents). • Chef de projet : accès limité aux projets où il est assigné et aux demandes/tickets liés à ces projets uniquement. • Client : accès à son propre profil, ses projets, demandes et tickets.
Scénarios alternatifs	• L'utilisateur exporte les informations du profil en PDF. • L'utilisateur clique sur "Contacter" pour envoyer un e-mail ou un message via la messagerie interne de l'application. • L'utilisateur clique sur l'adresse de la société pour visualiser sa localisation sur une carte interactive. • L'administrateur ajoute une note privée sur le client.

2.4 Description textuelle de cas d'utilisation « Désactiver client »

TABLE IV.4 – Description textuelle « Désactiver un client »

Nom du cas d'utilisation	Désactiver un client
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié L'utilisateur possède la permission de désactiver les clients Liste des clients affichée Le client à désactiver doit être actuellement Actif
Postconditions	Le compte client est désactivé
Scénario principal	1. L'administrateur choisit un client dans la liste. 2. L'administrateur clique sur l'action (Désactiver). 3. Le système vérifie si le client possède des contrats actifs, des demandes en cours ou des tickets actifs. 4. Le système affiche une boîte de dialogue de confirmation avec les détails des contrats, demandes et tickets actifs. 5. L'administrateur confirme l'action. 6. Le système met à jour le statut du client. 7. Le système révoque les sessions actives (suppression des refresh tokens). 8. Le système envoie un e-mail automatique au contact principal du client pour l'informer de la désactivation. 9. Le système affiche le message de succès.
Scénarios d'exception	• L'administrateur tente de désactiver un client qui possède des contrats actifs sans les clôturer. • L'administrateur tente de désactiver un client ayant des demandes en attente de traitement. • L'administrateur tente de désactiver un client ayant des tickets actifs non résolus.
Scénarios alternatifs	L'administrateur clôture les contrats, traite les demandes et résout les tickets avant la désactivation.

3 Analyse de la fonctionnalité « Gérer Contrats »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la gestion des Contrats afin de comprendre les principales fonctionnalités du système, ce qui permet d'avoir une vue globale des interactions.

3.1 Raffinement de cas d'utilisation « Gérer Contrats »

Au cours de cette partie, nous allons analyser chaque cas d'utilisation raffiné pour le cas de gestion des Contrats :

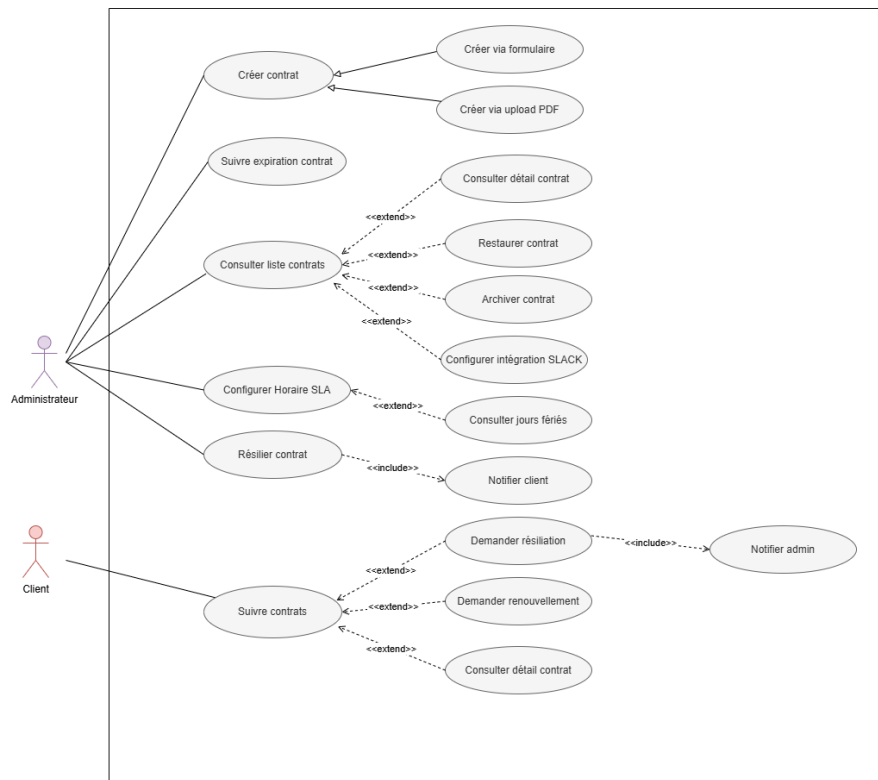


FIGURE IV.3 – Diagramme des cas d'utilisation « Gérer Contrats »

3.2 Description textuelle de cas d'utilisation « Créer contrat »

3.3 Description textuelle de cas d'utilisation « Archiver contrat »

Nom du cas d'utilisation	Archiver un contrat
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié. L'utilisateur possède la permission d'archiver des contrats. Le contrat est expiré et non encore archivé.
Postconditions	Le contrat est marqué comme archivé et n'apparaît plus dans la liste principale. Le motif d'archivage est enregistré et associé au contrat.
Scénario principal	1. L'administrateur sélectionne un contrat expiré depuis la liste. 2. L'administrateur clique sur « Archivage définitif » dans le menu contextuel. 3. Le système affiche un dialogue de confirmation demandant un motif d'archivage. 4. L'administrateur saisit un motif et confirme. 5. Le système archive le contrat et met à jour la liste.
Scénarios alternatifs	• L'administrateur sélectionne plusieurs contrats expirés et clique sur « Archiver la sélection ». Le système archive tous les contrats éligibles et affiche le nombre de contrats archivés avec succès.
Scénarios d'exception	• Le système détecte que le contrat n'est pas encore expiré et bloque l'action d'archivage. • Le système détecte que le motif saisi contient moins de 10 caractères et désactive le bouton de confirmation.

TABLE IV.6 – Description textuelle « Archiver contrat »

Nom du cas d'utilisation	Créer un contrat
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié. L'utilisateur possède la permission de créer des contrats. La liste des contrats est accessible.
Postconditions	Le contrat est enregistré avec le statut ACTIF et une référence générée automatiquement. Le contrat est visible dans la liste des contrats et lié au client sélectionné.
Scénario principal	1. L'administrateur clique sur « Nouveau » depuis la liste des contrats. 2. Le système propose deux modes : <i>Saisie manuelle</i> ou <i>Import PDF</i>. 3. L'administrateur sélectionne <i>Saisie manuelle</i> et confirme. 4. L'administrateur renseigne les informations du contrat : client, projet, dates, montant, cycle de facturation (mensuel ou semestriel) et taux de TVA. 5. L'administrateur configure le niveau de service (SLA) avec les délais de réponse et de résolution par priorité. 6. Le système calcule automatiquement le montant TVA et le montant TTC. 7. L'administrateur valide le formulaire. 8. Le système enregistre le contrat, émet un événement <code>contrat.created</code> afin de créer le canal de communication associé, puis redirige l'utilisateur vers la liste des contrats.
Scénarios alternatifs	• L'administrateur choisit le mode <i>Import de document</i> et dépose un fichier PDF. Le système extrait automatiquement les données et pré-remplit le formulaire. L'administrateur vérifie les données, sélectionne le client et le projet, puis valide.
Scénarios d'exception	• Le système détecte qu'un contrat actif existe déjà pour ce client/projet et affiche un message de blocage. • Le système détecte que le client, les dates ou le montant ne sont pas renseignés et affiche un message d'erreur. • Le système détecte que la date de début est antérieure à aujourd'hui ou que la date d'expiration est antérieure ou égale à la date de début et affiche un message d'erreur. • Le système détecte que le montant saisi est nul ou non numérique et affiche un message d'erreur.

TABLE IV.5 – Description textuelle « Créer contrat »

3.4 Description textuelle de cas d'utilisation « Configurer intégration Slack »

Nom du cas d'utilisation	Configurer intégration Slack
Acteurs	Administrateur
Préconditions	L'administrateur est authentifié. Un contrat est sélectionné.
Postconditions	La configuration Slack est enregistrée et liée au contrat. Les notifications sont actives selon les préférences choisies.
Scénario principal	1. L'administrateur clique sur « Intégration Slack » depuis le menu contextuel du contrat. 2. Le système affiche le dialogue de configuration Slack. 3. L'administrateur saisit l'URL du webhook et le canal Slack. 4. L'administrateur configure les types de notifications souhaités : ticket créé, mise à jour, résolu, critique seulement. 5. L'administrateur clique sur « Configurer ». 6. Le système enregistre la configuration et l'associe au contrat.
Scénarios alternatifs	• Le système détecte qu'une configuration Slack existe déjà pour ce contrat et pré-remplit le formulaire. L'administrateur modifie les champs souhaités et clique sur « Mettre à jour ». • Après sauvegarde, l'administrateur clique sur « Tester ». Le système envoie un message de test sur le canal Slack configuré.

4 Analyse de la fonctionnalité « Suivre contrats »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs au suivi des contrats par le client afin de comprendre les interactions disponibles depuis son espace personnel.

4.1 Description textuelle du cas d'utilisation « Consulter ses contrats »

TABLE IV.8 – Description textuelle « Consulter ses contrats »

Nom du cas d'utilisation	Consulter ses contrats
Acteurs	Client
Préconditions	L'utilisateur est authentifié. L'utilisateur possède la permission <code>contracts.read.own</code> .
Postconditions	La liste des contrats du client est affichée.
Scénario principal	1. Le client accède à la section « Mes contrats ». 2. Le système filtre automatiquement les contrats liés au profil client de l'utilisateur connecté. 3. Le système affiche la liste des contrats avec leurs statuts, dates et niveaux SLA. 4. Le client sélectionne un contrat pour consulter ses détails. 5. Le système affiche les informations complètes : référence, période, montant TTC, conditions SLA et tickets associés.
Scénarios alternatifs	• Le client télécharge le fichier PDF du contrat via le bouton « Télécharger ». • Le client consulte le statut d'expiration de ses contrats et reçoit des alertes à J-30, J-15 et J-7 avant l'échéance.

4.2 Description textuelle du cas d'utilisation « Soumettre demande de résiliation »

TABLE IV.9 – Description textuelle « Soumettre demande de résiliation »

Nom du cas d'utilisation	Soumettre demande de résiliation
Acteurs	Client
Préconditions	L'utilisateur est authentifié. Le contrat est actif et appartient au client.
Postconditions	La demande de résiliation est enregistrée. L'administrateur est notifié par e-mail et notification interne.
Scénario principal	1. Le client accède au détail d'un contrat actif. 2. Le client clique sur « Demander la résiliation ». 3. Le système affiche un formulaire de saisie du motif. 4. Le client saisit un motif (minimum 10 caractères) et confirme. 5. Le système enregistre la demande et notifie les administrateurs. 6. Le système affiche un message de confirmation au client.
Scénarios d'exception	• Le motif saisi contient moins de 10 caractères : le bouton de confirmation reste désactivé. • Le contrat n'est pas actif : l'action est indisponible.

5 Analyse de la fonctionnalité « Consulter journal d'activités »

Dans cette partie, nous allons présenter les cas d'utilisation relatifs à la consultation du journal d'activités, qui assure la traçabilité complète de toutes les opérations effectuées sur la plateforme.

5.1 Description textuelle du cas d'utilisation « Consulter journal d'activités »

TABLE IV.10 – Description textuelle « Consulter journal d'activités »

Nom du cas d'utilisation	Consulter journal d'activités
Acteurs	Administrateur
Préconditions	L'utilisateur est authentifié. L'utilisateur possède la permission <code>logs.read</code> .
Postconditions	Le journal d'activités est affiché avec les filtres appliqués.
Scénario principal	1. L'administrateur accède à la section « Journal d'activités ». 2. Le système affiche la liste paginée des logs triés par date décroissante, avec pour chaque entrée : la date, l'utilisateur, l'action, le type d'entité, la méthode HTTP, le chemin et le statut. 3. L'administrateur applique des filtres : par utilisateur, par type d'entité, par action, par statut (SUCCESS / FAILED), ou par plage de dates. 4. Le système filtre et met à jour la liste en temps réel. 5. L'administrateur clique sur une entrée pour consulter ses détails complets, incluant les métadonnées associées.
Scénarios alternatifs	• L'administrateur clique sur « Exporter » pour télécharger les logs au format CSV (jusqu'à 10 000 entrées). • L'administrateur consulte les statistiques globales : total du jour, taux de succès, utilisateurs uniques, actions les plus fréquentes et activité par heure. • L'administrateur consulte les statistiques d'un utilisateur spécifique : activité des 30 derniers jours, taux de succès et entités les plus actives.
Scénarios d'exception	• Aucun log ne correspond aux filtres appliqués : le système affiche un message « Aucun résultat trouvé ».

III Diagramme de classes du deuxième sprint

IV Diagramme de séquence

1 Diagramme de séquences du cas d'utilisation « Demande Résiliation »

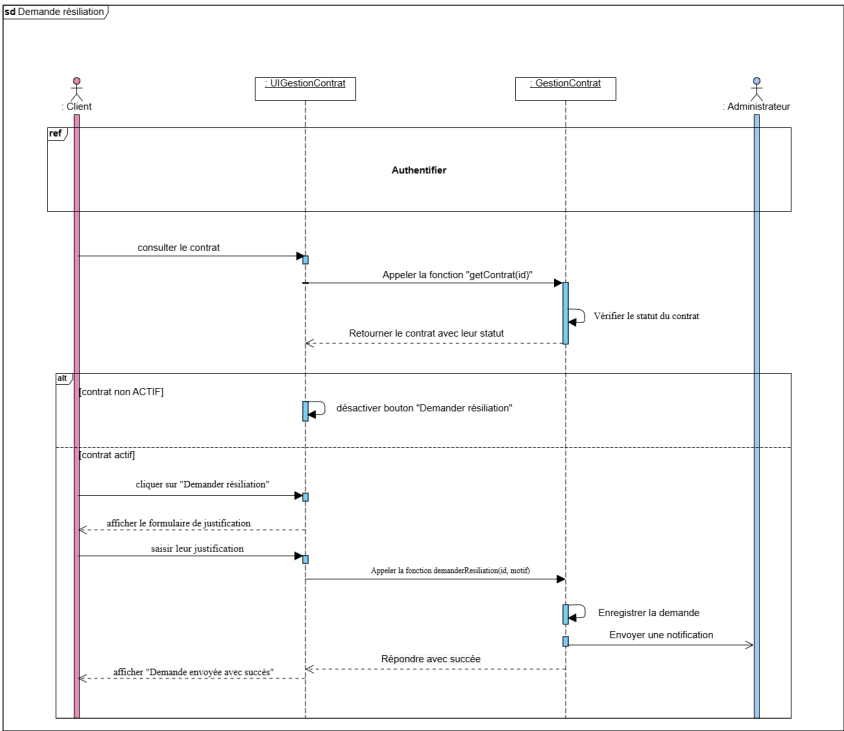


FIGURE IV.4 – Diagramme de séquence du cas d'utilisation « Demande Résiliation »

2 Diagramme de séquences du cas d'utilisation « Résilier contrat »

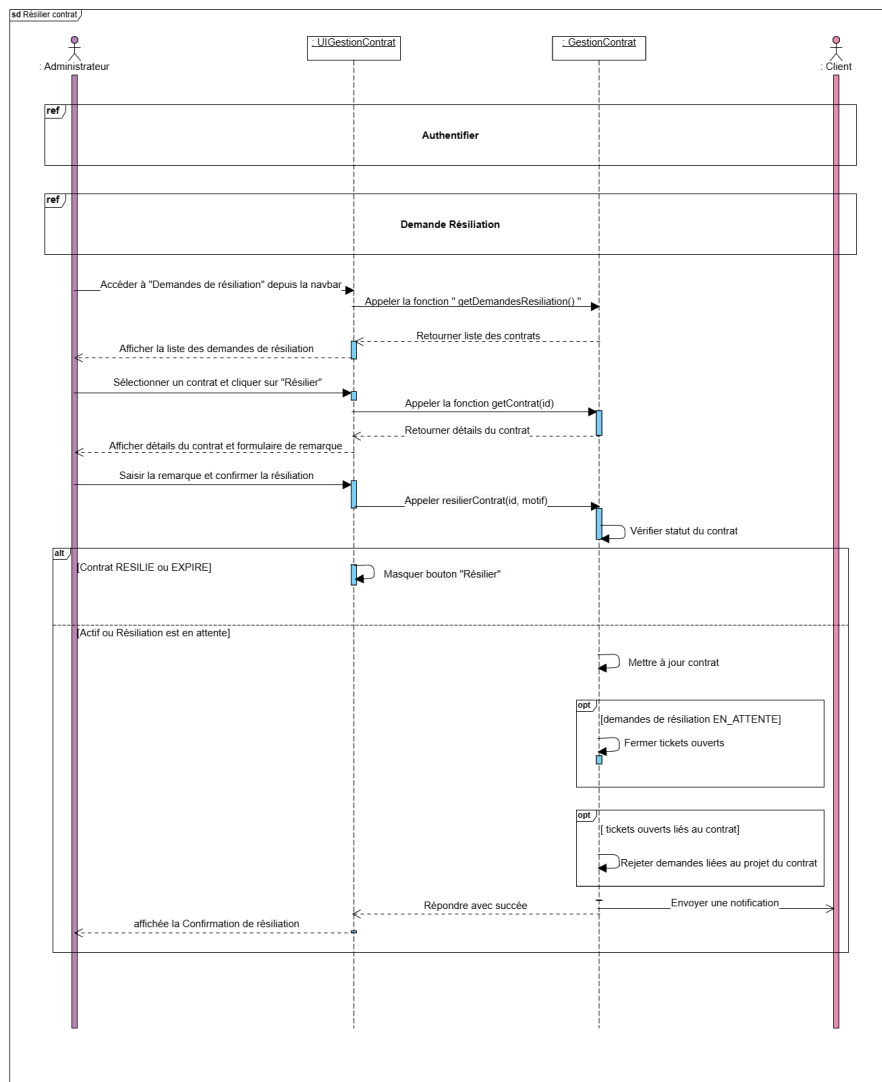


FIGURE IV.5 – Diagramme de séquence du cas d'utilisation « Résilier contrat »

V Schéma relationnel

Les entités introduites dans ce sprint complètent le schéma relationnel établi lors du premier sprint. Les relations clés sont les suivantes :

Contrat (idContrat, referenceContrat, #idProfilClient, #idProjet, dateDebutContratM, dateExpirationContratM, montantMensuel, montantSemestriel, cycleFacturation, tauxTVA, montantTVA, montantTTC, typeSLA, conditionsSLA, fichierContrat, fichierHash, statutContratM, estArchive, dateArchivage, motifArchivage, dateDemandeResiliation, motifResiliation, alerte30JoursEnvoyee, alerte15JoursEnvoyee, alerte7JoursEnvoyee, dateCreation)

ConfigurationSLAContrat (idConfig, #idContrat, creneauxHoraires, motifPersonnalisation, dateCreation)

ReglesDelaiSLA (idRegle, #idConfigContrat, #idConfigGlobale, priorite, delaiReponseMn, delaiResolu-

tionMn)

SlackConfig (idSlack, **#idContrat**, webhookUrl, canal, notificationsActives, dateCreation)

Projet (idProjet, nomProjet, descriptionProjet, statutP, tags, dateDebutP, dateFinP, **#idProfilClient**, **#idChefProjet**, dateCreationP)

MembreProjet (idMembre, **#idProjet**, **#idProfilEmploye**, rolesDansProjet, statut, dateAffectation, dateDebut, dateFin)

ActivityLog (idLog, **#idUtilisateur**, action, entityType, entityId, method, path, ipAddress, status, description, metadata, createdAt)

FeedbackExpirationContrat (idFeedback, **#idContrat**, note, commentaire, souhaitRenouveler, raisonsNonRenouvellement, dateCreation)

VI Réalisation

1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 2, incluant les modules de gestion des clients, projets et contrats.

[À compléter avec les détails d'implémentation]

2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 2.

[À compléter avec les captures d'écran]

VII Tests et Validation

La phase de test du deuxième sprint valide les fonctionnalités de gestion des contrats, des projets et de la traçabilité. Nous avons adopté la même stratégie de tests structurée que lors du premier sprint, organisée sur trois niveaux complémentaires.

0.1 Tests unitaires

Les tests unitaires ont été réalisés avec **Vitest** en isolant chaque service grâce à des mocks Prisma. Deux modules sont couverts : la création de contrats et la gestion des projets.

Bilan : 18 tests exécutés — 18 réussis — 0 échec.

□ Module Contrats

Méthode : `create()`

Cette suite valide les règles métier de création de contrat : validations des entités, des dates, des montants et de la détection des doublons. Les cas suivants ont été couverts :

- Des données valides sont soumises : le contrat est créé avec une référence au format `CTR-{timestamp}-{suffixe}` et le statut `ACTIF`.
- Le client fourni est inexistant : une `NotFoundException` est levée.
- Le projet fourni est inexistant : une `NotFoundException` est levée.
- Le projet n'appartient pas au client sélectionné : une `NotFoundException` est levée.
- Un contrat similaire existe déjà pour ce projet : une `BadRequestException` est levée.
- La date de début est antérieure à aujourd'hui : une `BadRequestException` est levée avec le message « ne peut pas être dans le passé ».
- La date d'expiration est antérieure à la date de début : une `BadRequestException` est levée avec le message « doit être postérieure à la date de début ».
- Le montant mensuel est nul ou négatif : une `BadRequestException` est levée avec le message « doit être supérieur à 0 ».
- Un montant valide est soumis : le montant est enregistré correctement.
- La référence générée respecte le format `CTR-`.

□ Module Gestion des clients

Méthodes : `mettreAJourClient()` · `desactiverClient()` · `reactiverClient()` · `telechargerLogo()` · `supprimerLogo()`

Cette suite valide les opérations de gestion des clients, la gestion des fichiers logo et le service de géocodage.

Les cas suivants ont été couverts :

- Les informations d'un client existant sont mises à jour : le résultat validé est retourné correctement.

- La mise à jour est effectuée sur un client inexistant : une `NotFoundException` est levée.
- Un client actif est désactivé : l'opération est réalisée avec succès.
- Un client désactivé est réactivé : l'opération est réalisée avec succès.
- La réactivation est effectuée sur un client inexistant : une `NotFoundException` est levée.
- Un logo est téléversé pour un client existant : le fichier est traité et stocké correctement.
- Le logo d'un client est supprimé : un message de succès est retourné.
- Le téléversement ou la suppression du logo est effectué sur un client inexistant : une `NotFoundException` est levée.
- Le service de géocodage est interrogé avec une adresse valide présente en cache : l'adresse est retournée correctement.
- Le service de géocodage est interrogé avec une requête trop courte : un tableau vide est retourné.
- Les résultats du service de géocodage sont mis en cache correctement après la première requête.
- Les résultats filtrés du géocodage éliminent les doublons de type pays.

```

PASS src/modules/clients/clients-management.spec.ts
ClientsService - Gestion Client
  update
    ✓ devrait mettre à jour les informations du client avec succès (11 ms)
    ✓ devrait lever NotFoundException si le client n'existe pas (40 ms)
  désactiver et réactiver
    ✓ devrait désactiver le compte client avec succès (3 ms)
    ✓ devrait réactiver le compte client avec succès (2 ms)
    ✓ devrait lever NotFoundException lors de la désactivation si client inexistant (3 ms)
  updateClientLogo et removeClientLogo
    ✓ devrait mettre à jour le logo du client avec succès (2 ms)
    ✓ devrait supprimer le logo du client avec succès (3 ms)
    ✓ devrait lever NotFoundException si le client n'existe pas pour updateClientLogo (3 ms)
    ✓ devrait lever NotFoundException si le client n'existe pas pour removeClientLogo (4 ms)
    ✓ devrait ignorer la suppression d'un ancien logo s'il n'existe pas (4 ms)
GeocodingService - searchAddress
  ✓ devrait rechercher et retourner une adresse valide (293 ms)
  ✓ devrait retourner un tableau vide pour une query trop courte (4 ms)
  ✓ devrait mettre en cache les résultats de recherche (996 ms)
  ✓ devrait gérer les requêtes avec espaces multiples (1004 ms)
  ✓ devrait filtrer les résultats trop vagues (pays seul) (1003 ms)

Test Suites: 1 passed, 1 total
Tests: 15 passed, 15 total
Snapshots: 0 total
Time: 4.615 s
Ran all test suites matching src/modules/clients/clients-management.spec.ts.

```

Gestion des clients

FIGURE IV.6 – Résultats des tests unitaires — Gestion des clients

□ **Module Projets** *Méthodes : `create()` · `assignerMembre()` · `verifierArchivage()`*

Cette suite valide les règles métier de création de projet, d'affectation des membres et de vérification des conditions d'archivage. Les cas suivants ont été couverts :

- Un projet est créé avec des données valides : le chef de projet et les membres sont notifiés par e-mail et notification interne.
- Le client fourni est inexistant : une `NotFoundException` est levée.
- Le nom du projet est déjà utilisé : une `ConflictException` est levée.
- Le chef de projet n'est pas disponible (`disponibilite = false`) : une `BadRequestException`

est levée.

- Aucun membre d'équipe n'est fourni : une `BadRequestException` est levée.
- La date de fin est antérieure à la date de début : une `BadRequestException` est levée.
- Un membre est proposé pour un projet : la proposition est créée avec le statut `EN_ATTENTE`.
- Un projet avec des tickets actifs est soumis à l'archivage : le système retourne la liste des tickets bloquants.

0.2 Tests d'intégration

Les tests d'intégration valident les endpoints REST des contrats via **Swagger UI** contre le backend en cours d'exécution.

Bilan : 6 tests exécutés — 6 réussis — 0 échec.

- **Gestion des contrats** *Endpoints : `POST /api/contrats` · `PATCH /api/contrats/:id/archive` · `PATCH /api/contrats/:id/resilier`*

Cette suite valide le comportement complet des endpoints de gestion des contrats. Les cas suivants ont été couverts :

- Un administrateur authentifié crée un contrat avec des données valides : l'endpoint retourne le code **201 Created** avec la référence générée, le montant TTC calculé et le statut `ACTIF`.
- Un utilisateur sans la permission `contracts.create` tente de créer un contrat : l'endpoint retourne le code **403 Forbidden**.
- Un contrat actif est soumis à l'archivage sans motif suffisant (moins de 10 caractères) : l'endpoint retourne le code **400 Bad Request**.
- Un contrat expiré est archivé avec un motif valide : l'endpoint retourne le code **200 OK** et le contrat est marqué `estArchive = true`.
- Un contrat actif est résilié par l'administrateur avec un motif valide : l'endpoint retourne le code **200 OK** et le statut du contrat est mis à jour.
- Un client tente d'accéder à la liste complète des contrats (endpoint `GET /api/contrats` avec permission `contracts.read.all`) : l'endpoint retourne le code **403 Forbidden**, seuls ses propres contrats lui sont accessibles via `contracts.read.own`.

0.3 Tests End-to-End

Les tests End-to-End simulent les parcours utilisateurs complets dans un navigateur réel piloté par **Playwright**, en interaction avec le frontend Next.js, le backend NestJS et la base de données PostgreSQL.

Bilan : 4 tests exécutés — 4 réussis — 0 échec.

- **Création et suivi de contrat**

2 scénarios : création manuelle, suivi client

Cette suite valide les flux complets de création d'un contrat et de consultation par le client. Les cas suivants ont été couverts :

- L'administrateur crée un contrat via le formulaire multi-étapes en mode saisie manuelle : le système calcule automatiquement le montant TTC, génère la référence et affiche le contrat dans la liste avec le statut `ACTIF`.
- Le client se connecte et consulte ses contrats : le système filtre automatiquement les contrats liés à son profil, affiche les alertes d'expiration et permet le téléchargement du fichier PDF.

□ **Journal d'activités**

2 scénarios : consultation, filtrage

Cette suite valide le flux complet de consultation du journal d'activités par l'administrateur. Les cas suivants ont été couverts :

- L'administrateur accède au journal d'activités : le système affiche la liste paginée des logs avec les informations complètes (utilisateur, action, statut, date) et les statistiques globales du jour.
- L'administrateur applique des filtres combinés (statut `FAILED` et plage de dates) : le système filtre la liste et affiche uniquement les entrées correspondantes, permettant l'identification rapide des anomalies.

Conclusion

Ce chapitre a présenté le développement du deuxième sprint de la plateforme **SolidMaint**. Nous avons mis en place la gestion complète des contrats de maintenance avec leurs configurations SLA, l'intégration Slack par contrat, la gestion des projets et de leurs équipes, ainsi que le journal d'activités assurant la traçabilité de toutes les opérations. L'ensemble de ces fonctionnalités a été validé par 28 tests répartis sur trois niveaux.

Le chapitre suivant détaillera l'implémentation du troisième sprint, consacré au cœur opérationnel de la plateforme : la gestion des demandes et des tickets avec le suivi SLA en temps réel.

CHAPITRE 5

Chapitre 5



Sommaire

5

I	Analyse	127
II	Conception	128
III	Réalisation	129
IV	Tests	130

Introduction

Dans ce chapitre, nous abordons le troisième sprint du projet **SolidMaint**, couvrant la période du 3 avril au 1er mai 2026, soit quatre semaines de développement consacrées au cœur fonctionnel de l’application, à savoir la gestion des demandes clients et des tickets de maintenance. Nous commencerons par la présentation du backlog du sprint regroupant trente-six user stories, suivie de la phase d’analyse et de conception, puis de la réalisation des interfaces intégrant un tableau Kanban, un suivi du temps de travail, un système de gestion des SLA et des dashboards adaptés à chaque rôle, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l’effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 8 - Gestion des demandes			
US39	En tant que client , je souhaite soumettre une demande de maintenance afin de signaler un besoin.	Formulaire de soumission, upload de pièces jointes (max 5), génération numéro unique, confirmation email	2
US40	En tant que chef de projet , je souhaite consulter les demandes en attente afin de les traiter.	Liste filtrée par statut EN_ATTENTE, tri par date, indicateurs de priorité	1
US41	En tant que chef de projet , je souhaite valider une demande afin de la convertir en ticket.	Interface de validation, définition catégorie/priorité, création automatique de ticket lié	2
US42	En tant que chef de projet , je souhaite demander des informations complémentaires afin de clarifier une demande.	Formulaire de questions, changement statut INFOS_REQUISES, relance automatique après 48h	2
US43	En tant que client , je souhaite compléter ma demande afin de fournir les informations manquantes.	Formulaire de réponse, retour automatique au statut EN_ATTENTE	1

ID US	User Story	Description technique	Effort (j)
US44	En tant que chef de projet , je souhaite requalifier une demande afin de corriger sa catégorie ou sa priorité.	Modification type/priorité avec justification obligatoire, traçabilité	1
US45	En tant qu' utilisateur , je souhaite commenter une demande afin de communiquer sur le contexte.	Éditeur riche, mentions @utilisateur, pièces jointes possibles	2
US46	En tant qu' utilisateur , je souhaite consulter l'historique d'une demande afin de suivre son évolution.	Timeline chronologique avec tous les événements, filtres par type, export PDF	1
Epic 9 - Gestion des tickets			
US47	En tant que développeur , je souhaite visualiser mes tickets dans un tableau Kanban afin de gérer mes tâches.	Tableau Kanban avec colonnes (OUVERT, EN_COURS, EN_REVUE, RESOLU, FERME), drag & drop, indicateurs SLA	3
US48	En tant que chef de projet , je souhaite assigner un ticket à un développeur afin de répartir la charge.	Sélection développeur, vérification disponibilité/compétences, alerte surcharge	2
US49	En tant que chef de projet , je souhaite assigner un ticket à plusieurs développeurs afin de collaborer sur des tâches complexes.	Multi-sélection, définition rôles (principal/support), répartition temps estimé	2
US50	En tant que développeur , je souhaite me désassigner d'un ticket afin de libérer ma charge.	Désassignation avec motif obligatoire, retour statut OUVERT, notification chef projet	1
US51	En tant que développeur , je souhaite démarrer un chronomètre sur un ticket afin de tracker mon temps.	Bouton Démarrer, création session de travail, indicateur visuel chrono actif	2
US52	En tant que développeur , je souhaite mettre en pause mon chronomètre afin de gérer les interruptions.	Bouton Pause, enregistrement temps écoulé, possibilité de reprise	1

ID US	User Story	Description technique	Effort (j)
US53	En tant que développeur , je souhaite arrêter le chronomètre afin de finaliser ma session.	Arrêt avec commentaire optionnel, ajout session à l'historique, notification si dépassement	2
US54	En tant que développeur , je souhaite ajouter manuellement une session de travail afin de corriger des oublis.	Formulaire (date, heures, durée, commentaire), validation sans chevauchement	1
US55	En tant que développeur , je souhaite changer le statut d'un ticket afin de refléter l'avancement.	Workflow avec transitions validées, notification parties prenantes	2
US56	En tant que client , je souhaite valider la résolution d'un ticket afin de confirmer que le problème est résolu.	Interface de validation avec commentaire, fermeture ticket, notification développeur	1
US57	En tant que chef de projet , je souhaite rouvrir un ticket fermé afin de traiter un problème récurrent.	Réouverture avec motif, réinitialisation SLA, notification assignés	1
US58	En tant que chef de projet , je souhaite consulter le temps passé par ticket afin d'analyser la productivité.	Affichage temps total et détaillé par développeur, graphiques	2
US59	En tant que développeur , je souhaite signaler un blocage afin d'alerter sur un obstacle.	Formulaire (type, description, impact), notification urgence chef projet, pause SLA	2
US60	En tant que membre d'équipe , je souhaite résoudre un blocage afin de permettre au développeur de reprendre son travail.	Résolution avec commentaire obligatoire, reprise chrono SLA, traçabilité	1
US61	En tant que développeur , je souhaite demander une extension de délai SLA afin de disposer de plus de temps.	Formulaire de demande (durée, justification), soumission pour validation	1
US62	En tant que chef de projet , je souhaite approuver ou rejeter les demandes d'extension SLA.	Liste demandes en attente, approbation/rejet avec commentaire, mise à jour auto SLA	1

ID US	User Story	Description technique	Effort (j)
US63	En tant qu' utilisateur , je souhaite commenter un ticket avec mentions afin de communiquer efficacement.	Éditeur riche, autocomplétion @utilisateur, distinction internes/externes	2
US64	En tant qu' utilisateur , je souhaite ajouter des pièces jointes aux tickets afin de fournir du contexte.	Upload par drag & drop (images/PDF/logs, max 10MB), prévisualisation	1
US65	En tant que développeur , je souhaite être informé lorsqu'un autre utilisateur modifie le même ticket.	WebSocket temps réel, toast notification, indicateur "X est en train d'éditer"	2
US66	En tant qu' utilisateur , je souhaite consulter l'historique complet d'un ticket afin de comprendre son évolution.	Timeline chronologique inversée, filtres par type, export PDF	1
Epic 13 - Dashboards			
US89	En tant qu' administrateur , je souhaite consulter un dashboard global afin de piloter le système.	KPIs (utilisateurs actifs, tickets ouverts, taux SLA, CA), graphiques, alertes, filtres période	3
US90	En tant que chef de projet , je souhaite consulter un dashboard d'équipe afin de piloter l'activité.	Vue équipe (charge travail par dev, performance SLA, temps passé vs estimé)	3
US91	En tant que développeur , je souhaite consulter mon dashboard personnel afin d'organiser ma journée.	Tickets assignés avec priorités, chronomètre actif, temps passé, échéances SLA	2
US92	En tant que client , je souhaite consulter mon dashboard afin de suivre mes projets.	Contrats actifs, demandes/tickets en cours, statistiques, documents récents, contacts équipe	2
Total Sprint 3 :			54 j

I Analyse

1 Diagramme des cas d'utilisation du Sprint 3

Cette section présentera le diagramme de cas d'utilisation du troisième sprint, couvrant la gestion des demandes, tickets et dashboards.

[À compléter avec le diagramme de cas d'utilisation]

2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 3.

[À compléter avec les descriptions textuelles]

II Conception

1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la gestion des demandes, tickets et suivi du temps.

[À compléter avec les diagrammes de séquence]

2 Modèle de données

Cette section présentera le modèle de données relatif à la gestion des demandes, tickets, sous-tâches et sessions de travail.

[À compléter avec le schéma de base de données]

III Réalisation

1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 3, incluant les modules de gestion des demandes, tickets et dashboards.

[À compléter avec les détails d'implémentation]

2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 3, notamment le tableau Kanban et les dashboards.

[À compléter avec les captures d'écran]

IV Tests

1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 3.

[À compléter avec les résultats des tests]

2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]

CHAPITRE 6

Chapitre 6



Sommaire

6

I	Analyse	138
II	Conception	139
III	Réalisation	140
IV	Tests	141

Introduction

Dans ce chapitre, nous abordons le quatrième et dernier sprint du projet **SolidMaint**, couvrant la période du 2 mai au 31 mai 2026, soit quatre semaines de développement consacrées aux fonctionnalités avancées de collaboration, de gestion documentaire et d'intelligence artificielle. Nous commencerons par la présentation du backlog du sprint regroupant trente-quatre user stories réparties en six epics, suivie de la phase d'analyse et de conception, puis de la réalisation des interfaces intégrant un système de messagerie en temps réel, un module de gestion documentaire avec versioning, un système de notifications multicanaux et un moteur d'intelligence artificielle dédié à la catégorisation automatique des demandes et à la recommandation de solutions, et enfin des tests conduits pour valider les fonctionnalités livrées.

Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé avec les descriptions techniques et l'effort estimé pour chaque user story.

ID US	User Story	Description technique	Effort (j)
Epic 10 - Messagerie et collaboration			
US67	En tant qu'utilisateur, je souhaite accéder à une messagerie intégrée afin de communiquer avec mon équipe.	Sidebar avec channels, indicateurs messages non lus, statut présence utilisateurs	2
US68	En tant qu'utilisateur, je souhaite envoyer des messages dans un channel afin de communiquer.	Éditeur riche, mentions @, pièces jointes, livraison via WebSocket	2
US69	En tant qu'utilisateur, je souhaite répondre à un message afin de créer un fil de discussion.	Thread dédié pour réponses, notifications uniquement aux participants	2
US70	En tant qu'utilisateur, je souhaite mentionner des collègues afin d'attirer leur attention.	Autocomplétion @nom, notification push, badge mention non lue	1
US71	En tant qu'utilisateur, je souhaite éditer mes messages afin de corriger des erreurs.	Édition possible dans les 15min, indicateur "(modifié)", historique modifications	1
US72	En tant qu'utilisateur, je souhaite supprimer mes messages afin de retirer du contenu inapproprié.	Suppression avec confirmation, remplacement par "[Message supprimé]", conservation pour audit	1
US73	En tant qu'utilisateur, je souhaite épingler des messages importants afin de les retrouver facilement.	Épinglage (permission requise), limite 10 par channel, possibilité désépinglage	1
US74	En tant que chef de projet, je souhaite voir qui est en ligne dans mon équipe afin de savoir qui je peux solliciter.	Liste membres en ligne, filtrage par projet/équipe, dernière activité	1

ID US	User Story	Description technique	Effort (j)
US75	En tant qu'utilisateur, je souhaite voir qui est en ligne afin de savoir qui est disponible.	Indicateur statut (EN_LIGNE, ABSENT, NE_PAS_DERANGER), modifiable	1
Epic 11 - Gestion des documents			
US76	En tant qu'utilisateur, je souhaite uploader des documents afin de centraliser les fichiers.	Upload drag & drop, association contrat/projet/ticket, métadonnées (titre, description, tags)	2
US77	En tant qu'utilisateur, je souhaite télécharger des documents afin de les consulter hors ligne.	Téléchargement individuel/multiple (ZIP), vérification permissions, enregistrement consultation	1
US78	En tant qu'utilisateur, je souhaite gérer les versions des documents afin d'assurer la traçabilité.	Upload nouvelle version, numérotation auto (v1, v2), commentaire obligatoire	2
US79	En tant qu'utilisateur, je souhaite consulter l'historique des consultations afin de voir qui a accédé au document.	Historique pour traçabilité (qui, quand, durée)	1
US80	En tant qu'utilisateur, je souhaite archiver des documents afin de nettoyer la vue active.	Archivage avec confirmation, conservation données, restauration possible	1
US81	En tant qu'utilisateur, je souhaite être alerté si un fichier identique existe déjà afin d'éviter les doublons.	Calcul hash à l'upload, alerte si doublon, options (remplacer/créer version/annuler)	2
US82	En tant que chef de projet, je souhaite générer automatiquement un PV à partir d'un ticket afin de documenter les interventions.	Génération PDF professionnel (logo, en-tête, description, actions, temps, signature électronique)	3
US83	En tant que client, je souhaite télécharger les PV de mes tickets afin de conserver une trace des interventions.	Accès PV depuis ticket, historique consultable, notification email à génération	1

ID US	User Story	Description technique	Effort (j)
Epic 12 - Notifications et alertes			
US84	En tant qu'utilisateur, je souhaite recevoir des notifications in-app afin d'être informé en temps réel.	Icône avec compteur, dropdown listant notifications, marquage lu/non lu, accès direct élément	2
US85	En tant qu'utilisateur, je souhaite recevoir des notifications par email afin d'être informé hors connexion.	Préférences fréquence (temps réel/digest), templates professionnels	2
US86	En tant que membre technique, je souhaite recevoir des notifications Slack afin d'être alerté sur mon outil de travail.	Configuration Slack par contrat/projet, webhook URL, événements sélectionnables	2
US87	En tant qu'utilisateur, je souhaite recevoir des notifications push navigateur afin d'être alerté même hors onglet.	Demande permission, notifications événements critiques, clic ouvre application	2
US88	En tant qu'utilisateur, je souhaite consulter l'historique de mes notifications afin de retrouver une information.	Liste complète avec filtres (type/statut/période), recherche, pagination	1
Epic 13 - Rapports			
US93	En tant qu'administrateur, je souhaite générer un rapport d'activité mensuel afin de présenter les statistiques.	Rapport (tickets traités, temps passé, respect SLA), export PDF/Excel, graphiques	3
US94	En tant qu'administrateur, je souhaite générer des rapports de conformité SLA afin de justifier les performances.	Rapport taux respect par contrat/période, détail par ticket, calcul pénalités, PDF professionnel	3
US95	En tant que chef de projet, je souhaite générer des rapports d'activité afin de présenter l'avancement.	Rapport par projet/équipe/période (tickets traités, temps passé), graphiques évolution	2
Epic 15 - Versioning avancé			

ID US	User Story	Description technique	Effort (j)
US98	En tant qu'utilisateur, je souhaite créer une nouvelle version d'un document afin de maintenir l'historique.	Upload avec commentaire obligatoire, numérotation auto (v1.0, v1.1, v2.0), conservation versions	2
US99	En tant qu'utilisateur, je souhaite comparer deux versions d'un document afin de voir les différences.	Sélection deux versions, affichage côte à côte, mise en évidence modifications	2
US100	En tant qu'administrateur, je souhaite voir qui a consulté un document afin de suivre sa diffusion.	Liste consultations (date/heure/utilisateur, durée), statistiques	1
Epic 16 - Assistance intelligente			
US101	En tant que client, je souhaite que le système suggère automatiquement une catégorie pour ma demande.	Analyse titre/description, suggestion catégorie, acceptation/modification possible	3
US102	En tant qu'administrateur, je souhaite que le modèle de suggestion s'améliore afin d'offrir des recommandations précises.	Apprentissage des validations/rejets, réentraînement périodique, suivi métriques précision	3
US103	En tant que chef de projet, je souhaite que le système suggère une priorité afin de traiter les demandes efficacement.	Analyse contenu et contexte client, suggestion priorité, ajustable par chef projet	2
US104	En tant que développeur, je souhaite des suggestions de solutions basées sur le passé afin de résoudre plus rapidement.	Identification tickets similaires, suggestion solutions précédentes, acceptation/rejet	3
US105	En tant que développeur, je souhaite évaluer la pertinence des suggestions afin d'améliorer la qualité.	Système notation (utile/pas utile), collecte retours, ajustement modèle	2
US106	En tant qu'administrateur, je souhaite que le système détecte automatiquement les doublons afin d'éviter les demandes redondantes.	Analyse similarité à création, signalement doublons, liaison automatique demandes similaires	2

ID US	User Story	Description technique	Effort (j)
Total Sprint 4 :			64 j

I Analyse

1 Diagramme des cas d'utilisation du Sprint 4

Cette section présentera le diagramme de cas d'utilisation du quatrième sprint, couvrant la messagerie, gestion documentaire, notifications et IA.

[À compléter avec le diagramme de cas d'utilisation]

2 Description textuelle des cas d'utilisation

Cette section détaillera les scénarios principaux et alternatifs pour les cas d'utilisation du Sprint 4.

[À compléter avec les descriptions textuelles]

II Conception

1 Diagrammes de séquence

Cette section présentera les diagrammes de séquence détaillant les interactions pour la messagerie, gestion documentaire et système d'IA.

[À compléter avec les diagrammes de séquence]

2 Modèle de données

Cette section présentera le modèle de données relatif à la messagerie, documents, notifications et modèles d'IA.

[À compléter avec le schéma de base de données]

III Réalisation

1 Architecture technique

Cette section présentera l'architecture technique mise en place pour le Sprint 4, incluant les modules de messagerie temps réel, gestion documentaire et IA.

[À compléter avec les détails d'implémentation]

2 Interfaces utilisateur

Cette section présentera les principales interfaces développées durant le Sprint 4, notamment la messagerie et le système de notifications.

[À compléter avec les captures d'écran]

IV Tests

1 Tests unitaires

Cette section présentera les tests unitaires réalisés pour valider les fonctionnalités du Sprint 4.

[À compléter avec les résultats des tests]

2 Tests d'intégration

Cette section présentera les tests d'intégration effectués pour s'assurer du bon fonctionnement global du système.

[À compléter avec les résultats des tests]